spintent

SPANISH PARSE INTENTS

2.4 2024/09/08*

©2026 by Pablo González L†

CTAN: https://www.ctan.org/pkg/spintent
https://github.com/pablgonz/spintent

Resumen

El paquete spintent proporciona una serie de utilidades para docentes de educación primaria y secundaria que necesiten crean documentos PDF *accesibles* (*tagged* PDF) en español usando LuaLaTeX.

Índice

Motivación y agradecimientos

Desde que desarrollé scontents[7] en 2019 y, más tarde, enumext[8] en 2024, me he ido interesando cada vez más por la accesibilidad en los documentos PDF. Al principio era un tema prácticamente desconocido para mí, pero a medida que fui aprendiendo sobre él comprendí la importancia que tiene, especialmente en el ámbito educativo.

El equipo del proyecto de LaTeX[19] ha realizado un trabajo extraordinario en este campo. Hoy es posible generar documentos etiquetados, validarlos con herramientas como veraPDF[20] o inspeccionar su estructura con ngpdf[21]. Sin embargo, siempre me quedó una duda: cuando un estudiante utiliza NVDA[11] junto con MathCAT[12], ¿realmente escucha lo que el o la docente quiso expresar?

Basta con generar un documento accesible siguiendo las recomendaciones de The LaTeX Tagged PDF Project y escucharlo con NVDA/MathCAT para darse cuenta de que, en muchos casos, la *verbalización* no coincide con lo que uno espera. Esto no resulta extraño: las reglas del español, y en particular el lenguaje matemático, dependen en gran medida del contexto en que se utilizan.

El objetivo de spintent nace precisamente de esta necesidad: proporcionar herramientas para que el o la docente pueda indicar explícitamente el contexto de determinadas expresiones matemáticas desde el propio código fuente de LaTeX, de modo que el documento PDF resultante no solo sea *accesible*, sino que también produzca una *verbalización más natural* y adecuada para el estudiante al utilizar NVDA/MathCAT

El nuevo estándar PDF 2.0[14, 18, 16], las especificaciones de WTPDF[15] y la guía más reciente de la PDF Association, «Best Practice Guide: Math in PDF»[17], coinciden en que el camino hacia una accesibilidad matemática de calidad pasa por el uso de MathML. En ese contexto, LuaLaTeX ofrece una plataforma especialmente adecuada para incorporar esta tecnología.

*Este archivo describe la documentación para la versión 2.4 revisada por última vez el 2024/09/08.

†E-mail: «pablgonz@educarchile.cl».

Licencia y requerimientos

El paquete spintent carga y requiere el paquete unicode-math[1] y que se ejecute bajo LuaLaTeX por lo que es necesario contar con una distribución moderna de TeX como TeX Live 2026. Ha sido probado con las clases estándar proporcionadas por LaTeX: book, report, article y letter en tamaños 10pt, 11pt y 12pt.

Se otorga permiso para copiar, distribuir y/o modificar este paquete bajo los términos de LaTeX Project Public License (LPPL), versión 1.3 o posterior (https://www.latex-project.org/lppl.txt). El paquete tiene la condición de mantenido al estilo «*A Work in Progress*[1]».

El requerimiento mínimo es LaTeX release 2026-06-01.

Este paquete solo se puede utilizar solo con LuaLaTeX y *tagged* PDF activo. Está presente en TeX Live y MiKTeX, para instalarlo, utilice el gestor de paquetes de su distribución. Si desea una instalación manual, descargue spintent.zip y descomprímalo, luego ejecute `luatex spintent.ins`, mueva todos los archivos a las ubicaciones correspondientes y luego ejecute `mktexlsr`. Para generar la documentación, ejecute `arara spintent.dtx`.

```
spintent.sty  >  TDS:tex/lualatex/spintent/
spintent.lua  >  TDS:tex/lualatex/spintent/
spintent.pdf  >  TDS:doc/lualatex/spintent/
README.md     >  TDS:doc/lualatex/spintent/
spintent.dtx  >  TDS:source/lualatex/spintent/
spintent.ins  >  TDS:source/lualatex/spintent/
```

1 Descripción

El nombre spintent puede entenderse como «*Spanish Parse Intent*» o también como «*School Parse Intent*». Ambas interpretaciones reflejan el objetivo del paquete y el público para el que fue diseñado: docentes de educación primaria y secundaria.
Está pensado para crear «*apuntes*», «*hojas de ejercicios*», «*clases escritas*» enfocadas en el estudiante. Solo un docente sabe cual es el nivel de aprendizaje del estudiante. El paquete intenta sobrescribir muchas de las reglas dictadas por la heurística de NVDA/MathCAT teniendo esto presente.

El paquete spintent usa y abusa del atributo :intent de MathML por medio del comando \MathMLintent definido en latex-lab-mathintent[30], un encapsulado de \luamml_attribute:een proveída por luamml[2]. ¡Gracias Marcel¡

1.1 Carga del paquete

El paquete spintent trabaja solo con LuaLaTeX y *tagged* PDF activo usando tagging=on dentro del comando \DocumentMetadata al inicio del archivo. Las llaves (⟨*keys*⟩) aceptadas por \DocumentMetadata y en general para el código de *tagged* PDF están documentadas en los paquetes y tagpdf[29] documentmetadata-support[30], es deber del usuario revisar la documentación asociada a ellos.

```
\DocumentMetadata
  {
    lang=es-CL,
    pdfversion=2.0,
    pdfstandard=ua-2,
    tagging=on,
    tagging-setup={math/setup={mathml-SE}},
  }
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage{spintent}
```

1.2 El paquete babel

El paquete babel-spanish[10] está marcado como «parcialmente compatible» para *tagged* PDF, pero, es absolutamente necesario para la creación de documentos en español. En caso de encontrar problemas de compatibilidad con otros paquetes se puede desactivar usando:

```
\usepackage[spanish,shorthands=off]{babel}
```

Muchas de las «shorthands» proveidas por babel-spanish hoy no son necesarias, más allá de \sptext para las abreviaturas y ordinales el resto son solo atajos de teclas para escritura que hoy en día no se ocupan. El paquete spintent provee el comando \spshort (§3.4) que cubre estos casos, de esta manera podemos cargar babel[9] con todas sus funcionalidades sin problemas.

[1] *A Work in Progress* es un álbum en vivo de Neil Peart (∗ 1952–† 2020), baterista y principal letrista de Rush. El nombre no pretende formar parte de la terminología de la LPPL; simplemente describe el estado permanente de este paquete mientras sigo aprendiendo sobre *accesibilidad* en PDF.

1.3 Fuentes tipográficas

Como spintent es un paquete LuaLaTeX debemos distinguir entre las fuentes tipográficas `OpenType/TrueType` versus las fuentes de `8bit` estándar usadas por LaTeX para el *modo texto* y el *modo matemático*. Para evitar problemas con *tagged* `PDF` preferiremos no combinar fuentes `OpenType/TrueType` con fuentes `8bit` y trabajaremos solo con fuentes `OpenType/TrueType` que tengan soporte para matemáticas.

La carga de las fuentes tipográficas para LuaLaTeX se maneja por medio del paquete `fontspec`[3], si no se espcifica ninguna se usarán las fuentes `Latin Modern` para el *modo texto* y `Latin Modern Math` para el *modo matemático*.

El tipo de fuente tipográfica escogida es preferencia de cada usuario, en lo personal utilizo las fuente `Libertinus` para el *modo texto* por medio del paquete `libertinus-otf`[4] y la fuente `Erewhon-Math` para el modo matemático cargada por el paquete `fourier-otf`[5]. Adicionalmente cargo la fuente `Source Code Pro` para el texto tipo «`máquina de escribir`» por medio del paquete `sourcecodepro`[6].

```
% \DocumentMetadata{...}
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\usepackage{spintent}
```

1.4 El paquete `hyperref`

El paquete `hyperref`[23] es *«casi obligatorio»* cuando se crean documentos *tagged* `PDF`, se encarga de las referencias cruzadas `\label` y `\ref`, las notas al pie `\footnote`, los hipervínculos `\href` y muchos otros elementos como marcadores, índices y metadatos.

```
\DocumentMetadata
  {
    lang=es-CL,
    pdfversion=2.0,
    pdfstandard=ua-2,
    tagging=on,
    tagging-setup={math/setup={mathml-SE}},
  }
\documentclass{article}
\usepackage[spanish]{babel}
\usepackage[osf,nomath,mono=false]{libertinus-otf}
\usepackage[osf,semibold]{sourcecodepro}
\usepackage[no-text]{fourier-otf}
\setmathfont{Erewhon-Math}[range={cal,bfcal},StylisticSet=1]
\usepackage{spintent, hyperref}
\hypersetup
  {
    colorlinks         = true,
    pdfstartview       = FitH,
    pdfdisplaydoctitle = true,
    pdftitle           = {Test para el paquete spintent},
    pdfinfo            =
      {
        Title   = {Test},
        Subject = {spintent},
      },
  }
```

2 El comando \setspintent

\setspintent `\setspintent{⟨comando⟩}{⟨key-uno = valor, key-dos = valor, key-tres = valor, ...⟩}`

El comando \setspintent establece las llaves (⟨keys⟩) de forma global para los comandos provistos por spintent. Puede utilizarse tanto en el preámbulo como en el cuerpo del documento tantas veces como se desee.

Las llaves (⟨keys⟩) definidas en el *argumento opcional* [⟨key = val⟩] de los comandos tienen prioridad sobre las establecidas por este, anulando las configuraciones globales de \setspintent.

Muchas de las utilidades que provee el paquete spintent implementan el sistema [⟨key = val⟩] y están pensadas para ser utilizadas en la generación de *«hojas de ejercicios»* o *«clases escritas»*. Se recomienda usar \setspintent y establecerlas de manera global para cada documento, para mantener la coherencia en la verbalización de NVDA/MathCAT a lo largo del mismo.

El sistema [⟨key = val⟩] utilizado por spintent se implementa usando el módulo l3keys de expl3[24]. Por diseño, las llaves que reciben un *valor* después del signo '=' NO aceptan *valores vacíos* y es obligatorio pasar uno valido; las llaves documentadas como ⟨valor prohibido⟩ no reciben *ningun valor*. En caso de omitir un *valor* valido o pasar un *valor vacío* a una llave que acepte *valor* o pasar un *valor* a una llave del tipo ⟨valor prohibido⟩ se devolverá un "error" y se detendrá la compilación del documento.

3 The big four

El paquete spintent provee cuatro grandes comandos *(«the big four[2]»)*: \spnum para el manejo de números, \spunit para el manejo de unidades, \spmoney para el manejo de dinero y \spshort para el manejo de abreviaturas, acrónimos y números ordinales.

Tambien se incluyen las utilidades \spnewunit y \spunitalias asociadas a al comando \spunit.

3.1 El comando \spnum

\spnum

`\spnum{⟨número⟩}`
`\spnum[⟨key = val⟩]{⟨+número⟩}`
`\spnum[⟨key = val⟩]{⟨-número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número⟩}`
`\spnum[⟨key = val⟩]{⟨número.número⟩}`
`\spnum[⟨key = val⟩]{⟨número,número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número.número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número;número⟩}`
`\spnum[⟨key = val⟩]{⟨número e exponente⟩}`
`\spnum[⟨key = val⟩]{⟨número E exponente⟩}`

El comando \spnum trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨número⟩}, donde la *«parte entera»* puede iniciar con los signos '+' o '−', la *«parte decimal finita»* **debe** iniciar con una *coma* ',' o con un *punto* '.', y la *«parte decimal infinita (periódica)»* **debe** iniciar con un *punto y coma* ';'.

Si no se proporciona el *argumento opcional* [⟨key = val⟩] y se ejecuta `$\spnum{+23}$` se imprimirá '+23' en la salida PDF y NVDA/MathCAT verbalizará *«más veintitrés»*, si se ejecuta `$\spnum{3,14}$` se imprimirá '3,14' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma catorce»*, si se ejecuta `$\spnum{3,1;4}$` se imprimirá '$3{,}1\overline{4}$' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma uno con cuatro periódico»*.

Además, el *argumento obligatorio* {⟨número⟩} admite al final de este un sufijo de *«notación científica»*. Este se indica mediante las letras 'e' o 'E', acompañadas de los signos opcionales '+' o '−' seguido por los dígitos del exponente. Por ejemplo `$\spnum{3,14e3}$` imprimirá '$3{,}14 \cdot 10^{3}$' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma catorce por diez al cubo»*.

La *coma* ',' o el *punto* '.' son *separadores decimales* y NO de millares, por lo tanto solo pueden aparecer una «única vez» dentro del {⟨argumento⟩}. El {⟨argumento⟩} puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como \textstyle, \mathrm{#1} o \mathbf{#1} por ejemplo. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.1.1 Llaves para \spnum

`cifras-sep` = {⟨*true* | *false*⟩} default: *true*

Establece el *espaciado* utilizado para agrupar los dígitos en bloques de tres, tanto en la parte entera como en la parte decimal finita. Si se establece en `true` se siguen las normas de la RAE y se utiliza un *espacio fino* '\,' entre las cifras; si se establece en `false`, se respetan los espacios matemáticos introducidos en el argumento. Por ejemplo `$\spnum{100000}$` imprimirá '100 000' en la salida PDF y `$\spnum[cifras-sep=false]{10\,00\,00}$` imprimirá '10 00 00' en la salida PDF.

`read-sign` = {⟨*terse* | *clear*⟩} default: *terse*

Establece la *forma* en la que NVDA/MathCAT verbalizará los signos '+' o '−'. Si se establece en `terse`, NVDA/MathCAT verbalizará *«más»* o *«menos»* **antes** de leer el número; si se establece en `clear`, NVDA/MathCAT verbalizará *«positivo»* o *«negativo»* **después** de leer el número. Por ejemplo `$\spnum{-23}$` imprimirá '−23' en la salida PDF y NVDA/MathCAT verbalizará *«menos veintitrés»* y `$\spnum[read-sign=clear]{-23}$` imprimirá '−23' en la salida PDF y NVDA/MathCAT verbalizará *«veintitrés negativo»*.

`read-period-sep` = {⟨*coma* | *punto*⟩} default: *coma*

[2] Los *«cuatro grandes»* del Thrash Metal: Metallica, Megadeth, Slayer y Anthrax, que me acompañan mientras desarrollo spintent.

Establece la *forma* en la que NVDA/MathCAT verbalizará e imprimirá el signo de separación '**;**' de la *«parte decimal infinita (periódica)»* cuando no se tiene la *«parte decimal finita»*. Si se establece en coma, se imprimirá '**,**' en la salida PDF y NVDA/MathCAT verbalizará *«coma»*; si se establece en punto, se imprimirá '**.**' en la salida PDF y NVDA/MathCAT verbalizará *«punto»*. Por ejemplo $\spnum{3;4}$ imprimirá '3,$\overline{4}$' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma cuatro periódico»* y $\spnum[read-period-sep=punto]{3;4}$ imprimirá '3.$\overline{4}$' en la salida PDF y NVDA/MathCAT verbalizará *«tres punto cuatro periódico»*

Esta llave es necesaria SOLO cuando se utilizan *«decimales periódicos puros»*, es decir, cuando no se ha incluido en el {⟨*argumento*⟩} una *coma* '**,**' o un *punto* '**.**' antes del signo de separación '**;**' periódica.

notation-sym = {⟨*times* | *cdot*⟩} default: *cdot*

Establece el *símbolo* que se utilizará para imprimir la *«notación científica»* al usar '**e**' o '**E**'. Si se establece en times imprimirá '**×**' en la salida PDF y NVDA/MathCAT verbalizará *«por»*; si se establece en cdot imprimirá '**·**' en la salida PDF y NVDA/MathCAT verbalizará *«por»*. Por ejemplo $\spnum{3,14e3}$ imprimirá '$3{,}14 \cdot 10^3$' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma catorce por diez al cubo»* y $\spnum[notation-sym=times]{3,14e3}$ imprimirá '$3{,}14 \times 10^3$' en la salida PDF y NVDA/MathCAT verbalizará *«tres coma catorce por diez al cubo»*.

Decir *«notación científica»* aquí es un poco exagerado, en realidad es *«multiplicación por potenicas de* 10*»*, no se verfica si la entrada cumple o no con la condición de estár escrita en *«notación científica»*.

3.2 El comando \spunit

\spunit

\spunit{⟨*unidad*⟩}
\spunit[⟨*key* = *val*⟩]{⟨*unidad*⟩}
\spunit[⟨*key* = *val*⟩]{⟨*número unidad*⋆*unidad*⟩}
\spunit[⟨*key* = *val*⟩]{⟨*número unidad*⋆*unidad*/*unidad*⟩}
\spunit[⟨*key* = *val*⟩]{⟨*número unidad*⋆*unidad*/*unidad*⋆*unidad*⟩}

El comando \spunit trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*unidad*⟩}, el cual puede contener una *«parte numérica»* opcional la cual se procesa internamente por el comando \spnum al inicio del {⟨*argumento*⟩} seguido de una {⟨*unidad*⟩} o un {⟨*alias*⟩} valido descritos en §3.2.2 o una {⟨*unidad*⟩} creada por el comando \spnewunit (§3.2.3) o un {⟨*alias*⟩} creado por el comando \spunitalias (§3.2.4).

Si no se proporciona el *argumento opcional* [⟨*key* = *val*⟩] y se ejecuta $\spunit{23 m}$ se imprimirá '23 m' en la salida PDF y NVDA/MathCAT verbalizará *«veintitrés metros»*.

La multiplicación entre las {⟨*unidades*⟩} se realiza mediante un *asterisco* '*' y las potencias de estas se indican con un *circunflejo seguido del exponente entre llaves* '^{⟨*exponente*⟩}'. Por ejemplo si se ejecuta $\spunit{23 m*s^{2}}$ se imprimirá '23 m s^2' en la salida PDF y NVDA/MathCAT verbalizará *«veintitrés metros segundos al cuadrado»*.

Se pueden usar *fracciones lineales* con las {⟨*unidades*⟩} utilizando una *«única barra»* '/' para la separación entre las {⟨*unidades*⟩} del numerador y el denominador, teniendo presente que el denominador NO puede contener números de ningún tipo. Por ejemplo si se ejecuta $\spunit{23 m/s^{2}}$ se imprimirá '23 m/s^2' en la salida PDF y NVDA/MathCAT verbalizará *«veintitrés metros por segundos al cuadrado»*.

Solo imprimiremos *fracciones lineales*, si se requiere otra forma fraccionaria para explicaciones y otros usos se debe usar el comando por separado. El {⟨*argumento*⟩} puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como \textstyle, \mathrm{#1} o \mathbf{#1} por ejemplo, NO puede contener más de una *«única barra»* '/' y NO puede contener números en el denominador. Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.2.1 Llaves para \spunit

El comando \spunit posee las ⟨*meta-keys*⟩ cifras-sep, read-period-sep y notation-sym pasadas al comando \spnum (§3.1.1).

print-cdot = {⟨*true* | *false*⟩} default: *false*

Establece si se *imprime* \cdot '·' o un *espacio fino* '\,' para la multiplicación de {⟨*unidades*⟩}. Si se establece en true, se imprimirá '·' entre las {⟨*unidades*⟩} en la salida PDF; si se establece en false se imprimirá un *espacio fino* '\,' entre las {⟨*unidades*⟩} en la salida PDF.

read-cdot = {⟨*true* | *false*⟩} default: *false*

Establece la *forma* en la que NVDA/MathCAT verbalizará la multiplicación de {⟨*unidades*⟩}. Si se establece en true, NVDA/MathCAT verbalizará «por» entre las {⟨*unidades*⟩}; si se establece en false, NVDA/MathCAT no verbalizará nada entre las {⟨*unidades*⟩}.

read-slash = {⟨*por* | *partido-por*⟩} default: *por*

Establece la *forma* en la que NVDA/MathCAT verbalizará la *«barra»* '/' de separación entre el numerador y el denominador. Si se establece en por, NVDA/MathCAT verbalizará *«por»*; si se establece en partido-por, NVDA/MathCAT verbalizará *«partido por»*.

3.2.2 Argumentos para \spunit

Resumen de {⟨*unidades*⟩} y {⟨*alias*⟩} soportados por \spunit.

©2026 by Pablo González L

Entrada	Alias soportados	Nombre unidad	Salida	NVDA/MathCAT
m	metro, metros	Metro	m	*«metro»*
g	gramo, gramos	Gramo	g	*«gramo»*
s	segundo, segundos	Segundo (Tiempo)	s	*«segundo (Tiempo)»*
l	litro, l-litro	Litro (minúscula)	l	*«litro (minúscula)»*
h	hora, horas	Hora	h	*«hora»*
d	dia, dias	Día	d	*«día»*
A	amperio, ampere	Amperio	A	*«amperio»*
V	voltio, volt	Voltio	V	*«voltio»*
W	vatio, watt	Vatio	W	*«vatio»*
J	julio, joule	Julio	J	*«julio»*
N	newton	Newton	N	*«newton»*
Pa	pascal	Pascal	Pa	*«pascal»*
Hz	hercio, hertz	Hercio	Hz	*«hercio»*
Ω	ohmio, ohm, Omega	Ohmio	Ω	*«Ohmio»*
F	faradio	Faradio	F	*«Faradio»*
C	culombio	Culombio	C	*«Culombio»*
K	kelvin, Kelvin	Kelvin	K	*«Kelvin»*
°C	celsius, degC, ℃	Grados Celsius	°C	*«Grados Celsius»*
°F	fahrenheit, degF, ℉	Grados Fahrenheit	°F	*«Grados Fahrenheit»*
cal	caloria, calorias	Caloría	cal	*«Caloría»*
b	bit	Bit	b	*«Bit»*
B	byte, bytes	Byte	B	*«Byte»*
in	pulgada, pulgadas	Pulgada	in	*«Pulgada»*
ft	pie, pies	Pie	ft	*«Pie»*
yd	yarda, yardas	Yarda	yd	*«Yarda»*
mi	milla, millas	Milla	mi	*«Milla»*
lb	libra, libras	Libra	lb	*«Libra»*
oz	onza, onzas	Onza	oz	*«Onza»*
gal	galon, galones	Galón	gal	*«Galón»*
Å	angstrom	Angstrom	Å	*«Angstrom»*
µ	micro	Micro (Prefijo)	µ	*«Micro (Prefijo)»*
℧	mho	Mho / Siemens	℧	*«Mho / Siemens»*
′	minmin	Minuto (Arco)	′	*«Minuto (Arco)»*
″	segseg	Segundo (Arco)	″	*«Segundo (Arco)»*
u	u-masa	Unidad masa atómica	u	*«Unidad masa atómica»*
		Unidades directas (sin alias):		
L	-	Litro (Mayúscula)	L	*«Litro (Mayúscula)»*
ℓ	-	Litro (Cursiva)	ℓ	*«Litro (Cursiva)»*
w	-	Vatio (Minúscula)	w	*«Vatio (Minúscula)»*
°K	-	Grado Kelvin	°K	*«Grado Kelvin»*
Bq	-	Becquerel	Bq	*«Becquerel»*
Da	-	Dalton	Da	*«Dalton»*
Gy	-	Gray	Gy	*«Gray»*
S	-	Siemens	S	*«Siemens»*
Sv	-	Sievert	Sv	*«Sievert»*
T	-	Tesla	T	*«Tesla»*
Wb	-	Weber	Wb	*«Weber»*
atm	-	Atmósfera	atm	*«Atmósfera»*
au	-	Unidad astronómica	au	*«Unidad astronómica»*
bar	-	Bar	bar	*«Bar»*
cd	-	Candela	cd	*«Candela»*
dB	-	Decibelio	dB	*«Decibelio»*
dyn	-	Dina	dyn	*«Dina»*
eV	-	Electronvoltio	eV	*«Electronvoltio»*
erg	-	Ergio	erg	*«Ergio»*
ha	-	Hectárea	ha	*«Hectárea»*
kat	-	Katal	kat	*«Katal»*
kg	-	Kilogramo	kg	*«Kilogramo»*
km	-	Kilómetro	km	*«Kilómetro»*
lm	-	Lumen	lm	*«Lumen»*

Continúa en la siguiente página...

©2026 by Pablo González L

Continuación de la tabla 1

Entrada	Alias soportados	Nombre unidad	Salida	MathCAT
lx	-	Lux	lx	*«Lux»*
cm	-	Centímetro	cm	*«Centímetro»*
min	-	Minuto (Tiempo)	min	*«Minuto (Tiempo)»*
mm	-	Milímetro	mm	*«Milímetro»*
mol	-	Mol	mol	*«Mol»*
pc	-	Pársec	pc	*«Pársec»*
rad	-	Radián	rad	*«Radián»*
rpm	-	Rev. por minuto	rpm	*«Rev. por minuto»*
sr	-	Estereorradián	sr	*«Estereorradián»*
t	-	Tonelada	t	*«Tonelada»*
a	-	Año / Área (are)	a	*«Año / Área (are)»*
y	-	Año (Year)	y	*«Año (Year)»*
°	-	Grado (Arco)	°	*«Grado (Arco)»*

3.2.3 El comando `\spnewunit`

`\spnewunit` `\spnewunit{⟨nueva unidad⟩}{⟨verbalización NVDA⟩}`

El comando `\spnewunit` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: {⟨*nueva unidad*⟩} que establece lo que se imprimirá en la salida PDF y {⟨*verbalización* NVDA⟩} que establece como NVDA/MathCAT *verbalizará* la {⟨*nueva unidad*⟩}.

La {⟨*nueva unidad*⟩} no debe estar registrada como {⟨*unidad*⟩} o {⟨*alias*⟩} para `\spunit` descritos en §3.2.2. Si la {⟨*nueva unidad*⟩} ya está registrada se devolverá un "error" y se detendrá la compilación del documento.

La declaración de la {⟨*nueva unidad*⟩} es «global» y se almacena en el módulo `spintent.lua` como una {⟨*unidad*⟩} valida para pasar como {⟨*argumento*⟩} a `\spunit`. Por ejemplo: `\spnewunit{px}{píxeles}` creará la {⟨*nueva unidad*⟩} 'px' y cuando se utilice `\spunit{100 px}` se imprimirá '100 px' en la salida PDF y NVDA/MathCAT verbalizará *«cien píxeles»*.

3.2.4 El comando `\spunitalias`

`\spunitalias` `\spunitalias{⟨nuevo alias⟩}{⟨expresión de unidades⟩}`

El comando `\spunitalias` trabaja en *modo texto* y recibe dos *argumentos obligatorios*: {⟨*nuevo alias*⟩} que establece es el nuevo {⟨*alias*⟩} de {⟨*argumento*⟩} para `\spunit` y {⟨*expresión de unidades*⟩} que puede contener multiplicación o división de {⟨*unidades*⟩} ya registradas que se imprimirán en la salida PDF.

El {⟨*nuevo alias*⟩} no debe estar registrado como {⟨*unidad*⟩} o {⟨*alias*⟩} para `\spunit` descritos en §3.2.2 o estar definido como {⟨*unidad*⟩} por medio del comando `\spnewunit`. Si el {⟨*nuevo alias*⟩} ya está registrado se devolverá un "error" y se detendrá la compilación del documento.

La declaración de un {⟨*nuevo alias*⟩} es «global» y se almacena en el módulo `spintent.lua` como una {⟨*alias*⟩} valido para pasar como {⟨*argumento*⟩} a `\spunit`. Por ejemplo: `\spunitalias{caudal}{m^{3}/s}` creará el {⟨*alias*⟩} 'caudal' y cuando se utilice `$\spunit{50 caudal}$` se imprimirá '50 m^3/s' en la salida PDF y NVDA/MathCAT verbalizará *«cincuenta metros al cubo por segundos»*.

3.3 El comando `\spmoney`

`\spmoney` `\spmoney{⟨cifra⟩}`
`\spmoney{⟨cifra moneda⟩}`
`\spmoney[⟨key = val⟩]{⟨cifra moneda⟩}`

El comando `\spmoney` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*cifra*⟩} que contiene la *«cifra numérica»* positiva o negativa la se cual procesa internamente por el comando `\spnum` al inicio del {⟨*argumento*⟩} acompañada de un *«símbolo de moneda»* o *«alias de moneda»* opcional descritos en §3.3.2.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] y se ejecuta `$\spmoney{500}$` se imprimirá '$500' en la salida PDF y NVDA/MathCAT verbalizará *«quinientos pesos»*.

El comportamiento de `\spmoney` depende de como se pase el *argumento obligatorio* que tiene prioridad por sobre los valores por defecto. Por ejemplo: `$\spmoney{500 dolares}$` imprimirá '$500' en la salida PDF y NVDA/MathCAT verbalizará *«quinientos dolares»* aunque la moneda por defecto sean pesos.

Si se pasa un *«alias de moneda»* del tipo ISO (código de divisas) como CLP o USD (mayúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país y se ajusta la salida PDF al estandar ISO. Por ejemplo: `$\spmoney{500 CLP}$` imprimirá 'CLP500' en la salida PDF y NVDA/MathCAT verbalizará *«quinientos pesos chilenos»*.

Si se pasa un *«alias de moneda»* del tipo ISO (código de divisas) como clp o usd (minúsculas) se modifica la verbalización en NVDA/MathCAT agregando el país, pero, NO se modifica la salida PDF imprimiendo el

«símbolo de moneda» correspondiente. Por ejemplo: `$\spmoney{500 clp}$` imprimirá '$500' en la salida PDF y NVDA/MathCAT verbalizará *«quinientos pesos chilenos»*

Pasar de manera directa en el *argumento obligatorio* un *«símbolo de moneda»* como '$', '€', '£' o un *«alias de moneda»* como `peso`, `euro` o `libra` produce la salida PDF y verbalización estándar esperada para NVDA/MathCAT.

- La posición del *«símbolo de moneda»* a la izquierda o a la derecha de la *«cifra numérica»* y la verbalización para NVDA/MathCAT se manejan desde el módulo Lua `spintent.lua` y no se proveen llaves para modificar este comportamiento.
- El *«símbolo de moneda»* es tomado de la fuente tipográfica definida para el *modo matemático* al momento de ejecutar el comando, se debe tener presente en caso de querer usar un *«símbolo de moneda»* poco habitual que esté soportada por spintent. El {⟨*argumento*⟩} puede contener espacios matemáticos validos, pero, NO puede contener comandos con o sin argumentos como `\textstyle`, `\mathrm{#1}` o `\mathbf{#1}` por ejemplo. La *coma* '`,`' o el *punto* '`.`' como *separador decimal* solo pueden aparecer una «única vez». Cualquier entrada fuera de esta sintaxis devolverá un "error" deteniendo la compilación del documento.

3.3.1 Llaves para `\spmoney`

El comando `\spmoney` posee la ⟨*meta-key*⟩ `cifras-sep` pasada al comando `\spnum` (§3.1.1).

`currency` = {⟨*moneda* | *alias*⟩} default: *peso*

Establece el *símbolo de moneda* por defecto que se imprimirá en la salida PDF y cómo lo verbalizará NVDA/MathCAT cuando SOLO se pasa una *«cifra numérica»* como {⟨*argumento*⟩} al comando `\spmoney`. El *«símbolo de moneda»* o el *«alias de moneda»* pasado a esta llave debe ser alguno de los descritos en §3.3.2.

`sub-units` = {⟨*true* | *false*⟩} default: *false*

Establece si se usarán *subunidades* de la moneda en curso para la verbalización en NVDA/MathCAT. La moneda en curso debe aceptar *subunidades*. Por ejemplo: `$\spmoney[sub-units=true]{100,5 dolares}$` imprimirá '$100,5' en la salida PDF y NVDA/MathCAT verbalizará *«cien dólares con cincuenta centavos»*.

`dolar-math` = {⟨*true* | *false*⟩} default: *true*

Establece si el *símbolo* '`$`' se imprimirá en *modo texto* o en *modo matemático*. Si se establece en `true`, se usará la fuente tipográfica definida para el *modo matemático* en curso para imprimir '`$`' en la salida PDF; si se establece en `false`, se usará la la fuente tipográfica definida para el *modo texto* para imprimir '`$`' en la salida PDF.

3.3.2 Monedas y alias para `\spmoney`

Resumen de los {⟨*símbolos de moneda*⟩} y {⟨*alias de moneda*⟩} soportados como {⟨*argumento*⟩} para `\spmoney`.

3.4 El comando `\spshort`

`\spshort`

`\spshort`{⟨*abreviaturas* | *acrónimos* | *números ordinales*⟩}
`\spshort`[⟨*key* = *val*⟩]{⟨*abreviaturas* | *acrónimos* | *números ordinales*⟩}
`\spshort`[⟨*read-arg*={⟨*verbalización del argumento para* NVDA⟩}⟩]{⟨*comandos LaTeX*⟩}

El comando `\spshort` trabaja *solo en modo texto* y recibe *un argumento obligatorio* del tipo {⟨*abreviaturas*⟩}, {⟨*acrónimos*⟩} o {⟨*números ordinales*⟩} descritos en §3.4.2.

Si no se proporciona el *argumento opcional* [⟨*key* = *val*⟩] y se ejecuta `\spshort{dr.a}` se imprimirá 'Dr.ª' en la salida PDF y NVDA/MathCAT verbalizará *«doctora»*; si se ejecuta `\spshort{onu}` se imprimirá 'ONU' en la salida PDF y NVDA/MathCAT verbalizará *«organización de las naciones unidas»* y si se ejecuta `\spshort{mineduc}` se imprimirá 'Mineduc' en la salida PDF y NVDA/MathCAT verbalizará *«ministerio de educación»*.

Para la escritura de {⟨*números ordinales*⟩} se puede ejecutar `\spshort{1.a}` que imprimirá '1.ª' en la salida PDF y NVDA/MathCAT verbalizará *«primera»*; si se ejecuta `\spshort{1.o}` imprimirá '1.º' en la salida PDF y NVDA/MathCAT verbalizará *«primero»*.

Para crear un nueva {⟨*abreviatura*⟩}, {⟨*acrónimo*⟩} o {⟨*número ordinal*⟩} que no se encuentre registrada por spintent dentro del módulo Lua `spintent.lua` descritas en §3.4.2 se debe utilizar la llave `read-arg` y definir esta dentro del *argumento obligatorio* que puede contener {⟨*comandos LaTeX*⟩} *en modo texto* los cuales se imprimirán en salida PDF.

- Para la escritura de {⟨*números ordinales*⟩} se debe tener cuidado y no confundir la *«letra o voladita»* 'º' con el símbolo de grado '°'. Si la fuente tipográfica del *modo texto* está utilizando `Numbers=OldStyle` los números pueden quedar muy separados de la *«letra voladita»*, la solución para esto es agregar `\addfontfeatures{Numbers=Lining}` dentro de un grupo junto al comando: `{\addfontfeatures{Numbers=Lining}\spshort{1.o}}`.
- El comando `\spshort` está pensado como un reemplazo para *shorthands* y el comando `\sptext` provistos por el paquete babel en español compatible con *tagged* PDF. Es de las pocas utilidades que provee el paquete spintent que trabaja *solo en modo texto*. Internamente utiliza la llave `E` *(expansion text)* proveída por el paquete tagpdf[29].

Entrada	Alias	Nombre	Salida	NVDA/MathCAT
$	peso, pesos	peso	$	*«peso / pesos»*
€	euro, euros	euro	€	*«euro / euros»*
£	libra, libras	libra	£	*«libra / libras»*
¥	yen, yenes	yen	¥	*«yen / yenes»*
¥	yuan, yuanes	yuan	¥	*«yuan / yuanes»*
₩	won, wons	won	₩	*«won / wons»*
₪	shekel, shekels, sequel, sequeles	séquel	₪	*«séquel / séqueles»*
₹	rupia, rupias	rupia	₹	*«rupia / rupias»*
₽	rublo, rublos	rublo	₽	*«rublo / rublos»*
₺	lira, liras	lira	₺	*«lira / liras»*
₴	grivna, grivnas	grivna	₴	*«grivna / grivnas»*
¢	centavo, centavos	centavo	¢	*«centavo / centavos»*
		Divisas en formato ISO (Mayúsculas)		
CLP	—	peso chileno	CLP	*«peso chileno / pesos chilenos»*
MXN	—	peso mexicano	MXN	*«peso mexicano / pesos mexicanos»*
USD	—	dólar	USD	*«dólar estadounidense / dólares estadounidenses»*
EUR	—	euro	EUR	*«euro / euros»*
GBP	—	libra	GBP	*«libra / libras»*
JPY	—	yen	JPY	*«yen / yenes»*
CNY	—	yuan	CNY	*«yuan / yuanes»*
KRW	—	won	KRW	*«won / wons»*
ILS	—	séquel	ILS	*«séquel / séqueles»*
INR	—	rupia	INR	*«rupia / rupias»*
RUB	—	rublo	RUB	*«rublo / rublos»*
TRY	—	lira	TRY	*«lira / liras»*
UAH	—	grivna	UAH	*«grivna / grivnas»*
		Códigos ISO en minúsculas		
clp	—	peso chileno	$	*«peso chileno / pesos chilenos»*
mxn	—	peso mexicano	$	*«peso mexicano / pesos mexicanos»*
usd	dolar, dolares	dólar	$	*«dólar estadounidense / dólares estadounidenses»*
eur	—	euro	€	*«euro / euros»*
gbp	—	libra	£	*«libra / libras»*
jpy	—	yen	¥	*«yen / yenes»*
cny	—	yuan	¥	*«yuan / yuanes»*
krw	—	won	₩	*«won / wons»*
ils	—	séquel	₪	*«séquel / séqueles»*
inr	—	rupia	₹	*«rupia / rupias»*
rub	—	rublo	₽	*«rublo / rublos»*
try	—	lira	₺	*«lira / liras»*
uah	—	grivna	₴	*«grivna / grivnas»*

©2026 by Pablo González L

3.4.1 Llaves para \spshort

`read-arg` = {⟨*verbalización* NVDA⟩} default: *no usado*

Establece la *verbalización del argumento* {⟨*comandos* LaTeX⟩} de la nueva {⟨*abreviatura*⟩}, {⟨*acrónimo*⟩} o {⟨*número ordinal*⟩} para NVDA. El valor de esta llave *siempre* debe pasarse entre '{ }'. Por ejemplo: `\spshort[read-arg={octavo}]{8.\textsuperscript{\emph{avo}}}` creará una nueva {⟨*abreviatura*⟩} que imprimirá '8.^{avo}' en la salida PDF y NVDA verbalizará *«octavo»*.

`small-caps` = {⟨*true* | *false*⟩} default: *false*

Establece si se usarán *versalitas* para imprimir {⟨*acrónimos*⟩} y para la *«letra volada»* en {⟨*abreviaturas*⟩} y {⟨*números ordinales*⟩} respetando las reglas de la RAE para el uso de estas. Si se establece en `true`, las abreviaturas con mayúscula como `EE. UU.` y los acrónimos de cinco o más letras como `Unicef` no se verán afectados; si se establece en `false`, no se aplicarán *versalitas*. Por ejemplo: `\spshort[small-caps=true]{dr.a}` imprimirá 'Dr.^{A}' en la salida PDF y NVDA verbalizará *«doctora»*.

3.4.2 Argumentos para \spshort

El cuadro 2 resume las abreviaturas, expresiones latinas y títulos registrados en el módulo Lua `spintent.lua`.

Cuadro 2: Abreviaturas soportadas

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Abreviaturas con letras voladas			
`dr.a`	`dr.ª`	Dr.ª	*«doctora»*
`dr.as`	—	Dr.ªs	*«doctoras»*
`prof.a`	—	Prof.ª	*«profesora»*
`d.a`	`d.ª`	D.ª	*«doña»*
`m.a`	—	M.ª	*«maría»*
`n.o`	`n.º`	N.º	*«número»*
`n.os`	—	N.ºs	*«números»*
`c.ia`	—	C.ia	*«compañía»*
Abreviaturas comunes y expresiones latinas			
`pág.`	`pag.`	pág.	*«página»*
`vol.`	—	vol.	*«volumen»*
`etc.`	—	etc.	*«etcétera»*
`et al.`	`et. al.`	et al.	*«y otros»*
`ibíd.`	`ibid.`	ibíd.	*«ibídem»*
`op. cit.`	`op.cit.`	op. cit.	*«obra citada»*
`loc. cit.`	`loc.cit.`	loc. cit.	*«lugar citado»*
`v. gr.`	`v.gr.`	v. gr.	*«verbigracia»*
`i. e.`	`i.e.`	i. e.	*«esto es»*
`e. g.`	`e.g.`	e. g.	*«por ejemplo»*
`p. ej.`	`p.ej.`	p. ej.	*«por ejemplo»*
Títulos, profesiones y cargos			
`sr.`	—	Sr.	*«señor»*
`sra.`	—	Sra.	*«señora»*
`srta.`	—	Srta.	*«señorita»*
`dr.`	—	Dr.	*«doctor»*
`dra.`	—	Dra.	*«doctora»*
`dres.`	—	Dres.	*«doctores»*
`dras.`	—	Dras.	*«doctoras»*
`prof.`	—	Prof.	*«profesor»*
`profa.`	—	Profa.	*«profesora»*
`profs.`	—	Profs.	*«profesores»*
`ing.`	—	Ing.	*«ingeniero»*
`ings.`	—	Ings.	*«ingenieros»*
`lic.`	—	Lic.	*«licenciado»*
`v. b.`	`v.b.`	V. B.	*«visto bueno»*
Abreviaturas compuestas e institucionales			
`s. a.`	`s.a.`	S. A.	*«sociedad anónima»*
`ee. uu.`	`ee.uu.`	EE. UU.	*«estados unidos»*
`dd. hh.`	`dd.hh.`	DD. HH.	*«derechos humanos»*
`jj. oo.`	`jj.oo.`	JJ. OO.	*«juegos olímpicos»*

Continúa en la siguiente página...

©2026 by Pablo González L

Cuadro 2: *(Continuación - Abreviaturas soportadas)*

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
`ff. aa.`	`ff.aa.`	FF. AA.	*«fuerzas armadas»*
`rr. ee.`	`rr.ee.`	RR. EE.	*«relaciones exteriores»*
`cc. aa.`	`cc.aa.`	CC. AA.	*«comunidades autónomas»*
`p. d.`	`p.d.`	P. D.	*«posdata»*
`a. c.`	`a.c.`	a. C.	*«antes de Cristo»*
`d. c.`	`d.c.`	d. C.	*«después de Cristo»*
`d. n. i.`	`d.n.i.`	D. N. I.	*«documento nacional de identidad»*
`r. u. t.`	`r.u.t.`	R. U. T.	*«rol único tributario»*
`r. u. n.`	`r.u.n.`	R. U. N.	*«rol único nacional»*

El cuadro 3 detalla las siglas y acrónimos registrados en el módulo Lua `spintent.lua`.

Cuadro 3: Siglas y acrónimos soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Siglas (Deletreo o lectura expandida)			
`dni`	—	DNI	*«documento nacional de identidad»*
`rut`	—	RUT	*«rol único tributario»*
`run`	—	RUN	*«rol único nacional»*
`onu`	—	ONU	*«organización de las naciones unidas»*
`rae`	—	RAE	*«real academia española»*
`ong`	`ongs`	ONG	*«organización no gubernamental»*
`urss`	—	URSS	*«unión de repúblicas socialistas soviéticas»*
`oea`	—	OEA	*«organización de los estados americanos»*
`oms`	—	OMS	*«organización mundial de la salud»*
`fmi`	—	FMI	*«fondo monetario internacional»*
`bid`	—	BID	*«banco interamericano de desarrollo»*
`otan`	—	OTAN	*«organización del tratado del atlántico norte»*
`ue`	—	UE	*«unión europea»*
`ru`	—	RU	*«reino unido»*
Acrónimos silábicos			
`unicef`	—	Unicef	*«fondo de las naciones unidas para la infancia»*
`unesco`	—	Unesco	*«organización de las naciones unidas para la educación, la ciencia y la cultura»*
`mercosur`	—	Mercosur	*«mercado común del sur»*
`cepal`	—	Cepal	*«comisión económica para américa latina»*
`mineduc`	—	Mineduc	*«ministerio de educación»*
`conicet`	—	Conicet	*«consejo nacional de investigaciones»*

El cuadro 4 muestra ejemplos de números ordinales registrados en el módulo Lua `spintent.lua`..

Cuadro 4: Números ordinales soportados

Entrada	Alias / Variantes	Salida PDF	Lectura NVDA
Sufijos para ordinales (Ejemplos)			
`1.a`	`1.ª`	1.ª	*«primera»*
`2.o`	`2.º`	2.º	*«segundo»*
`3.er`	—	3.er	*«tercer»*
`4.os`	—	4.os	*«cuartos»*
`5.as`	—	5.as	*«quintas»*

©2026 by Pablo González L

4 Utilidades generales

Además de los *«cuatro grandes»* y sus asociados, el paquete spintent provee tres utilidades de uso general: `\spsiglo` para la escritura y lectura de siglos, `\sptime` para la escritura y lectura de tiempo y `\spdate` para la escritura y lectura de fechas.

El comando `\spsiglo`

`\spsiglo` `\spsiglo{⟨siglo⟩}`
`\spsiglo[⟨key = val⟩]{⟨siglo⟩}`

El comando `\spsiglo` trabaja en *modo texto* y en *modo matemático*, recibe el *argumento obligatorio* {⟨*siglo*⟩}, el cual puede ser un *número arábigo* '`21`' o un *número romano* '`XXI`' mayor que '`0`' y menor que '`4000`'.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩] y se ejecuta `\spsiglo{21}` o `\spsiglo{XXI}` se imprimirá 'xxi' en la salida PDF y NVDA/MathCAT verbalizará *«veintiuno»*.

El *argumento obligatorio* {⟨*siglo*⟩} *«siempre»* es convertido a *número romano en versalitas* y es verbalizado por NVDA/MathCAT como *número arábigo*. Esta es la única utilidad que provee el paquete spintent que trabaja en *modo texto* y en *modo matemático*. Internamente utiliza la llave `alt` proveída por el paquete tagpdf en *modo texto* y usa `\MathMLintent` para el *modo matemático*.

Llaves para `\spsiglo`

`print-era` = {⟨*ac* | *dc* | *none*⟩} default: *none*

Establece si se *imprime* o no la abreviatura después de imprimir el argumento {⟨*siglo*⟩}. Si se establece en `ac`, `\spsiglo[print-era=ac]{XXI}` imprimirá 'xi a. C.' en la salida PDF y NVDA/MathCAT verbalizará *«veintiuno antes de cristo»*; si se establece en `dc`, `\spsiglo[print-era=dc]{XXI}` imprimirá 'xi d. C.' en la salida PDF y NVDA/MathCAT verbalizará *«veintiuno después de cristo»*; si se establece en `none`, `\spsiglo[print-era=none]{XXI}` imprimirá 'xi' en la salida PDF y NVDA/MathCAT verbalizará *«veintiuno»*.

El comando `\sptime`

`\sptime` `\sptime{⟨horas : minutos⟩}`
`\sptime{⟨horas : minutos : segundos⟩}`

El comando `\sptime` trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*horas : minutos*⟩} o {⟨*horas : minutos : segundos*⟩}. Por ejemplo: `$\sptime{23:45}$` imprimirá '23:45' en *modo texto* la salida PDF y NVDA/MathCAT verbalizará *«las veintitrés y cuarenta y cinco minutos»*.

El comando `\spdate`

`\spdate` `\spdate{⟨DD/ MM/YYYY⟩}` `\spdate{⟨DD- MM-YYYY⟩}`
`\spdate{⟨YYYY/ MM/DD⟩}` `\spdate{⟨YYYY- MM-DD⟩}`

El comando `\spdate` trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨DD/MM/YYYY⟩}, {⟨DD-MM-YYYY⟩} o {⟨YYYY-MM-DD⟩}. Por ejemplo: `$\spdate{1981-01-02}$` y `$\spdate{02/01/1981}$` imprimirán en *modo texto* la salida PDF '1981-01-02' y '02/01/1981' respectivamente y en ambos casos, NVDA/MathCAT verbalizará *«dos de enero de mil novecientos ochenta y uno»*.

Si se pasa el argumento {⟨YYYY/MM/DD⟩}, internamente se convertirá en {⟨DD/MM/YYYY⟩} y se imprimirá de esa forma por compatibilidad con NVDA/MathCAT.

5 Utilidades para símbolos

El paquete spintent provee tres comandos `\spread`, `\spdsym` y `\spmsym` para trabajar con *«símbolos matemáticos»* descritos en esta sección. Están pensandos desde el punto de vista docente, para situaciones donde la lectura de un *«símbolo»* (acorde al contexto) es relevante.

A diferencia de las otras utilidades implementados por spintent (que analizan y dan formato a la salida), estos no procesan los *«símbolos matemáticos»*, solo afectan la *forma* (semántica) en que serán verbalizados por NVDA/MathCAT.

El comando `\spread`

`\spread` `\spread{⟨verbalización NVDA⟩}{⟨símbolo matemático⟩}`

El comando `\spread` trabaja en *modo matemático* y recibe dos *argumentos obligatorios*: El primer *argumento* {⟨*verbalización* NVDA⟩} establece la *«forma»* en la cual NVDA/MathCAT verbalizará al segundo *argumento* {⟨*símbolo matemático*⟩}. Por ejemplo: `$\spread{es-paralela}{\parallel}$` imprimirá '∥' en la salida PDF y NVDA/MathCAT verbalizará *«es paralela»*; `$\spread{es-paralelo}{\parallel}$` imprimirá '∥' en la salida PDF y NVDA/MathCAT verbalizará *«es paralelo»*.

©2026 by Pablo González L

El comando \spdsym

\spdsym `\spdsym`
`\spdsym[⟨key = val⟩]`

El comando `\spdsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[⟨key = val⟩]` mediante el cual se establece la *«forma»* en la que se imprimirá el *símbolo de división* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `$\spdsym$` imprimirá '÷' en la salida PDF y NVDA/MathCAT verbalizará *«dividido»*.

Llaves para \spdsym

`prefix` = {⟨*prefijo*⟩} default: *no usado*

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT *«antes»* de *verbalizar* el {⟨*valor*⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix` = {⟨*sufijo*⟩} default: *no usado*

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT *«después»* de *verbalizar* el {⟨*valor*⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym` = {⟨*old-style* | *two-dots* | *slash*⟩} default: *two-dots*

Establece el *símbolo de división* que se imprimirá en la salida PDF. Si se establece en `old-style`, imprimirá '÷'; si se establece en `two-dots`, imprimirá ':' y si se establece `slash`, imprimirá '/'.

`read-sym` = {⟨*dividido* | *dividido-por* | *dividido-entre*⟩} default: *dividido*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de división*. Si se establece en `dividido`, NVDA/MathCAT verbalizará *«dividido»*; si se establece en `dividido-por`, NVDA/MathCAT verbalizará *«dividido por»* y si se establece en `dividido-entre`, NVDA/MathCAT verbalizará *«dividido entre»*.

El comando \spmsym

\spmsym `\spmsym`
`\spmsym[⟨key = val⟩]`

El comando `\spmsym` trabaja en *modo matemático* y recibe el *argumento opcional* `[⟨key = val⟩]` mediante el cual se establece la *«forma»* en la que se imprimirá el *símbolo de multiplicación* y como se verbalizará en NVDA/MathCAT.

Si no se proporciona el *argumento opcional* `[⟨key = val⟩]`, `$\spmsym$` imprimirá '·' en la salida PDF y NVDA/MathCAT verbalizará *«por»*.

Llaves para \spmsym

`prefix` = {⟨*prefijo*⟩} default: *no usado*

Establece el *prefijo* que será *verbalizado* por NVDA/MathCAT *«antes»* de *verbalizar* el {⟨*valor*⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`suffix` = {⟨*sufijo*⟩} default: *no usado*

Establece el *sufijo* que será *verbalizado* por NVDA/MathCAT *«después»* de *verbalizar* el {⟨*valor*⟩} establecido por la llave `read-sym`. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

`set-sym` = {⟨*cross* | *dot* | *asterisk*⟩} default: *dot*

Establece el *símbolo de multiplicación* que se imprimirá en la salida PDF. Si se establece en `cross`, imprimirá '×'; si se establece en `dot`, imprimirá '·' y si se establece en `asterisk`, imprimirá '*'.

`read-sym` = {⟨*por* | *multiplicado-por* | *veces*⟩} default: *por*

Establece la *forma* con la cual NVDA/MathCAT verbalizará el *símbolo de multiplicación*. Si se establece en `por`, NVDA/MathCAT verbalizará *«por»*; si se establece en `multiplicado-por`, NVDA/MathCAT verbalizará *«multiplicado por»* y si se establece en `veces`, NVDA/MathCAT verbalizará *«veces»*.

`not-print` = {⟨*true* | *false*⟩} default: *false*

Deshabilita la *impresión* del *símbolo de multiplicación*. Cuando se establece esta llave, NVDA/MathCAT verbalizará el *símbolo de multiplicación* acorde al valor establecido por la llave `read-sym`, pero, no se imprimirá este en la salida PDF.

6 Utilidades para números

El paquete spintent provee utilidades para trabajar con números descritas en esta sección.

©2026 by Pablo González L

El comando \spdiv

\spdiv
\spdiv{⟨dividendo⟩}{⟨divisor⟩}
\spdiv[⟨key = val⟩]{⟨dividendo⟩}{⟨divisor⟩}

El comando \spdiv trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨dividendo⟩} y {⟨divisor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], $\spdiv{20}{26}$ imprimirá '20 : 26' en la salida PDF y NVDA/MathCAT verbalizará *«veinte dividido veinte y seis»*

Llaves para \spdiv

El comando \spdiv posee las ⟨meta-keys⟩ cifras-sep, read-sign, read-period-sep y notation-sym pasadas al comando \spnum (§3.1.1) junto a las ⟨meta-keys⟩ set-sym y read-sym pasadas al comando \spdsym (pág. 12).

wrap-parens = {⟨true | false⟩} default: *false*

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a \spdiv. Si se establece en true, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en false, no se imprimirán los paréntesis.

read-parens = {⟨true | false⟩} default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos '(...)' alrededor de los *argumentos* {⟨dividendo⟩} y {⟨divisor⟩} pasados a \spdiv. Si se establece en true, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en false, NVDA/MathCAT no los verbalizará.

El comando \spmul

\spmul
\spmul{⟨factor⟩}{⟨factor⟩}
\spmul[⟨key = val⟩]{⟨factor⟩}{⟨factor⟩}

El comando \spmul trabaja en *modo matemático* y recibe dos *argumentos obligatorios* {⟨factor⟩} y {⟨factor⟩}.

Si no se proporciona el *argumento opcional* [⟨key = val⟩], $\spmul{20}{26}$ imprimirá '20 · 26' en la salida PDF y NVDA/MathCAT verbalizará *«veinte por veinte y seis»*.

Llaves para \spmul

El comando \spmul posee las ⟨meta-keys⟩ cifras-sep, read-sign, read-period-sep y notation-sym pasadas al comando \spnum (§3.1.1) junto a las ⟨meta-keys⟩ set-sym y read-sym pasadas al comando \spmsym (pág. 12).

wrap-parens = {⟨true | false⟩} default: *false*

Establece si se *imprimen* paréntesis redondos '(...)' alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a \spmul. Si se establece en true, se imprimirán paréntesis redondos alrededor de ambos *argumentos*; si se establece en false, no se imprimirán los paréntesis.

read-parens = {⟨true | false⟩} default: *true*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos '(...)' alrededor de los *argumentos* {⟨factor⟩} y {⟨factor⟩} pasados a \spmul. Si se establece en true, NVDA/MathCAT verbalizará la lectura de los paréntesis; si se establece en false, NVDA/MathCAT no los verbalizará.

El comando \spnD

\spnD
\spnD
\spnD(⟨número natural⟩)
\spnD[⟨key = val⟩](⟨número natural⟩)

El comando \spnD trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (⟨número natural⟩), por ejemplo: '(6)'. Si se usa sin (⟨argumento⟩), $\spnD$ imprimirá 'D(n)' en la salida PDF y NVDA/MathCAT verbalizará *«divisores de n»*.

Si se usa con (⟨argumento⟩), por ejemplo, $\spnD(6)$, imprimirá 'D(6)' en la salida PDF y NVDA/MathCAT verbalizará *«divisores de seis»*.

Llaves para \spnD

prefix = {⟨prefijo⟩} default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *«antes»* de verbalizar \spnD(⟨argumento⟩). El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

result = {⟨true | false⟩} default: *false*

Establece si se *imprime* el resultado de calcular los *divisores* del (⟨número natural⟩) pasado a \spnD. Si se establece en true, $\spnD[result=true](6)$ imprimirá 'D(6) = {1,2,3,6}' en la salida PDF y NVDA/MathCAT verbalizará *«los divisores de seis son uno, dos, tres y seis»*; si se establece en false, no se imprimirá en la salida PDF el resultado de calcular los *divisores*.

©2026 by Pablo González L

`result-num` = {⟨*número natural*⟩} default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF; si no se establece, se imprimirán como máximo `5` valores y luego se añadirá '...' en la salida PDF.

`sample-arg` = {⟨*true* | *false*⟩} default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a `\spnD`, permitiendo usar ejemplos *«no numéricos»* como entrada. Si se establece en `true`, `$\spnD[sample-arg=true](m)$` imprimirá 'D(m)' en la salida PDF y NVDA/MathCAT verbalizará *«divisores de m»*; si se establece en `false`, el (⟨*argumento*⟩) debe ser un número natural, en caso contrario se devolverá un "error".

`silent-cmd` = {⟨*true* | *false*⟩} default: *false*

Establece si se *silencia* la verbalización para NVDA/MathCAT de `\spnD`(⟨*argumento*⟩) cuando se establece `sample-arg=true` o cuando `\spnD` se usa sin (⟨*argumento*⟩). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false`, NVDA/MathCAT SÍ lo verbalizará.

El comando `\spnM`

`\spnM`

`\spnM`
`\spnM`(⟨*número natural*⟩)
`\spnM`[⟨*key = val*⟩](⟨*número natural*⟩)

El comando `\spnM` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (⟨*número natural*⟩), por ejemplo: '`(6)`'. Si se usa sin (⟨*argumento*⟩), `$\spnM$` imprimirá 'M(n)' en la salida PDF y NVDA/MathCAT verbalizará *«múltiplos de n»*.

Si se usa con (⟨*argumento*⟩), por ejemplo, `$\spnM(6)$`, imprimirá 'M(6)' en la salida PDF y NVDA/MathCAT verbalizará *«múltiplos de seis»*.

Llaves para `\spnM`

`prefix` = {⟨*prefijo*⟩} default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *«antes»* de verbalizar `\spnM`(⟨*argumento*⟩). El valor de esta llave *siempre* debe pasarse entre llaves '`{ }`'.

`result` = {⟨*true* | *false*⟩} default: *false*

Establece si se *imprime* el resultado de calcular los *múltiplos* del (⟨*número natural*⟩) pasado a `\spnM`. Si se establece en `true`, `$\spnM[result=true](6)$` imprimirá 'M(6) = {6, 12, 18, 24, 30, ...}' en la salida PDF y NVDA/MathCAT verbalizará *«los múltiplos de seis son seis, doce, dieciocho, veinticuatro, treinta puntos suspencivos»*; si se establece en `false`, no se imprimirá en la salida PDF el resultado de calcular los *múltiplos*.

`result-num` = {⟨*número natural*⟩} default: *5*

Establece la *cantidad* de números que se mostrarán en la salida PDF al activar `result=true`. Si se establece en `2`, solo se imprimirán dos valores en la salida PDF seguidos de '...'; si no se establece, se imprimirán por defecto los `5` primeros valores y luego se añadirá '...' en la salida PDF.

`sample-arg` = {⟨*true* | *false*⟩} default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a `\spnM`, permitiendo usar ejemplos *«no numéricos»* como entrada. Si se establece en `true`, `$\spnM[sample-arg=true](m)$` imprimirá 'M(m)' en la salida PDF y NVDA/MathCAT verbalizará *«múltiplos de m»*; si se establece en `false`, el (⟨*argumento*⟩) debe ser un número natural, en caso contrario se devolverá un "error".

`silent-cmd` = {⟨*true* | *false*⟩} default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de `\spnM`(⟨*argumento*⟩) cuando se establece `sample-arg=true` o cuando `\spnM` se usa sin (⟨*argumento*⟩). Si se establece en `true` NVDA/MathCAT NO lo verbalizará; si se establece en `false` NVDA/MathCAT lo verbalizará.

El comando `\spmcd`

`\spmcd`

`\spmcd`
`\spmcd`(⟨*números separados por coma*⟩)
`\spmcd`[⟨*key = val*⟩](⟨*números separados por coma*⟩)

El comando `\spmcd` trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (⟨*números separados por coma*⟩), el cual debe ser una lista de *números naturales*, por ejemplo: '`(2, 3, 6)`'. Si se usa sin (⟨*argumento*⟩) `$\spmcd$` imprimirá 'mcd' en la salida PDF, y NVDA/MathCAT verbalizará *«máximo común divisor»* o *«m c d»*, acorde al {⟨*valor*⟩} establecido por la llave `read-cmd`.

Si se usa con (⟨*argumento*⟩) `$\spmcd(2, 3, 6)$` imprimirá 'mcd(2,3,6)' en la salida PDF, y NVDA/MathCAT verbalizará *«máximo común divisor entre dos, tres y seis»* o *«m c d entre dos, tres y seis»* acorde al {⟨*valor*⟩} establecido por la llave `read-cmd`.

©2026 by Pablo González L

Llaves para \spmcd

upper-case = {⟨*true* | *false*⟩} default: *false*

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando \spmcd. Si se establece en true, se imprimirá 'MCD' en la salida PDF; si se establece en false, se imprimirá 'mcd' en la salida PDF.

read-cmd = {⟨*terse* | *clear*⟩} default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando \spmcd. Si se establece en terse, NVDA/MathCAT verbalizará *«m c d»*; si se establece en clear, NVDA/MathCAT verbalizará *«máximo común divisor»*.

prefix = {⟨*prefijo*⟩} default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *«antes»* de verbalizar el valor establecido por la llave read-cmd. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

result = {⟨*true* | *false*⟩} default: *false*

Establece si se *imprime* el resultado de calcular el *«máximo común divisor»* del (⟨*argumento*⟩) pasado a \spmcd. Si se establece en true: $\spmcd[result=true](2, 3, 6)$ imprimirá 'mcd(2,3,6) = 1' en la salida PDF y NVDA/MathCAT verbalizará *«máximo común divisor entre dos, tres y seis es igual a uno»*; si se establece en false no se imprimirá en la salida PDF el resultado de calcular el *«máximo común divisor»*.

sample-arg = {⟨*true* | *false*⟩} default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a \spmcd, permitiendo usar ejemplos *«no numéricos»* como entrada. Si se establece en true: $\spmcd[sample-arg=true](m, n)$, imprimirá 'mcd($m, n$)' en la salida PDF y NVDA/MathCAT verbalizará *«máximo común divisor entre m y n»*; si se establece en false el (⟨*argumento*⟩) debe ser una lista de *números naturales*, en caso contrario se devolverá un "error".

silent-cmd = {⟨*true* | *false*⟩} default: *false*

Establece si se *silencia* la verbalización en NVDA/MathCAT de \spmcd(⟨*argumento*⟩) cuando se establece sample-arg=true o cuando \spmcd se usa sin (⟨*argumento*⟩). Si se establece en true NVDA/MathCAT NO lo verbalizará; si se establece en false, NVDA/MathCAT SÍ lo verbalizará.

El comando \spmcm

\spmcm

\spmcm
\spmcm(⟨*números separados por coma*⟩)
\spmcm[⟨*key = val*⟩](⟨*números separados por coma*⟩)

El comando \spmcm trabaja en *modo matemático* y recibe el *argumento delimitado por paréntesis* (⟨*números separados por coma*⟩), el cual debe ser una lista de *números naturales*, por ejemplo: '(2, 3, 6)'. Si se usa sin (⟨*argumento*⟩) $\spmcm$ imprimirá 'mcm' en la salida PDF, y NVDA/MathCAT verbalizará *«mínimo común múltiplo»* o *«m c m»*, acorde al {⟨*valor*⟩} establecido por la llave read-cmd.

Si se usa con (⟨*argumento*⟩) $\spmcm(2, 3, 6)$ imprimirá 'mcm(2,3,6)' en la salida PDF, y NVDA/MathCAT verbalizará *«mínimo común múltiplo entre dos, tres y seis»* o *«m c m entre dos, tres y seis»* acorde al {⟨*valor*⟩} establecido por la llave read-cmd.

Llaves para \spmcm

upper-case = {⟨*true* | *false*⟩} default: *false*

Establece si se usarán *mayúsculas* o *minúsculas* en la salida PDF para el comando \spmcm. Si se establece en true se imprimirá 'MCM' en la salida PDF; si se establece en false se imprimirá 'mcm' en la salida PDF.

read-cmd = {⟨*terse* | *clear*⟩} default: *clear*

Establece la *forma* en la que NVDA/MathCAT verbalizará el comando \spmcm. Si se establece en terse, NVDA/MathCAT verbalizará *«m c m»*; si se establece en clear, NVDA/MathCAT verbalizará *«mínimo común múltiplo»*.

prefix = {⟨*prefijo*⟩} default: *no usado*

Establece el *prefijo* que será verbalizado por NVDA/MathCAT *«antes»* de verbalizar el valor establecido por la llave read-cmd. El valor de esta llave *siempre* debe pasarse entre llaves '{ }'.

result = {⟨*true* | *false*⟩} default: *false*

Establece si se *imprime* el resultado de calcular el *«mínimo común múltiplo»* del (⟨*argumento*⟩) pasado a \spmcm. Si se establece en true: $\spmcm[result=true](2, 3, 6)$ imprimirá 'mcm(2,3,6) = 6' en la salida PDF y NVDA/MathCAT verbalizará *«mínimo común múltiplo entre dos, tres y seis es igual a seis»*; si se establece en false no se imprimirá en la salida PDF el resultado de calcular el *«mínimo común múltiplo»*.

sample-arg = {⟨*true* | *false*⟩} default: *false*

Desactiva la *validación* interna del (⟨*argumento*⟩) pasado a \spmcm, permitiendo usar ejemplos *«no numéricos»* como entrada. Si se establece en true: $\spmcm[sample-arg=true](m, n)$, imprimirá 'mcm($m, n$)' en la salida PDF y NVDA/MathCAT verbalizará *«mínimo común múltiplo entre m y n»*; si se establece en false el (⟨*argumento*⟩) debe ser una lista de *números naturales*, en caso contrario se devolverá un "error".

silent-cmd = {⟨*true* | *false*⟩} default: *false*

©2026 by Pablo González L

Establece si se *silencia* la verbalización en NVDA/MathCAT de \spmcm(⟨*argumento*⟩) cuando se establece sample-arg=true o cuando \spmcm se usa sin (⟨*argumento*⟩). Si se establece en true NVDA/MathCAT NO lo verbalizará; si se establece en false NVDA/MathCAT SÍ lo verbalizará.

El comando \spnZ

\spnZ
\spnZ{⟨*número entero*⟩}
\spnZ[⟨*key = val*⟩]{⟨*número entero*⟩}

El comando \spnZ trabaja en *modo matemático* y recibe el *argumento* obligatorio {⟨*número entero*⟩} el cual se procesa internamente bajo el comando \spnum. Este comando está dedicado exclusivamente al manejo de *«números enteros»*, y añade algunas utilidades prácticas para el demarcado e interacción con NVDA/MathCAT.

Llaves para \spnZ

El comando \spnZ posee las ⟨*meta-keys*⟩ cifras-sep y read-sign pasadas al comando \spnum (§3.1.1).

wrap-parens = {⟨*true* | *false*⟩} default: *false*

Establece si se *imprimen* paréntesis redondos '(...)' alrededor del argumento {⟨*número entero*⟩} pasado a \spnZ. Si se establece en true, se imprimirán paréntesis redondos alrededor del {⟨*número entero*⟩}; si se establece en false, no se imprimirán los paréntesis.

read-parens = {⟨*true* | *false*⟩} default: *false*

Establece si NVDA/MathCAT *verbalizará* los paréntesis redondos '(...)' alrededor del *argumento* {⟨*número entero*⟩} pasado a \spnZ. Si se establece en true, NVDA/MathCAT verbalizará la lectura del paréntesis '(...)'; si se establece en false, NVDA/MathCAT no los verbalizará.

El comando \spmQ

\spmQ
\spmQ{⟨*número entero*⟩}{⟨*numerador*⟩}{⟨*denominador*⟩}
\spmQ[⟨*key = val*⟩]{⟨*número entero*⟩}{⟨*numerador*⟩}{⟨*denominador*⟩}

El comando \spmQ trabaja en *modo matemático* y recibe *tres argumentos* obligatorios: {⟨*número entero*⟩}, {⟨*numerador*⟩} y {⟨*denominador*⟩}, los procesa internamente y devuelve un *«número mixto»*.

Si no se proporciona el *argumento opcional* [⟨*key = val*⟩], $\spmQ{2}{5}{7}$ imprimirá '$2\frac{5}{7}$' en la salida PDF y NVDA/MathCAT verbalizará *«dos y cinco séptimos»*.

En el caso de las fracciones, y en especial con los *«números mixtos»*, no se pueden manejar muchas cosas desde el lado de spintent. Por ejemplo, la entrada $3\frac{5}{7}$ con *tagged* PDF se verbaliza por NVDA/MathCAT como *«tres más cinco séptimos»*, lo cual no está mal, pero pedagógicamente hablando no es del todo correcto.

Llaves para \spmQ

join-word = {⟨*entero* | *enteros* | *y*⟩} default: *y*

Establece la *forma* en la que NVDA/MathCAT verbaliza la conexión entre el entero y la fracción que forman el número mixto. Si se establece en entero, al usar $\spmQ[join-word=entero]{1}{5}{7}$ imprimirá '$1\frac{5}{7}$' en la salida PDF, y NVDA/MathCAT verbalizará *«un entero cinco séptimos»*; si se establece en enteros, al usar $\spmQ[join-word=enteros]{2}{5}{7}$ imprimirá '$2\frac{5}{7}$' en la salida PDF, y NVDA/MathCAT verbalizará *«dos enteros cinco séptimos»*; si se establece en y, al usar $\spmQ[join-word=y]{2}{5}{7}$ imprimirá '$2\frac{5}{7}$' en la salida PDF, y NVDA/MathCAT verbalizará *«dos y cinco séptimos»*.

7 Utilidades para geometría

El paquete spintent provee utilidades para trabajar con geometría (plana), que describiremos en esta sección.

El comando \spang

\spang
\spang{⟨*grados*⟩}
\spang{⟨*grados* : *minutos*⟩}
\spang{⟨*grados* : *minutos* : *segundos*⟩}

El comando \spang trabaja en *modo matemático* y recibe el *argumento obligatorio* {⟨*grados*⟩}, {⟨*grados* : *minutos*⟩} o {⟨*grados* : *minutos* : *segundos*⟩}. Los procesa internamente bajo los comandos \spnum y \spunit para imprimir y verbalizar el *«ángulo sexagesimal»*. Por ejemplo, $\spang{45:30:15}$ imprimirá '45°30′15″' en la salida PDF y NVDA/MathCAT verbalizará *«cuarenta y cinco grados treinta minutos quince segundos»*.

©2026 by Pablo González L

8 Referencias

[1] Robertson, Will . “unicode-math - Unicode mathematics support for XeTeX and LuaTeX”. Available from CTAN, https://www.ctan.org/pkg/unicode-math, 2023.

[2] Krueger, Marcel. “LuaMML - Automatically generate MathML from LuaTeX math mode material”. Available from CTAN, https://ctan.org/pkg/luamml, 2026.

[3] The LaTeX Project. “fontspec - Advanced font selection for XeLaTeX and LuaTeX”. Available from CTAN, https://www.ctan.org/pkg/fontspec, 2025.

[4] Voß, Herbert. “libertinus-otf - Support for Libertinus OpenType”. Available from CTAN, https://www.ctan.org/pkg/libertinus-otf, 2025.

[5] Flipo, Daniel. “erewhon-math - Utopia based OpenType Math font”. Available from CTAN, https://ctan.org/pkg/erewhon-math, 2026.

[6] Hofstra, Silke. “sourcecodepro - Use SourceCodePro with TeX(-alike) systems”. Available from CTAN, https://ctan.org/pkg/sourcecodepro, 2025.

[7] González, Pablo. “scontents - Stores LaTeX contents in memory or files”. Available from CTAN, https://www.ctan.org/pkg/scontents, 2026.

[8] González, Pablo. “enumext - Enumerate exercise sheets”. Available from CTAN, https://www.ctan.org/pkg/enumext, 2026.

[9] Braams, Johannes and Bezos, Javier. “babel - Multilingual support for LaTeX, LuaLaTeX, XeLaTeX, and Plain TeX”. Available from CTAN, https://www.ctan.org/pkg/babel, 2026.

[10] Bezos, Javier. “babel-spanish - Babel support for Spanish”. Available from CTAN, https://www.ctan.org/pkg/babel-spanish, 2026.

[11] NV Access. “NVDA - NonVisual Desktop Access screen reader”. Available from NV Access, https://www.nvaccess.org/, 2026.

[12] Soiffer, Neil. “MathCAT - Math to speech and braille translation”. Available from GitHub, https://nsoiffer.github.io/MathCAT/, 2026.

[13] Adobe Inc. “Adobe Acrobat Reader - Reliable PDF viewer”. Available from Adobe, https://get.adobe.com/reader/, 2026.

[14] ISO. “ISO 32000-2:2020 - Portable Document Format (PDF 2.0)”. Available from PDF Association, https://pdfa.org/resource/iso-32000-2/, 2020.

[15] PDF Association. “WTPDF - Well-Tagged PDF specification”. Available from PDFA, https://pdfa.org/wtpdf/, 2024.

[16] PDF Association. “PDF 2.0 Errata Collection 3”. Available from PDFA, https://pdfa.org/PDF-2-0-errata-collection-3-now-available/, 2026.

[17] LaTeX Project LWG. “Best Practice Guide: Math in PDF”. Available from PDFA, https://pdfa.org/resource/best-practice-guide-math-in-PDF/, 2026.

[18] Johnson, Duff. “PDF 2.0: New Features, Real-World Impact”. Available from PDFA, https://pdfa.org/PDF-2-0-new-features-real-world-impact/, 2026.

[19] The LaTeX Project. “LaTeX Tagged PDF Project - Documentation and resources”. Available from LaTeX Project, https://latex3.github.io/tagging-project/, 2026.

[20] veraPDF Consortium. “veraPDF - Industry supported PDF/A and PDF/UA validator”. Available from veraPDF, https://verapdf.org/, 2026.

[21] Dual Lab. “ngPDF - Next Generation PDF demonstrator and validator”. Available from ngPDF, https://ngpdf.com/, 2026.

[22] Berry, Karl. “LaTeX 2_ε: An Unofficial Reference Manual”. Available from CTAN, https://ctan.org/pkg/latex2e-help-texinfo, 2026.

[23] The LaTeX Project. “Extensive support for hypertext in LaTeX”. Available from CTAN, https://www.ctan.org/pkg/hyperref, 2026.

[24] The LaTeX Project. “The expl3 package”. Available from CTAN, https://www.ctan.org/pkg/l3kernel, 2026.

©2026 by Pablo González L

[25] The LaTeX Project. “The LaTeX3 Interfaces”. Available from CTAN, https://www.ctan.org/pkg/l3kernel, 2026.

[26] The LaTeX Project. “The LaTeX 2_ε sources”. Available from CTAN, https://ctan.org/tex-archive/macros/latex/base, 2026.

[27] The LaTeX Project. “LaTeX News, Issue 41, June 2025”. Available from CTAN, https://ctan.org/tex-archive/macros/latex/base, 2025.

[28] The LaTeX Project. “LaTeX for authors current version”. Available from CTAN, https://ctan.org/pkg/latex-base, 2026.

[29] Fischer, Ulrike. “tagpdf – LaTeX kernel code for PDF tagging”. Available from CTAN, https://www.ctan.org/pkg/tagpdf, 2026.

[30] The LaTeX Project. “latex-lab – LaTeX laboratory”. Available from CTAN, https://www.ctan.org/pkg/latex-lab, 2026.

9 Change history

2.4 (ctan), 2024/09/08 – First public release.

©2026 by Pablo González L

10 Índice de la Documentación

Los números en cursiva indican las páginas donde se describe la entrada correspondiente.

©2026 by Pablo González L

©2026 by Pablo González L

11 Implementation

The most recent publicly released version of spintent is available at CTAN: https://www.ctan.org/pkg/spintent. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: https://github.com/pablgonz/spintent/issues.

The documentation presented here is far from professional, it contains a lot of obvious information that to the eye of a TEXpert are superfluous, but, after so many years developing this project is the only way to remember what does what.

11.1 Initial set up

Start the DocStrip guards.

```
1 ⟨*package⟩
```

Identify the internal prefix (LaTeX3 DocStrip convention) for l3doc class.

```
2 ⟨@@=spintent⟩
```

11.2 General conventions

Functions of the form `\__spintent_luafun_some:n` or `\__spintent_luafun_some:nn` are defined (created) in the Lua module `spintent.lua` and only their variants are declared in `expl3`.

Similarly, variables of the form `\l__spintent_luaset_some_tl` or `\l__spintent_luaset_some_str` are defined in the Lua module `spintent.lua` and only declared in `expl3`.

All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

11.3 Declaration of the package

First we will make sure we have a minimum (super updated) version of LaTeX to work correctly. `NeedsTeXFormat`

```
3 \NeedsTeXFormat{LaTeX2e}[2026-11-01]
```

Now declare the spintent package.

```
4 \ProvidesExplPackage {spintent} {2026-08-11} {0.95} {Spanish parse intents}
```

We check if the requirements for running the spintent package are met; that is: `\DocumentMetadata` must be active, LuaTeX must be running, and the `unicode-math` or `lua-unicode-math` package must be loaded. If neither is present, we will load `unicode-math`. If the conditions cannot be met, we will return a “fatal error”.

```
5  \IfDocumentMetadataTF
6    {
7      \sys_if_engine_luatex:TF
8        {
9          \msg_new:nnn { spintent } { package-not-load }
10           {
11             The~'#1'~package~will~be~loaded~as~a~dependency.
12           }
13         \IfPackageLoadedF { unicode-math }
14           {
15             \IfPackageLoadedF { lua-unicode-math }
16               {
17                 \msg_info:nnn { spintent } { package-not-load } { unicode-math }
18                 \RequirePackage{unicode-math}
19               }
20           }
21         \msg_new:nnn { spintent } { env-success }
22           { Everything~looks~correct,~spintent~will~make~the~intent! }
23         \msg_info:nn { spintent } { env-success }
24       }
25       {
26         \msg_new:nnnn { spintent } { fatal-engine-requires-luatex }
27           { The~spintent~package~requires~LuaLaTeX. }
28           { This~package~relies~on~Lua~module.~Compile~with~lualatex. }
29         \msg_fatal:nn { spintent } { fatal-engine-requires-luatex }
30       }
31   }
32   {
33     \msg_new:nnnn { spintent } { fatal-metadata-missing }
34       { \token_to_str:N \DocumentMetadata \c_space_tl is~missing. }
35       {
36         The~spintent~package~requires~tagged~PDF~active.\\
37         You~must~declare~\token_to_str:N \DocumentMetadata { tagging=on } ~
38       }
39     \msg_fatal:nn { spintent } { fatal-metadata-missing }
40   }
```

©2026 by Pablo González L

11.4 Some kernel variants

\str_if_eq:nVTF
\str_if_eq:nVF
\str_uppercase:V
\str_lowercase:V
\seq_set_from_clist:Ne

Non-standard kernel variants.

```
\cs_generate_variant:Nn \str_if_eq:nnTF { V }
\cs_generate_variant:Nn \str_if_eq:nnF { V }
\cs_generate_variant:Nn \str_uppercase:n { V }
\cs_generate_variant:Nn \str_lowercase:n { V }
\cs_generate_variant:Nn \seq_set_from_clist:Nn { Ne }
```

(*End of definition for* `\str_if_eq:nVTF` *and others.*)

11.5 Loading the Lua module and functions

__spintent_luafun_spunit_lookup_alias:V
__spintent_luafun_spmoney_lookup_data:V
__spintent_luafun_spmQ_scale_int:Vn
__spintent_luafun_clean_split_and_set:V

Now we load the Lua module `spintent.lua` and generate the variants for the functions `\__spintent_luafun_spunit` used by `\spunit` (§11.13), `\__spintent_luafun_spmoney_lookup_data:V` used by `\spmoney` (§11.15), `\__spintent_luafun_spmQ_scale_int:Vn` used by `\spmQ` (§11.28) and `\__spintent_luafun_clean_split_and` used by `\spnum` (§11.12) defined in it.

```
\lua_load_module:n { spintent.lua }
\cs_generate_variant:Nn \__spintent_luafun_spunit_lookup_alias:n { V }
\cs_generate_variant:Nn \__spintent_luafun_spmoney_lookup_data:n { V }
\cs_generate_variant:Nn \__spintent_luafun_spmQ_scale_int:nn { VV, Vn }
\cs_generate_variant:Nn \__spintent_luafun_clean_split_and_set:n { V }
```

(*End of definition for* `\__spintent_luafun_spunit_lookup_alias:V` *and others.*)

11.6 Internal copy of the unicode invisible separator

__spintent_invisible_sep:
\c__spintent_invisible_sep_tl

The function `\__spintent_invisible_sep:` is an internal copy of the Unicode invisible separator `U-2063` that uses `\luamml_annotate:en` from the package `luamml`[2] and let `<mo>...</mo>` in the MathML structure embedded in the PDF output.

```
\cs_new_protected:Nn \__spintent_invisible_sep:
  {
    \luamml_annotate:en
      {
        core = {[0] = 'mo', intent=':silent', "^^^^2063"}
      }
      {
        \latelua{}
      }
  }
```

The token list constant `\c__spintent_invisible_sep_tl` is an internal copy of the Unicode invisible separator `U-2063` using the primitive `\Umathchardef` from LuaTEX which leaves `<mi>...</mi>` in the structure MathML embedded in the output PDF.

```
\hook_gput_code:nnn { begindocument } { spintent }
  {
    \Umathchardef \c__spintent_invisible_sep_tl = "0 "0 "2063
  }
```

Both copies are used for different purposes, the function `\__spintent_invisible_sep:` will be used to force the start of a `<mrow>` in the structure MathML, the token list constant `\c__spintent_invisible_sep_tl` will be used to inject text into the structure MathML when we use `\MathMLintent{`⟨*text*⟩`}`.

(*End of definition for* `\__spintent_invisible_sep:` *and* `\c__spintent_invisible_sep_tl`.)

11.7 Normalize argument and affixes

We normalize the {⟨*argument*⟩} to support affixes (prefixes/suffixes) by trimming leading and trailing spaces, converting internal spaces to hyphens, collapsing consecutive hyphens, and removing leading or trailing hyphens.

__spintent_collapse_hyphens:N

A recursive helper to collapse multiple hyphens into a single one.

```
\cs_new_protected:Nn \__spintent_collapse_hyphens:N
  {
    \tl_if_in:NnT #1 { -- }
      {
        \tl_replace_all:Nnn #1 { -- } { - }
        \__spintent_collapse_hyphens:N #1
      }
  }
```

(*End of definition for* `\__spintent_collapse_hyphens:N`.)

__spintent_sanitize_affix:Nn
__spintent_sanitize_affix:cn

The function __spintent_sanitize_affix:Nn checks if the {⟨*argument*⟩} passed in '#2' is *blank*. If so, it clears the target token list variable passed in '#1'. Otherwise, trimming bounds, converting spaces '␣' to hyphens '-', collapsing multiple hyphens, and safely stripping leading/trailing hyphens. For example, if you pass a target variable and raw text:

```
\__spintent_sanitize_affix:Nn \l__spintent_spsomesym_prefix_tl{ foo bar baz }
```

The function will *"store"* the normalized string `foo-bar-baz` inside \l__spintent_spsomesym_prefix_tl, ready to be conditionally appended.

```
\cs_new_protected:Nn \__spintent_sanitize_affix:Nn
  {
    \tl_if_blank:nTF {#2}
      { \tl_clear:N #1 }
      {
        \tl_set:Ne #1 {#2}
        \tl_trim_spaces:N #1
        \tl_replace_all:Nnn #1 { ~ } { - }
        \__spintent_collapse_hyphens:N #1
        \tl_put_left:Nn  #1 { \q_mark }
        \tl_put_right:Nn #1 { \q_stop }
        \tl_replace_all:Nnn #1 { \q_mark - } { \q_mark }
        \tl_replace_all:Nnn #1 { - \q_stop } { \q_stop }
        \tl_remove_all:Nn #1 { \q_mark }
        \tl_remove_all:Nn #1 { \q_stop }
      }
  }
\cs_generate_variant:Nn \__spintent_sanitize_affix:Nn { cn }
```

(*End of definition for* __spintent_sanitize_affix:Nn.)

11.8 The command \setspintent

The \setspintent command is used to configure ⟨*keys*⟩ *"globally"* for the utilities provided by spintent.

11.8.1 Internal variables and error messages for \setspintent

\l__spintent_setspintent_input_arg_cmd_tl
unknown-command-setup

The variable \l__spintent_setspintent_input_arg_cmd_tl will *store* the command or command name passed to the *"first"* {⟨*argument*⟩} '#1' of the command \setspintent.

```
\tl_new:N \l__spintent_setspintent_input_arg_cmd_tl
```

Error message if the {⟨*argument*⟩} '#1' passed to \setspintent is a *"non-existent"* command.

```
\msg_new:nnnn { spintent } { unknown-command-setup }
  {
    Cannot~configure~unknown~command~'\exp_not:c{#1}'.
  }
  {
    You~attempted~to~use~\token_to_str:N \setspintent{#1}{...},~but~the~
    command~\exp_not:c{#1}~is~not~defined.~Please~check~for~typos.
  }
```

(*End of definition for* \l__spintent_setspintent_input_arg_cmd_tl *and* unknown-command-setup.)

__spintent_setspintent_set_keys:nn
__spintent_setspintent_set_keys:en

The function __spintent_setspintent_set_keys:nn will check if the command passed as {⟨*argument*⟩} exists in '#1' and set the ⟨*keys*⟩ passed as {⟨*argument*⟩} in '#2' to the correct path; otherwise, it will return an error.

```
\cs_new_protected:Nn \__spintent_setspintent_set_keys:nn
  {
    \cs_if_exist:cTF { #1 }
      {
        \keys_set:nn { spintent / #1 } { #2 }
      }
      {
        \msg_error:nnn { spintent } { unknown-command-setup } { #1 }
      }
  }
\cs_generate_variant:Nn \__spintent_setspintent_set_keys:nn { en }
```

(*End of definition for* __spintent_setspintent_set_keys:nn.)

__spintent_setspintent_parse_args:nn
__spintent_setspintent_parse_args:Vn

The function __spintent_setspintent_parse_args:nn will check if the argument passed in #1 begins with '\' or is just a *command name*. If it begins with '\', it will remove it using \cs_to_str:N and then execute __spintent_setspintent_set_keys:en. Otherwise, i.e., if '\' was not passed, it will directly execute __spintent_setspintent_set_keys:nn.

```
\cs_new_protected:Nn \__spintent_setspintent_parse_args:nn
  {
    \tl_if_single:nTF { #1 }
      {
        \token_if_cs:NTF #1
          {
            \__spintent_setspintent_set_keys:en { \cs_to_str:N #1 } { #2 }
          }
          {
            \__spintent_setspintent_set_keys:nn { #1 } { #2 }
          }
      }
      {
        \__spintent_setspintent_set_keys:nn { #1 } { #2 }
      }
  }
\cs_generate_variant:Nn \__spintent_setspintent_parse_args:nn { Vn }
```

(*End of definition for* __spintent_setspintent_parse_args:nn.)

\setspintent

Now we define the user command \setspintent.

```
\NewDocumentCommand \setspintent { m m }
  {
    \tl_if_blank:nTF { #1 }
      {
        \msg_error:nnn { spintent } { unknown-command-setup } { <empty> }
      }
      {
        \tl_set:Nn \l__spintent_setspintent_input_arg_cmd_tl { #1 }
        \tl_trim_spaces:N \l__spintent_setspintent_input_arg_cmd_tl
        \__spintent_setspintent_parse_args:Vn \l__spintent_setspintent_input_arg_cmd_tl { #2 }
      }
  }
```

(*End of definition for* \setspintent. *This function is documented on page* 3.)

11.9 Handling unknown keys

\l__spintent_command_name_str
unknown-choice
cmd-key-unknown
cmd-key-value-unknown

We define the variable \l__spintent_command_name_str which will *store* the *"name"* of each command within the implementation along with the messages unknown-choice used by ⟨*keys*⟩ that use .choice:n in their implementation and the messages cmd-key-unknown, cmd-key-value-unknown used by the ⟨*key*⟩ unknown in commands that use [⟨*key* = *val*⟩].

```
\str_new:N  \l__spintent_command_name_str
\msg_new:nnn { spintent } { unknown-choice }
  {
    The~value~'#3'~for~'#1'~key~is~invalid~use~('#2').
  }
\msg_new:nnnn { spintent } { cmd-key-unknown }
  {
    The~key~'#1'~is~unknown~by~command~
    \c_backslash_str \l__spintent_command_name_str \c_space_tl
    and~is~being~ignored~\msg_line_context:.
  }
  {
    The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
    does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
\msg_new:nnnn { spintent } { cmd-key-value-unknown }
  {
    The~key~'#1=#2'~is~unknown~by~command~
    \c_backslash_str \l__spintent_command_name_str \c_space_tl
    and~is~being~ignored~\msg_line_context:.
  }
  {
    The~command~\c_backslash_str \l__spintent_command_name_str \c_space_tl
    does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
```

©2026 by Pablo González L

```
166   }
```

(*End of definition for* `\l__spintent_command_name_str` *and others.*)

`\__spintent_unknown_keys:n`
`\__spintent_unknown_keys:nn`

The internal functions `\__spintent_unknown_keys:n` and `\__spintent_unknown_keys:nn` used by the ⟨*key*⟩ unknown.

```
167 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:n
168   {
169     \exp_args:NV \__spintent_unknown_keys_cmd:nn \l_keys_key_str {#1}
170   }
171 \cs_new_protected:Nn \__spintent_unknown_keys_cmd:nn
172   {
173     \tl_if_blank:nTF {#2}
174       { \msg_error:nnn { spintent } { cmd-key-unknown } {#1} }
175       { \msg_error:nnnn { spintent } { cmd-key-value-unknown } {#1} {#2} }
176   }
```

(*End of definition for* `\__spintent_unknown_keys:n` *and* `\__spintent_unknown_keys:nn`.)

11.10 Definition of core base variables

`\__spintent_luafun_clean_split_and_set:n`
`\__spintent_luafun_clean_split_and_set:V`

The function `\__spintent_luafun_clean_split_and_set:n` defined in the Lua module `spintent.lua`, is common to several commands in this implementation. It analyzes, processes, cleans and splits the {⟨*argument*⟩} passed to commands using internal functions defined in the Lua module `spintent.lua`. It isolates signs, integer components, decimal separators, decimal parte, decimal periodic part, exponents and final physical *"units"*, assigning them directly to `expl3` variables.

This function assigns the states of the variables '`_str`' or '`_tl`' described below to `true`, `false`, `error` or *stores* information according to the role it fulfills within the implementation.

(*End of definition for* `\__spintent_luafun_clean_split_and_set:n`.)

`\l__spintent_luaset_sign_tl`
`\l__spintent_luaset_part_int_tl`
`\l__spintent_luaset_dec_sep_tl`
`\l__spintent_luaset_part_dec_tl`
`\l__spintent_luaset_part_period_tl`
`\l__spintent_luaset_has_integer_str`
`\l__spintent_luaset_has_natural_str`

The variable `\l__spintent_luaset_sign_tl` *stores* the explicit '+' or '−' sign if present. The variable `\l__spintent_luaset_part_int_tl` *stores* the raw digit sequence of the *"integer part"*. The variable `\l__spintent_luaset_dec_sep_tl` *stores* the matched '`,`' or '`.`' used as the decimal separator. The variable `\l__spintent_luaset_part_dec_tl` *stores* the raw digits of the *"finite decimal part"* that follow the separator, and the variable `\l__spintent_luaset_part_period_tl` *stores* the digits of the *"infinite periodic decimal part"* that follow a semicolon '`;`'.

The variable `\l__spintent_luaset_has_integer_str` *store* the state `true` when it is *only* an *"integer"*. The variable `\l__spintent_luaset_has_natural_str` *store* the state `true` when it is *only* an *"natural number"*.

```
177 \tl_new:N  \l__spintent_luaset_sign_tl
178 \tl_new:N  \l__spintent_luaset_part_int_tl
179 \tl_new:N  \l__spintent_luaset_dec_sep_tl
180 \tl_new:N  \l__spintent_luaset_part_dec_tl
181 \tl_new:N  \l__spintent_luaset_part_period_tl
182 \str_new:N \l__spintent_luaset_has_integer_str
183 \str_new:N \l__spintent_luaset_has_natural_str
```

(*End of definition for* `\l__spintent_luaset_sign_tl` *and others.*)

`\l__spintent_luaset_only_part_int_str`
`\l__spintent_luaset_only_part_dec_str`
`\l__spintent_luaset_only_part_period_str`
`\l__spintent_luaset_format_part_int_str`
`\l__spintent_luaset_format_part_dec_str`

The literal entries of the numbers that we will pass to the *intent function* `:number` of `\MathMLintent` are *stored* in `\l__spintent_luaset_only_part_int_str` for the *"integer part"*, in `\l__spintent_luaset_only_part_dec_` for the *"finite decimal part"*, and in `\l__spintent_luaset_only_part_period_str` for the *"infinite periodic decimal part"*.

Formatted versions (`cifras-sep=true`) are *stored* in `\l__spintent_luaset_format_part_int_str` for the *"integer part"* and `\l__spintent_luaset_format_part_dec_str` for the *"finite decimal part"*.

```
184 \str_new:N \l__spintent_luaset_only_part_int_str
185 \str_new:N \l__spintent_luaset_only_part_dec_str
186 \str_new:N \l__spintent_luaset_only_part_period_str
187 \str_new:N \l__spintent_luaset_format_part_int_str
188 \str_new:N \l__spintent_luaset_format_part_dec_str
```

(*End of definition for* `\l__spintent_luaset_only_part_int_str` *and others.*)

`\l__spintent_luaset_millons_str`
`\l__spintent_luaset_decimal_period_str`
`\l__spintent_luaset_not_decimal_period_str`
`\l__spintent_luaset_not_decimal_not_period_str`
`\l__spintent_luaset_dec_is_all_zeros_str`
`\l__spintent_luaset_has_exponent_str`
`\l__spintent_luaset_exponent_tl`

The flag variable `\l__spintent_luaset_millons_str` indicates whether the integer part translates exactly to a multiple of one million when passed to `\MathMLintent`. To properly structure the reading and writing of numbers in `MathML`, we use the flag variables `\l__spintent_luaset_decimal_period_str`, `\l__spintent_luaset_not_decimal_period_str`, and `\l__spintent_luaset_not_decimal_not_period_st` The flag variable `\l__spintent_luaset_dec_is_all_zeros_str` monitors whether the *"finite decimal part"* is completely zero (useful for *"pure periodic decimals"*).

The flag variable \l__spintent_luaset_has_exponent_str indicates whether active *"scientific notation"* was detected at the end of the entered number, passed with 'e' or 'E'. If true, it will store the digits of the exponent in \l__spintent_luaset_exponent_tl.

```
\str_new:N \l__spintent_luaset_millons_str
\str_new:N \l__spintent_luaset_decimal_period_str
\str_new:N \l__spintent_luaset_not_decimal_period_str
\str_new:N \l__spintent_luaset_not_decimal_not_period_str
\str_new:N \l__spintent_luaset_dec_is_all_zeros_str
\str_new:N \l__spintent_luaset_has_exponent_str
\tl_new:N  \l__spintent_luaset_exponent_tl
```

(*End of definition for* \l__spintent_luaset_millons_str *and others.*)

\l__spintent_luaset_has_units_str
\l__spintent_luaset_status_str
\l__spintent_luaset_arg_above_tl
\l__spintent_luaset_arg_below_tl
\l__spintent_luaset_denom_has_number_str

Since the function __spintent_luafun_clean_split_and_set:n is common to \spnum (§11.12) and \spunit (§11.13) (as well as others), it determines the following unit-related variables after processing the {⟨*arguments*⟩} via the Lua module spintent.lua.

First, it checks if there is any residual text *after* the processed number that is a candidate for physical *"units"*. It sets the flag variable \l__spintent_luaset_has_units_str accordingly. Following this, it verifies if this text contains more than one "/". It then sets the variable \l__spintent_luaset_status_str to valid or multislash. If the structure is valid, it splits the unit into \l__spintent_luaset_arg_above_tl and \l__spintent_luaset_arg_below_tl. Finally, it analyzes \l__spintent_luaset_arg_below_tl; if the denominator starts with a number, it sets \l__spintent_luaset_denom_has_number_str to true, otherwise to false.

```
\str_new:N \l__spintent_luaset_has_units_str
\str_new:N \l__spintent_luaset_status_str
\tl_new:N  \l__spintent_luaset_arg_above_tl
\tl_new:N  \l__spintent_luaset_arg_below_tl
\str_new:N \l__spintent_luaset_denom_has_number_str
```

(*End of definition for* \l__spintent_luaset_has_units_str *and others.*)

11.11 Common messages for text mode and math mode

Messages shared by the different commands for text mode or math mode.

```
\msg_new:nnnn { spintent } { need-math-mode }
  {
    Math~mode~required~for~'#1'.
  }
  {
    Please~wrap~the~command~inside~$...$~or~a~math~environment.
  }
\msg_new:nnnn { spintent } { need-text-mode }
  {
    Text~mode~required~for~'#1'.
  }
  {
    Please~move~the~command~outside~of~$...$~or~the~math~environment.
  }
```

11.12 The command \spnum

The user command \spnum is *first* of *"the big four"* provided by the spintent package and is fundamental to the implementation of other commands. It handles the processing of numerical input, scientific notation, and units of measurement, using the Lua module spintent.lua, which is responsible for separating and assigning variables defined in expl3.

From the expl3 side, we handle the printing logic and the intent *functions* passed to \MathMLintent to achieve a valid MathML structure that is accepted by MathCAT.

11.12.1 Definition of keys and variables for \spnum

\l__spintent_spnum_read_terse_bool
\l__spintent_spnum_read_period_sep_str
\l__spintent_spnum_notation_sym_tl
\l__spintent_spnum_intent_read_sign_str
\l__spintent_spnum_intent_read_dec_sep_str

The variable \l__spintent_spnum_read_terse_bool handled by the key read-sign, controls the intent passed to \MathMLintent to spoken the '+' or '−'. If set to clear, it will say *"positivo"* or *"negativo"* *after* reading the number; if set to terse, it will say *"más"* or *"menos"* *before* reading the number.

The variable \l__spintent_spnum_read_period_sep_str handled by the key read-period-sep, controls how the decimal separator ';' (periodic part) is printed and spoken when the {⟨*argument*⟩} is a *pure decimal periodic* and is passed the intent to \MathMLintent. If set to coma, it will say *"coma"* and print ',', if set to punto it will say *"punto"* and print '.'.

The variable \l__spintent_spnum__notation_sym_tl, handled by the key notation-sym, determines whether '×' will be printed when set to times or '·' when set to cdot.

The variable `\l__spintent_spnum_intent_read_sign_str` will *store* the spoken translation used in the `intent` function passed to `\MathMLintent` for the '+' or '−' according to the value passed to the key `read-sign`.

The variable `\l__spintent_spnum_intent_read_dec_sep_str` *stores* the spoken translation that will be used in the `intent` function passed to `\MathMLintent` for reading the decimal separation as a *"coma"* or a *"punto"* according to the value found in the variable `\l__spintent_luaset_dec_sep_tl`.

```
\bool_new:N \l__spintent_spnum_read_terse_bool
\str_new:N  \l__spintent_spnum_read_period_sep_str
\tl_new:N   \l__spintent_spnum_notation_sym_tl
\str_new:N  \l__spintent_spnum_intent_read_sign_str
\str_new:N  \l__spintent_spnum_intent_read_dec_sep_str
```

(*End of definition for* `\l__spintent_spnum_read_terse_bool` *and others.*)

`cifras-sep` `read-sign` `read-period-sep` `notation-sym`

Now we add the keys `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym`.

```
\keys_define:nn { spintent/spnum }
  {
    cifras-sep                .bool_set:N = \l__spintent_spnum_cifras_sep_bool,
    cifras-sep                .initial:n = true,
    cifras-sep                .value_required:n = true,
    read-sign                 .choices:nn =
      { terse , clear }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \bool_set_true:N  \l__spintent_spnum_read_terse_bool }
            {2} { \bool_set_false:N \l__spintent_spnum_read_terse_bool }
          }
      },
    read-sign                 .initial:n  = terse,
    read-sign                 .value_required:n = true,
    read-sign / unknown       .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { read-sign } { terse , ~clear } { \exp_not:n {#1} },
    read-period-sep           .choices:nn =
      { coma , punto }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { coma } }
            {2} { \str_set:Nn \l__spintent_spnum_read_period_sep_str { punto } }
          }
      },
    read-period-sep           .initial:n  = coma,
    read-period-sep           .value_required:n = true,
    read-period-sep / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { read-period-sep } { coma, ~punto } { \exp_not:n {#1} },
    notation-sym              .choices:nn =
      { times , cdot }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \times } }
            {2} { \tl_set:Nn \l__spintent_spnum_notation_sym_tl { \cdot } }
          }
      },
    notation-sym              .initial:n  = cdot,
    notation-sym              .value_required:n = true,
    notation-sym / unknown    .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { notation-sym } { times , ~ cdot } { \exp_not:n {#1} },
    unknown                   .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spnum }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `cifras-sep` *and others.*)

11.12.2 Internal functions for \spnum

__spintent_spnum_render_part:n
__spintent_spnum_render_int:
__spintent_spnum_render_dec:

The function __spintent_spnum_render_part:n processes the {⟨*argument*⟩} passed in '#1' used by the functions __spintent_spnum_render_int: and __spintent_spnum_render_dec: to define the *intent function* :number for the integer and decimal part passed to \MathMLintent. It evaluates whether the key cifras-sep is active or not.

```
\cs_new_protected:Nn \__spintent_spnum_render_part:n
  {
    \MathMLintent { \str_use:c { l__spintent_luaset_only_part_#1_str } :number }
      {
        \bool_if:NTF \l__spintent_spnum_cifras_sep_bool
          { \str_use:c { l__spintent_luaset_format_part_#1_str } }
          { \tl_use:c { l__spintent_luaset_part_#1_tl } }
      }
  }
\cs_new_protected:Nn \__spintent_spnum_render_int:
  {
    \__spintent_spnum_render_part:n { int }
  }
\cs_new_protected:Nn \__spintent_spnum_render_dec:
  {
    \__spintent_spnum_render_part:n { dec }
  }
```

(*End of definition for* __spintent_spnum_render_part:n, __spintent_spnum_render_int:, *and* __spintent_spnum_render_dec:.)

__spintent_spnum_render_only_period:
__spintent_spnum_print_period_bar:
__spintent_spnum_render_period_sep:

The function __spintent_spnum_render_only_period: isolates the *"infinite periodic decimal part"* from the *"finite decimal part"*, assigning the *intent function* :number passed to \MathMLintent which will be used by \MathMLarg in the function __spintent_spnum_print_period_bar:.

```
\cs_new_protected:Nn \__spintent_spnum_render_only_period:
  {
    \MathMLintent { \l__spintent_luaset_only_part_period_str :number }
      {
        \l__spintent_luaset_part_period_tl
      }
  }
```

The function __spintent_spnum_print_period_bar: inserts the function __spintent_spnum_render_only_p as an argument within the *intent function* _periódico:postfix($dp) passed to \MathMLintent, generating the correct spoken for the *"infinite periodic decimal part"*.

```
\cs_new_protected:Nn \__spintent_spnum_print_period_bar:
  {
    \MathMLintent { _periódico:postfix($dp) }
      {
        {
          \overline
            {
              \MathMLarg { dp } { \__spintent_spnum_render_only_period: }
            }
        }
      }
  }
```

The function __spintent_spnum_render_period_sep: sets how the decimal separator handled by the key read-period-sep will be printed and spoken when a *"pure periodic decimal"* is passed.

```
\cs_new_protected:Nn \__spintent_spnum_render_period_sep:
  {
    \str_if_eq:VnTF \l__spintent_spnum_read_period_sep_str { coma }
      {
        \MathMLintent { _coma:prefix($pp) }
          {
            \mathord{,}
            \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
          }
       }
       {
         \MathMLintent { _punto:prefix($pp) }
           {
             \mathord{.}
             \MathMLarg { pp } { \__spintent_spnum_print_period_bar: }
           }
```

©2026 by Pablo González L

```
      }
  }
```

(*End of definition for* `\__spintent_spnum_render_only_period:`, `\__spintent_spnum_print_period_bar:`, *and* `\__spintent_spnum_render_period_sep:`.)

`\__spintent_spnum_render_decimal_sep:`

The function `\__spintent_spnum_render_decimal_sep:` parses the value of the variable `\l__spintent_luaset_` and sets the variable `\l__spintent_spnum_intent_read_dec_sep_str` with the function `_coma:prefix($dp)` for reading a *"coma"* or `_punto:prefix($dp)` for reading a *"punto"* to `\MathMLintent`. We print the decimal separator stored in `\l__spintent_luaset_dec_sep_tl` using `\mathord` to remove spaces around it and then use the command `\MathMLarg` where we insert `\__spintent_invisible_sep:` for compatibility with MathML and finally call the function `\__spintent_spnum_render_dec:` to print the finite decimal part.

```
\cs_new_protected:Nn \__spintent_spnum_render_decimal_sep:
  {
    \str_if_eq:VnTF \l__spintent_luaset_dec_sep_tl { , }
      { \str_set:Nn \l__spintent_spnum_intent_read_dec_sep_str { _coma:prefix($dp) } }
      { \str_set:Nn \l__spintent_spnum_intent_read_dec_sep_str { _punto:prefix($dp) } }
    \MathMLintent { \l__spintent_spnum_intent_read_dec_sep_str }
      {
        \mathord { \l__spintent_luaset_dec_sep_tl }
        \MathMLarg { dp }
          {
            \__spintent_invisible_sep:
            \__spintent_spnum_render_dec:
          }
      }
    % period part
    \str_if_eq:VnT \l__spintent_luaset_decimal_period_str { true }
      {
        \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
          {
            \__spintent_invisible_sep:
            \__spintent_spnum_print_period_bar:
          }
          {
            \MathMLintent { _con:prefix($pnum) }
              {
                \__spintent_invisible_sep:
                \MathMLarg { pnum }
                  {
                    \__spintent_spnum_print_period_bar:
                  }
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_spnum_render_decimal_sep:`.)

`\__spintent_spnum_print_part_dec:`
`\__spintent_spnum_print_part_period:`

The function `\__spintent_spnum_print_part_dec:` checks if a decimal part exists. If it does, it determines whether the decimal separator is a comma or a period to set the correct speech intent (*"coma"* or *"punto"*). It then prints the separator, renders the decimal digits, and if a periodic sequence follows, it decides how to attach the periodic bar.

The function `\__spintent_spnum_print_part_period:` handles cases where a periodic part exists. It evaluates if it should render the periodic separator directly, particularly when the number has a periodic part but lacks a standard decimal part.

```
\cs_new_protected:Nn \__spintent_spnum_print_part_dec:
  {
    \tl_if_empty:NF \l__spintent_luaset_part_dec_tl
      {
        \__spintent_spnum_render_decimal_sep:
      }
  }
\cs_new_protected:Nn \__spintent_spnum_print_part_period:
  {
    \tl_if_empty:NF \l__spintent_luaset_part_period_tl
      {
        \str_if_eq:VnTF \l__spintent_luaset_not_decimal_period_str { true }
          { \__spintent_spnum_render_period_sep: }
          {
```

```
            \tl_if_empty:NT \l__spintent_luaset_part_dec_tl
              {
                \__spintent_spnum_render_period_sep:
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_spnum_print_part_dec:` *and* `\__spintent_spnum_print_part_period:`.)

`\__spintent_spnum_print_integer:`
`\__spintent_spnum_print_num_start:`
`\__spintent_spnum_print_number_core:`

The function `\__spintent_spnum_print_integer:` checks if the number consists only of an integer. If so, it simply renders it. If a decimal or periodic part exists, it sets up a MathML intent to link the integer part (respecting digit separators if enabled) to the subsequent decimal or periodic functions.

The function `\__spintent_spnum_print_num_start:` checks if an explicit sign is present. If it finds one, it evaluates the terse reading flag to decide whether the sign should be read as a prefix (*"más"* or *"menos"*) or as a postfix (*"positivo"* or *"negativo"*), and wraps the intent around the rest of the number.

The function `\__spintent_spnum_print_number_core:` acts as the main wrapper. It triggers the sign and base number evaluation, and then checks if a scientific exponent is present. If it is, it appends the correct base-10 notation symbol and exponent to complete the sequence.

Note for `\__spintent_invisible_sep:`.

```
\cs_new_protected:Nn \__spintent_spnum_print_integer:
  {
    \str_if_eq:VnTF \l__spintent_luaset_not_decimal_not_period_str { true }
      { \__spintent_spnum_render_int: }
      {
        \MathMLintent { \l__spintent_luaset_only_part_int_str :number:prefix($next) }
          {
            \bool_if:NTF \l__spintent_spnum_cifras_sep_bool
              { \l__spintent_luaset_format_part_int_str }
              { \l__spintent_luaset_part_int_tl }
            \MathMLarg { next }
              {
                \tl_if_empty:NTF \l__spintent_luaset_part_dec_tl
                  { \__spintent_spnum_print_part_period: }
                  { \__spintent_spnum_print_part_dec: }
              }
          }
      }
  }
\cs_new_protected:Nn \__spintent_spnum_print_num_start:
  {
    \tl_if_empty:NTF \l__spintent_luaset_sign_tl
      {
        \__spintent_invisible_sep: % need
        \__spintent_spnum_print_integer:
      }
      {
        \bool_if:NTF \l__spintent_spnum_read_terse_bool
          {
            \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
              { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _más:prefix($n)} }
              { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _menos:prefix($n)} }
          }
          {
            \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
              { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _positivo:postfix($n) } }
              { \str_set:Nn \l__spintent_spnum_intent_read_sign_str { _negativo:postfix($n) } }
          }
        \__spintent_invisible_sep: % need
        \MathMLintent { \l__spintent_spnum_intent_read_sign_str }
          {
            \l__spintent_luaset_sign_tl
            \MathMLarg { n } { \__spintent_spnum_print_integer: }
          }
      }
  }
\cs_new_protected:Nn \__spintent_spnum_print_number_core:
  {
    \__spintent_spnum_print_num_start:
    \str_if_eq:VnT \l__spintent_luaset_has_exponent_str { true }
```

```
      {
        \MathMLintent { _por } { \l__spintent_spnum_notation_sym_tl }
        10^{ \l__spintent_luaset_exponent_tl }
      }
  }
```

(*End of definition for* `\__spintent_spnum_print_integer:`, `\__spintent_spnum_print_num_start:`, *and* `\__spintent_spnum_print_number_core:`.)

11.12.3 Definition of error messages

spnum-has-units
spnum-no-digits

We define two error messages for the `\spnum` command. The first "error" triggers if the user includes *"units"* in the {⟨*argument*⟩} passed in '`#1`', advising them to use the `\spunit` command instead. The second "error" is raised if the provided {⟨*argument*⟩} contains no numerical digits.

```
\msg_new:nnnn { spintent } { spnum-has-units }
  {
    The~\token_to_str:N \spnum \c_space_tl command~does~not~accept~units~('#1').
  }
  {
    Use~\token_to_str:N \spunit \c_space_tl if~you~need~to~print~numbers~with~units.
  }
\msg_new:nnnn { spintent } { spnum-no-digits }
  {
    The~\token_to_str:N \spnum \c_space_tl command~requires~a~valid~numerical~value.
  }
  {
    You~provided~'#1',~which~contains~no~digits.
  }
```

(*End of definition for* `spnum-has-units` *and* `spnum-no-digits`.)

11.12.4 Implementation of the `\spum` command

\spnum

The public user command `\spnum` accepts an optional [⟨*key = val*⟩] argument in '`#1`' and a mandatory {⟨*rational number*⟩} in '`#2`'. It first checks if it is executed inside math mode, raising an error if it is not. If correct, it opens a local group to apply the options and passes the input to Lua for parsing via `\__spintent_luafun_clean_split_and_set:n`.

Before printing, it validates the extracted data. It throws an error if it finds trailing units in the input or if the integer part is completely empty (no valid digits). If all checks pass, it calls `\__spintent_spnum_print_number_core:` to render the final number, and finally closes the group.

```
\NewDocumentCommand \spnum { O{} m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \keys_set:nn { spintent/spnum } { #1 }
          \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
          \str_if_eq:VnTF \l__spintent_luaset_has_units_str { true }
            { \msg_error:nnn { spintent } { spnum-has-units } { #2 } }
            {
              \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
                { \msg_error:nnn { spintent } { spnum-no-digits } { #2 } }
                { \__spintent_spnum_print_number_core: }
            }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spnum } }
  }
```

(*End of definition for* `\spnum`. *This function is documented on page 3.*)

11.13 The command `\spunit`

The user command `\spunit` is the *second* of *"the big four"* provided by the spintent package and is used to parse and print physical units and angular systems. It sends the {⟨*argument*⟩} to Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n` to separate and clean the {⟨*argument*⟩}.

This ensures that all *"units"* (such as SÍ fractions, products, or sexagesimal angles) are processed consistently by the `spintent.lua`.

11.13.1 Definition of variables

\l__spintent_spunit_luaset_canonical_str
_spintent_spunit_luaset_lookup_status_str
\l__spintent_spunit_luaset_read_str
\l__spintent_spunit_luaset_is_compact_str
l__spintent_spunit_luaset_compact_exp_str
spintent_spunit_luaset_is_sexagesimal_str
\l__spintent_spunit_luaset_status_str

These string variables store the data returned by the Lua module `spintent.lua`. They hold the canonical *"unit symbol"*, the dictionary lookup status, the correct speech text for NVDA/MathCAT, and specific flags that indicate if the *"unit"* is compact or represents a sexagesimal angle.

```
\str_new:N  \l__spintent_spunit_luaset_canonical_str
\str_new:N  \l__spintent_spunit_luaset_lookup_status_str
\str_new:N  \l__spintent_spunit_luaset_read_str
\str_new:N  \l__spintent_spunit_luaset_is_compact_str
\str_new:N  \l__spintent_spunit_luaset_compact_exp_str
\str_new:N  \l__spintent_spunit_luaset_is_sexagesimal_str
\str_new:N  \l__spintent_spunit_luaset_status_str
```

(*End of definition for* `\l__spintent_spunit_luaset_canonical_str` *and others.*)

\l__spintent_spunit_exponent_tl
\l__spintent_spunit_unit_name_tl
\l__spintent_spunit_read_slash_str
\l__spintent_spunit_exp_seq
\l__spintent_spunit_mult_seq
\l__spintent_spunit_is_mult_bool
\l__spintent_spunit_read_cdot_bool

These variables are used to process and format the *"unit"*. The sequences `\l__spintent_spunit_mult_seq` and `\l__spintent_spunit_exp_seq` split compound *"units"* at multiplication '`*`' or exponent '`^`' boundaries.

The boolean variables control specific rendering options, such as whether the multiplication dot '`\cdot`' is explicitly read aloud or silently skipped.

```
\tl_new:N   \l__spintent_spunit_exponent_tl
\tl_new:N   \l__spintent_spunit_unit_name_tl
\str_new:N  \l__spintent_spunit_read_slash_str
\seq_new:N  \l__spintent_spunit_exp_seq
\seq_new:N  \l__spintent_spunit_mult_seq
\bool_new:N \l__spintent_spunit_is_mult_bool
\bool_new:N \l__spintent_spunit_read_cdot_bool
```

(*End of definition for* `\l__spintent_spunit_exponent_tl` *and others.*)

11.13.2 Definition of error messages

spunit-invalid-unit
spunit-no-units
inline-numeric-denominator

We define three error messages for the `\spunit` command. The first "error" triggers if the *"unit expression"* provided in '`#1`' is unknown or malformed. The second "error" occurs if no *"units"* are found in the {⟨*argument*⟩}, advising the user to use `\spnum` for pure numbers. The third "error" is raised if a numeric coefficient is illegally placed in the denominator of an inline fraction (after a slash).

```
\msg_new:nnnn { spintent } { spunit-invalid-unit }
  {
    The~expression~'#1'~contains~an~unknown~or~malformed~unit~structure.
  }
  {
    Check~spelling~or~verify~the~unit~is~defined~in~the~dictionary.
  }
\msg_new:nnnn { spintent } { spunit-no-units }
  {
    The~\token_to_str:N \spunit \c_space_tl command~requires~at~least~one~unit~('#1').
  }
  {
    Use~\token_to_str:N \spnum \c_space_tl if~you~only~need~to~format~a~pure~number.
  }
\msg_new:nnnn { spintent } { inline-numeric-denominator }
  {
    Illegal~numeric~coefficient~in~inline~denominator~'#1'.
  }
  {
    The~inline~mode~of~\token_to_str:N \spunit \c_space_tl
    does~not~allow~numbers~after~the~slash~(/).
  }
```

(*End of definition for* `spunit-invalid-unit`, `spunit-no-units`, *and* `inline-numeric-denominator`.)

11.13.3 Definition of keys for `\spunit`

read-cdot
read-slash
cifras-sep
read-period-sep
notation-sym

Now we add the keys `print-cdot`, `read-cdot`, `read-slash`, `cifras-sep`, `read-period-sep` and `notation-sym`.

```
\keys_define:nn { spintent/spunit }
  {
    print-cdot          .bool_set:N = \l__spintent_spunit_print_cdot_bool,
    print-cdot          .initial:n  = false,
    print-cdot          .value_required:n = true,
    read-cdot           .bool_set:N = \l__spintent_spunit_read_cdot_bool,
    read-cdot           .initial:n  = false,
    read-cdot           .value_required:n = true,
    read-slash          .choices:nn =
```

```
      { por , partido-por }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \str_set:Nn \l__spintent_spunit_read_slash_str { por } }
            {2} { \str_set:Nn \l__spintent_spunit_read_slash_str { partido-por } }
          }
      },
    read-slash           .initial:n  = por,
    read-slash           .value_required:n = true,
    read-slash / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { read-slash } { por, ~partido-por } { \exp_not:n {#1} },
    cifras-sep           .meta:nn = { spintent/spnum } { cifras-sep = {#1} },
    cifras-sep           .value_required:n = true,
    read-period-sep      .meta:nn = { spintent/spnum } { read-period-sep = {#1} },
    read-period-sep      .value_required:n = true,
    notation-sym         .meta:nn = { spintent/spnum } { notation-sym = {#1} },
    notation-sym         .value_required:n = true,
    unknown              .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spunit }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `read-cdot` *and others.*)

11.13.4 Internal functions for `\spunit`

`\__spintent_spunit_format_canonical:`
`\__spintent_spunit_process_base_exp:n`

The function `\__spintent_spunit_format_canonical:` handles the visual formatting of the *"unit symbol"*. It specifically checks for the *liter symbol* 'ℓ' to print it as `\ell`, and formats all other valid *"units"* in standard upright text using `\mathrm`.

The function `\__spintent_spunit_process_base_exp:n` evaluates a *"single unit"* along with its exponent. It first looks up the unit '`#1`' in the dictionary. If the *"unit"* is not found, it flags it as invalid. If found, it handles the spacing and multiplication dot '`\cdot`' intents, and then renders the base unit and its exponent (either using a pre-defined compact exponent from Lua or by manually splitting the string at the '`^`' symbol).

```
\cs_new_protected:Nn \__spintent_spunit_format_canonical:
  {
    \str_case:VnF \l__spintent_spunit_luaset_canonical_str
      {
        { ℓ } { \ell }
      }
      { \mathrm { \l__spintent_spunit_luaset_canonical_str } }
  }
\cs_new_protected:Nn \__spintent_spunit_process_base_exp:n
  {
    \tl_clear:N \l__spintent_spunit_exponent_tl
    \__spintent_luafun_spunit_lookup_alias:n { #1 }
    \str_if_eq:VnTF \l__spintent_spunit_luaset_lookup_status_str { notfound }
      {
        \str_set:Nn \l__spintent_luaset_status_str { invalid }
      }
      {
        \bool_if:NF \l__spintent_spunit_is_mult_bool
          {
            \bool_if:NTF \l__spintent_spunit_print_cdot_bool
              {
                \bool_if:NTF \l__spintent_spunit_read_cdot_bool
                  { \MathMLintent { por } { \cdot } }
                  { \MathMLintent { :silent } { \cdot } }
              }
              {
                \bool_if:NTF \l__spintent_spunit_read_cdot_bool
                  { \MathMLintent { por } { \, } }
                  { \, }
              }
          }
        \str_if_eq:VnTF \l__spintent_spunit_luaset_is_compact_str { true }
          {
            {
              \MathMLintent { \l__spintent_spunit_luaset_read_str }
```

©2026 by Pablo González L

```
                {
                  \__spintent_spunit_format_canonical:
                }
            }
            ^ { \l__spintent_spunit_luaset_compact_exp_str }
          }
          {
            \seq_set_split:Nnn \l__spintent_spunit_exp_seq { ^ } { #1 }
            \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_unit_name_tl
            \seq_if_empty:NF \l__spintent_spunit_exp_seq
              {
                \seq_pop_left:NN \l__spintent_spunit_exp_seq \l__spintent_spunit_exponent_tl
              }
            \MathMLintent { \l__spintent_spunit_luaset_read_str }
              {
                \__spintent_spunit_format_canonical:
              }
            \tl_if_empty:NF \l__spintent_spunit_exponent_tl
              {
                ^ { \l__spintent_spunit_exponent_tl }
              }
          }
        \bool_set_false:N \l__spintent_spunit_is_mult_bool
      }
  }
```

(*End of definition for* `\__spintent_spunit_format_canonical:` *and* `\__spintent_spunit_process_base_exp:n`.)

`\__spintent_spunit_process_mult:n` `\__spintent_spunit_process_mult:V`

The function `\__spintent_spunit_process_mult:n` handles compound *"units"* joined by an explicit multiplication star '`*`'. It splits the {⟨*argument*⟩} at each *"star symbol"* and applies `\__spintent_spunit_process_base_ex` to evaluate and render each resulting unit individually.

```
\cs_new_protected:Nn \__spintent_spunit_process_mult:n
  {
    \bool_set_true:N \l__spintent_spunit_is_mult_bool
    \seq_set_split:Nnn \l__spintent_spunit_mult_seq { * } { #1 }
    \seq_map_function:NN \l__spintent_spunit_mult_seq \__spintent_spunit_process_base_exp:n
  }
\cs_generate_variant:Nn \__spintent_spunit_process_mult:n { V }
```

(*End of definition for* `\__spintent_spunit_process_mult:n`.)

`\__spintent_print_spunit_hierarchy:`

The function `\__spintent_print_spunit_hierarchy:` handles the layout of inline *"unit"* fractions. It first processes the *"units"* before the slash (the numerator). If a denominator exists, it prints the slash / '/' with its corresponding speech intent, and then processes the remaining *"units"*.

```
\cs_new_protected:Nn \__spintent_print_spunit_hierarchy:
  {
    \tl_if_empty:NF \l__spintent_luaset_arg_above_tl
      {
        \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_above_tl
        \tl_if_empty:NF \l__spintent_luaset_arg_below_tl
          {
            \MathMLintent { \l__spintent_spunit_read_slash_str } { / }
            \__spintent_spunit_process_mult:V \l__spintent_luaset_arg_below_tl
          }
      }
  }
```

(*End of definition for* `\__spintent_print_spunit_hierarchy:`.)

`\__spintent_spunit_safe_exec:n`

The function `\__spintent_spunit_safe_exec:n` performs the main validation and layout for *"units"*. It first checks if the {⟨*argument*⟩} has units, is a valid expression, and does not contain illegal numbers in the denominator, throwing the appropriate "errors" if any check fails.

If the unit is valid, it checks if it is a sexagesimal angle (degrees, minutes, or seconds). For sexagesimal *"units"*, it prints the number and the symbol *"without any space in between"*, applying a slight negative space '`\!`' for `arcseconds`. For standard units, it prints the number followed by a thin space '`\,`' and then processes the unit sequence.

```
\cs_new_protected:Nn \__spintent_spunit_safe_exec:n
  {
    \str_if_eq:VnTF \l__spintent_luaset_has_units_str { false }
      { \msg_error:nnn { spintent } { spunit-no-units } { #1 } }
      {
```

```
      \str_if_eq:VnTF \l__spintent_luaset_status_str { valid }
        {
          \str_if_eq:VnTF
            \l__spintent_luaset_denom_has_number_str { true }
            {
              \msg_error:nnn { spintent }
                { inline-numeric-denominator } { #1 }
            }
            {
              \__spintent_luafun_spunit_lookup_alias:V \l__spintent_luaset_arg_above_tl
              \str_if_eq:VnTF \l__spintent_spunit_luaset_is_sexagesimal_str { true }
                {
                  \tl_if_empty:NF \l__spintent_luaset_part_int_tl
                    {
                      \__spintent_spnum_print_number_core:
                    }
                  \MathMLintent { \l__spintent_spunit_luaset_read_str }
                    {
                      \mathord
                        {
                          \str_case:Vn \l__spintent_spunit_luaset_canonical_str
                            {
                              { " } { \prime \! \prime }
                              { ' } { \prime           }
                              { ° } { °                }
                            }
                        }
                    }
                }
                {
                  \tl_if_empty:NF \l__spintent_luaset_part_int_tl
                    {
                      \__spintent_spnum_print_number_core: \,
                    }
                  \__spintent_print_spunit_hierarchy:
                }
            }
        }
        { \msg_error:nnn { spintent } { spunit-invalid-unit } { #1 } }
    }
}
```

(*End of definition for* `\__spintent_spunit_safe_exec:n`.)

11.13.5 Implementation of the `\spunit` command

`\spunit` The public user command `\spunit` accepts an optional `[`⟨*key = val*⟩`]` argument in '`#1`' and a mandatory `{`⟨*unit expression*⟩`}` in '`#2`'. It first checks if it is executed inside *math mode*, raising an "error" if it is not. If correct, it opens a local group to apply the options and passes the `{`⟨*argument*⟩`}` to Lua module for parsing via `\__spintent_luafun_clean_split_and_set:n` function.

Finally, it delegates the execution to the function `\__spintent_spunit_safe_exec:n` to perform the safety checks and render the parsed data, before closing the group.

```
\NewDocumentCommand \spunit { O{} m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \keys_set:nn { spintent/spunit } { #1 }
          \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #2 } }
          \__spintent_spunit_safe_exec:n { #2 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spunit } }
  }
```

(*End of definition for* `\spunit`. *This function is documented on page* 4.)

11.14 The commands `\spnewunit` and `\spunitalias`

The user commands `\spnewunit` and `\spunitalias` allow users to add new custom *"units"* or *"aliases"* to the package's dictionary. Similar to `\spunit`, these commands delegate the processing and validation to the `spintent.lua` via `\__spintent_luafun_define_custom_unit:nn` and `\__spintent_luafun_define_spunit`

During the definition, the Lua module checks if the new *"unit"* or *"alias"* is valid and ensures it does not conflict with existing *"units"*. It returns the result of this operation to TEX using the `\l__spintent_spunit_luaset_status_st` variable. If a collision is detected or the {⟨*argument*⟩} is invalid, the package throws the corresponding "error" message.

11.14.1 Definition of error messages

spnewunit-duplicate
spunitalias-duplicate
spunitalias-invalid-expr

We define three error messages for the creation of *"units"* and *"aliases"*. The first two "errors" trigger if the user attempts to define a *"new unit"* or alias passed in '`#1`' that already exists in the dictionary. The third "error" is raised by `\spunitalias` if the target expression provided in '`#2`' is invalid (for example, if it contains unknown *"units"* or incorrect syntax).

```
\msg_new:nnnn { spintent } { spnewunit-duplicate }
  {
    The~unit~symbol~or~alias~'#1'~is~already~defined!
  }
  {
    You~cannot~redefine~with~\token_to_str:N \spnewunit.
  }
\msg_new:nnnn { spintent } { spunitalias-duplicate }
  {
    The~alias~'#1'~is~already~defined~or~reserved!
  }
  {
    You~cannot~redefine~an~existing~unit~or~another~alias.
  }
\msg_new:nnnn { spintent } { spunitalias-invalid-expr }
  {
    The~expression~'#2'~for~alias~'#1'~is~invalid.
  }
  {
    It~must~contain~only~defined~units,~exponents~and~one~'/'.
  }
```

(*End of definition for* `spnewunit-duplicate`, `spunitalias-duplicate`, *and* `spunitalias-invalid-expr`.)

`\spnewunit`

The public command `\spnewunit` defines a new custom unit. It passes the new unit symbol '`#1`' and its speech text '`#2`' to Lua via `\__spintent_luafun_define_custom_unit:nn`. If the module reports that the unit already exists, it throws the corresponding duplicate error.

```
\NewDocumentCommand \spnewunit { m m }
  {
    \mode_if_math:TF
      {
        \msg_error:nnn { spintent } { need-text-mode } { \spnewunit }
      }
      {
        \__spintent_luafun_define_custom_unit:nn { #1 } { #2 }
        \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
          {
            \msg_error:nnn { spintent } { spnewunit-duplicate } { #1 }
          }
      }
  }
```

(*End of definition for* `\spnewunit`. *This function is documented on page* 6.)

`\spunitalias`

The public command `\spunitalias` creates a shortcut for a complex unit expression. It passes the new alias name '`#1`' and the target expression '`#2`' to Lua via `\__spintent_luafun_define_spunit_alias:nn`. It throws an error if the alias already exists, or if the provided target expression contains invalid syntax or unknown units.

```
\NewDocumentCommand \spunitalias { m m }
  {
    \mode_if_math:TF
      {
        \msg_error:nnn { spintent } { need-text-mode } { \spunitalias }
      }
      {
        \__spintent_luafun_define_spunit_alias:nn { #1 } { #2 }
        \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { duplicate }
          {
            \msg_error:nnn { spintent } { spunitalias-duplicate } { #1 }
          }
```

©2026 by Pablo González L

```
      \str_if_eq:VnT \l__spintent_spunit_luaset_status_str { invalid-expr }
        {
          \msg_error:nnnn { spintent } { spunitalias-invalid-expr } { #1 } { #2 }
        }
    }
}
```

(*End of definition for* `\spunitalias`. *This function is documented on page 6.*)

11.15 The command \spmoney

The user command `\spmoney` is the *third* of *"the big four"* provided by the spintent package, designed to format monetary values and currencies. It delegates {⟨*argument*⟩} parsing to the Lua module `spintent.lua` via `\__spintent_luafun_clean_split_and_set:n`. The module then retrieves the correct *"currency symbol"*, pluralization rules, and layout structures from the database.

The command accepts *optional arguments* to override the default *"currency symbol"* or enable *"subunit"* rendering. Finally, it constructs the MathML intent for MathCAT.

`\l__spintent_spmoney_luaset_symbol_str` `\l__spintent_spmoney_luaset_position_str` `\l__spintent_spmoney_luaset_sing_str` `\l__spintent_spmoney_luaset_plur_str` `\l__spintent_spmoney_luaset_conde_str` `\l__spintent_spmoney_luaset_subsing_str` `\l__spintent_spmoney_luaset_subplur_str` `\l__spintent_spmoney_luaset_print_iso_str` `\l__spintent_spmoney_luaset_currency_arg_str` `\l__spintent_spmoney_luaset_status_str`

These string variables store the currency data returned by the Lua module `spintent.lua`. Upon a successful database lookup, the module populates these variables with the *"currency symbol"*, its layout position (`pre` or `post`), the appropriate singular and plural names for both the main unit and subunits, and the execution status flags. TEX then uses this information to render the visual layout and construct the correct MathML speech intents.

```
\str_new:N  \l__spintent_spmoney_luaset_symbol_str
\str_new:N  \l__spintent_spmoney_luaset_position_str
\str_new:N  \l__spintent_spmoney_luaset_sing_str
\str_new:N  \l__spintent_spmoney_luaset_plur_str
\str_new:N  \l__spintent_spmoney_luaset_conde_str
\str_new:N  \l__spintent_spmoney_luaset_subsing_str
\str_new:N  \l__spintent_spmoney_luaset_subplur_str
\str_new:N  \l__spintent_spmoney_luaset_print_iso_str
\str_new:N  \l__spintent_spmoney_luaset_currency_arg_str
\str_new:N  \l__spintent_spmoney_luaset_status_str
```

(*End of definition for* `\l__spintent_spmoney_luaset_symbol_str` *and others.*)

`\l__spintent_spmoney_sub_units_bool` `\l__spintent_spmoney_intent_money_str` `\l__spintent_spmoney_intent_sign_str` `\l__spintent_spmoney_intent_sub_unit_str` `\l__spintent_spmoney_res_main_voice_str` `\l__spintent_spmoney_subnum_int` `\l__spintent_spmoney_extracted_curr_str` `\l__spintent_spmoney_extracted_num_tl`

The boolean variable tracks whether subunit rendering is enabled. The string variables assemble the specific speech text (intents) for NVDA/MathCAT, including the sign, the main currency, and the subunits. Finally, the remaining variables temporarily store the numeric value and currency code extracted from the user's input.

```
\bool_new:N \l__spintent_spmoney_sub_units_bool
\str_new:N  \l__spintent_spmoney_intent_money_str
\str_new:N  \l__spintent_spmoney_intent_sign_str
\str_new:N  \l__spintent_spmoney_intent_sub_unit_str
\str_new:N  \l__spintent_spmoney_res_main_voice_str
\int_new:N  \l__spintent_spmoney_subnum_int
\str_new:N  \l__spintent_spmoney_extracted_curr_str
\tl_new:N   \l__spintent_spmoney_extracted_num_tl
```

(*End of definition for* `\l__spintent_spmoney_sub_units_bool` *and others.*)

11.15.1 Definition of error messages

`currency-no-subunits` `invalid-currency`

We define two messages to handle invalid currency configurations. The first "error" warns the user if subunit rendering is requested for a currency that does not support fractional divisions (such as JPY or CLP), safely ignoring the flag. The second "error" is triggered if the provided currency identifier is not registered in the Lua database.

```
\msg_new:nnnn { spintent } { currency-no-subunits }
  {
    The~currency~'#1'~does~not~support~subunits.~
    The~option~'sub-units=true'~is~ignored.
  }
  {
    Monies~like~the~Japanese~Yen~(JPY)~or~Chilean~Peso~(CLP)~
    do~not~have~fractional~subunits~registered~in~Lua~module.~
  }
\msg_new:nnnn { spintent } { invalid-currency }
  {
    Invalid~currency~identifier~'#1'~in~\token_to_str:N \spmoney .
  }
  {
    The~provided~symbol,~alias,~or~ISO~code~does~not~exist~in~
    the~Lua~module.
  }
```

©2026 by Pablo González L

(*End of definition for* `currency-no-subunits` *and* `invalid-currency`.)

11.15.2 Auxiliary functions for \spmoney

__spintent_spmoney_set_currency:n
__spintent_spmoney_normalize_key:n
__spintent_spmoney_normalize_key:V

The function __spintent_spmoney_set_currency:n validates the currency identifier provided in '#1' against the database. It delegates the lookup to the helper function __spintent_spmoney_normalize_key:n, which interfaces with the Lua module spintent.lua. If the module returns a notfound status, an "error" is triggered. Otherwise, it stores the valid currency identifier in \l__spintent_spmoney_luaset_currency_arg_str for subsequent processing.

```
\cs_new_protected:Nn \__spintent_spmoney_set_currency:n
  {
    \__spintent_spmoney_normalize_key:n {#1}
    \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
      {
        \msg_error:nnn { spintent } { invalid-currency } {#1}
      }
      { \str_set:Nn \l__spintent_spmoney_luaset_currency_arg_str {#1} }
  }
\cs_new_protected:Nn \__spintent_spmoney_normalize_key:n
  {
    \__spintent_luafun_spmoney_normalize_key:n { #1 }
  }
\cs_generate_variant:Nn \__spintent_spmoney_normalize_key:n { V }
```

(*End of definition for* `\__spintent_spmoney_set_currency:n` *and* `\__spintent_spmoney_normalize_key:n`.)

11.15.3 Definition of keys for \spmoney

currency
sub-units
dolar-math
cifras-sep

Now we add the keys currency, sub-units, dolar-math and cifras-sep.

```
\keys_define:nn { spintent / spmoney }
  {
    currency     .code:n      = { \__spintent_spmoney_set_currency:n {#1} },
    currency     .initial:n   = { pesos },
    currency     .value_required:n = true,
    sub-units    .bool_set:N  = \l__spintent_spmoney_sub_units_bool,
    sub-units    .value_required:n = true,
    dolar-math   .bool_set:N  = \l__spintent_spmoney_dolar_math_bool,
    dolar-math   .initial:n   = { true },
    dolar-math   .value_required:n = true,
    cifras-sep   .meta:nn     = { spintent/spnum } { cifras-sep = {#1} },
    cifras-sep   .value_required:n = true,
    unknown      .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spmoney }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `currency` *and others.*)

11.15.4 Internal functions for \spmoney

__spintent_spmoney_validate_money:

The function __spintent_spmoney_validate_money: verifies if the parsed input contains an embedded currency identifier by checking \l__spintent_luaset_has_units_str. If a currency is detected, it extracts and normalizes the key from \l__spintent_luaset_arg_above_tl. It throws an "error" if the currency is not found in the database. If valid, it updates \l__spintent_spmoney_luaset_currency_arg_str and retrieves the full currency metadata from the spintent.lua via __spintent_luafun_spmoney_lookup_data:V.

```
\cs_new_protected:Nn \__spintent_spmoney_validate_money:
  {
    \str_if_eq:VnT \l__spintent_luaset_has_units_str { true }
      {
        \__spintent_spmoney_normalize_key:V \l__spintent_luaset_arg_above_tl
        \str_if_eq:VnTF \l__spintent_spmoney_luaset_status_str { notfound }
          { \msg_error:nne { spintent } { invalid-currency } { \l__spintent_luaset_arg_above_tl }
          {
            \str_set:NV
              \l__spintent_spmoney_luaset_currency_arg_str
              \l__spintent_luaset_arg_above_tl
          }
      }
    \str_set:Ne \l__spintent_spmoney_luaset_status_str { found }
    \__spintent_luafun_spmoney_lookup_data:V \l__spintent_spmoney_luaset_currency_arg_str
  }
```

(*End of definition for* `\__spintent_spmoney_validate_money:`.)

`\__spintent_spmoney_print_symbol:`

The function `\__spintent_spmoney_print_symbol:` renders the *"currency symbol"*. If `\l__spintent_spmoney_lu` is `true`, it outputs the ISO code using `\mathrm`. Otherwise, it prints the *"standard symbol"*. For dollar signs '$', it checks `\l__spintent_spmoney_dolar_math_bool` to determine whether to render it directly as a math character '$' or wrap it inside `\text`.

```
\cs_new_protected:Nn \__spintent_spmoney_print_symbol:
  {
    \str_if_eq:VnTF \l__spintent_spmoney_luaset_print_iso_str { true }
      { \mathrm { \l__spintent_spmoney_luaset_currency_arg_str } }
      {
        \str_if_eq:VnTF \l__spintent_spmoney_luaset_symbol_str { $ }
          {
            \bool_if:NTF \l__spintent_spmoney_dolar_math_bool
              { \$ } { \text { \$ } }
          }
          { \l__spintent_spmoney_luaset_symbol_str }
      }
  }
```

(*End of definition for* `\__spintent_spmoney_print_symbol:`.)

`\__spintent_spmoney_render_layout:`

The function `\__spintent_spmoney_render_layout:` determines the layout order of the *"currency symbol"* relative to the numeric value. It evaluates `\l__spintent_spmoney_luaset_position_str`; if set to `pre`, it renders the *"currency symbol" before* the number node. Otherwise, it places the number first, followed by the *"currency symbol"*.

```
\cs_new_protected:Nn \__spintent_spmoney_render_layout:
  {
    \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
      {
        \__spintent_spmoney_print_symbol:
        \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
      }
      {
        \MathMLarg { n } { \__spintent_spmoney_render_number_node: }
        \__spintent_spmoney_print_symbol:
      }
  }
```

(*End of definition for* `\__spintent_spmoney_render_layout:`.)

`\__spintent_spmoney_render_number_node:`
`pintent_spmoney_render_only_integer_node:`

The function `\__spintent_spmoney_render_number_node:` renders the full numeric value, encompassing both the integer and decimal components. Conversely, the function `\__spintent_spmoney_render_only_integer_` outputs exclusively the integer portion, omitting any decimal data.

```
\cs_new_protected:Nn \__spintent_spmoney_render_number_node:
  {
    \__spintent_spnum_render_int:
    \__spintent_spnum_print_part_dec:
  }
\cs_new_protected:Nn \__spintent_spmoney_render_only_integer_node:
  {
    \__spintent_spnum_render_int:
  }
```

(*End of definition for* `\__spintent_spmoney_render_number_node:` *and* `\__spintent_spmoney_render_only_integer_node:`.)

`\__spintent_spmoney_print_int:`
`\__spintent_spmoney_intent_int:`

The function `\__spintent_spmoney_print_int:` evaluates the presence of an explicit sign in `\l__spintent_luase` If a sign is detected, it assigns a prefix speech intent '`más`' or '`menos`' and embeds the sign and subsequent layout inside a `\MathMLintent`. If no sign is present, it routes execution based on whether subunits are enabled.

The function `\__spintent_spmoney_intent_int:` determines the correct grammatical inflection for the currency string (singular, plural, or the prepositional '`de`' variant for exact millions). It evaluates the integer component, decimal boundaries, and zero-fills to select the appropriate string register, applying it as a postfix `MathML` intent over the layout rendered by `\__spintent_spmoney_render_layout:`.

```
\cs_new_protected:Nn \__spintent_spmoney_print_int:
  {
    \tl_if_empty:NTF \l__spintent_luaset_sign_tl
      {
        \bool_if:NTF \l__spintent_spmoney_sub_units_bool
          { \__spintent_spmoney_set_and_print_sub_units: }
```

```
852           { \__spintent_spmoney_intent_int: }
853       }
854       {
855         \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
856           {
857             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _más:prefix ($n) }
858           }
859           {
860             \str_set:Nn \l__spintent_spmoney_intent_sign_str { _menos:prefix ($n) }
861           }
862         \__spintent_invisible_sep:
863         \MathMLintent { \l__spintent_spmoney_intent_sign_str }
864           {
865             \l__spintent_luaset_sign_tl
866             \MathMLarg { n }
867               {
868                 \bool_if:NTF \l__spintent_spmoney_sub_units_bool
869                   { \__spintent_spmoney_set_and_print_sub_units: }
870                   { \__spintent_spmoney_intent_int: }
871               }
872           }
873       }
874   }
875 \cs_new_protected:Nn \__spintent_spmoney_intent_int:
876   {
877     \tl_if_empty:NTF \l__spintent_luaset_dec_sep_tl
878       {
879         \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
880           {
881             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_sing_st
882           }
883           {
884             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_st
885           }
886         \str_if_eq:VnT \l__spintent_luaset_millons_str { true }
887           {
888             \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_conde_s
889           }
890       }
891       {
892         \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plur_str
893         \str_if_eq:VnT \l__spintent_luaset_millons_str { true }
894           {
895             \str_if_eq:VnTF \l__spintent_luaset_dec_is_all_zeros_str { true }
896               {
897                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_con
898               }
899               {
900                 \str_set:NV \l__spintent_spmoney_intent_money_str \l__spintent_spmoney_luaset_plu
901               }
902           }
903       }
904     \__spintent_invisible_sep: % need for MathCAT
905     \MathMLintent { \l__spintent_spmoney_intent_money_str :postfix($n) }
906       {
907         \__spintent_spmoney_render_layout:
908       }
909   }
```

(*End of definition for* `\__spintent_spmoney_print_int:` *and* `\__spintent_spmoney_intent_int:`.)

`\__spintent_spmoney_check_sub_units:`
`spintent_spmoney_set_and_print_sub_units:`

The function `\__spintent_spmoney_check_sub_units:` verifies whether the selected currency supports *"subunits"*. If the database returns empty subunit data, it triggers a warning and automatically disables the *"subunit"* rendering flag.

The function `\__spintent_spmoney_set_and_print_sub_units:` processes and formats the speech intents when *"subunits"* are active. It normalizes the decimal component (for instance, padding a single-digit decimal like '.5' to '50'), evaluates the correct singular or plural inflections for both the main *"currency"* and the *"subunits"*, and delegates the final MathML tree assembly to `\__spintent_spmoney_build_tree:`.

```
910 \cs_new_protected:Nn \__spintent_spmoney_check_sub_units:
911   {
912     \str_if_empty:NT \l__spintent_spmoney_luaset_subsing_str
```

```
      {
        \msg_warning:nne { spintent } { currency-no-subunits }
          { \l__spintent_spmoney_luaset_currency_arg_str }
        \bool_set_false:N \l__spintent_spmoney_sub_units_bool
      }
  }
\cs_new_protected:Nn \__spintent_spmoney_set_and_print_sub_units:
  {
    \int_compare:nTF { \str_count:N \l__spintent_luaset_only_part_dec_str_ == 1 }
      {
        \int_set:Nn \l__spintent_spmoney_subnum_int
          { \l__spintent_luaset_only_part_dec_str * 10 }
        \str_set:Ne \l__spintent_luaset_only_part_dec_str
          { \l__spintent_luaset_only_part_dec_str 0 }
      }
      {
        \int_set:Nn \l__spintent_spmoney_subnum_int
          { \l__spintent_luaset_only_part_dec_str }
      }
    \int_compare:nTF { \l__spintent_spmoney_subnum_int == 1 }
      {
        \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
          { \l__spintent_spmoney_luaset_subsing_str :postfix($sub) }
      }
      {
        \str_set:Ne \l__spintent_spmoney_intent_sub_unit_str
          { \l__spintent_spmoney_luaset_subplur_str :postfix($sub) }
      }
    \str_if_eq:VnTF \l__spintent_luaset_only_part_int_str { 1 }
      {
        \str_set:NV \l__spintent_spmoney_res_main_voice_str
          \l__spintent_spmoney_luaset_sing_str
      }
      {
        \str_set:NV \l__spintent_spmoney_res_main_voice_str
          \l__spintent_spmoney_luaset_plur_str
      }
    \str_if_eq:VnT \l__spintent_luaset_millons_str { true }
      {
        \str_set:NV \l__spintent_spmoney_res_main_voice_str
          \l__spintent_spmoney_luaset_conde_str
      }
    \__spintent_spmoney_build_tree:
  }
```

(*End of definition for* `\__spintent_spmoney_check_sub_units:` *and* `\__spintent_spmoney_set_and_print_sub_units:`.)

`\__spintent_spmoney_build_tree:`
`_spintent_spmoney_print_integer_continue:`
`\__spintent_spmoney_print_sep_con:`
`\__spintent_spmoney_print_dec_sub_unit:`

The function `\__spintent_spmoney_build_tree:` structures the final MathML tree assembly for fractional *"currencies"*. It arranges the integer component, the decimal separator, and the *"subunit"* component based on the layout position (`pre` or `post`) of the *"currency symbol"*.

The function `\__spintent_spmoney_print_integer_continue:` isolates the rendering of the integer node with its associated main *"currency"* intent. Furthermore, `\__spintent_spmoney_print_sep_con:` applies the specific speech intent '`con`' to the decimal separator, while `\__spintent_spmoney_print_dec_sub_unit:` binds the computed *"subunit"* intent to the decimal digits, ensuring accurate accessibility output.

```
\cs_new_protected:Nn \__spintent_spmoney_build_tree:
  {
    \str_if_eq:VnTF \l__spintent_spmoney_luaset_position_str { pre }
      {
        \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
          {
            \__spintent_spmoney_print_symbol:
            \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
          }
        \__spintent_spmoney_print_sep_con:
        \__spintent_spmoney_print_dec_sub_unit:
      }
      {
        \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
          {
            \__spintent_invisible_sep:
            \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
```

```
          }
        \__spintent_spmoney_print_sep_con:
        \__spintent_spmoney_print_dec_sub_unit:
        \MathMLintent {:silent } { \__spintent_spmoney_print_symbol: }
      }
  }
\cs_new_protected:Nn \__spintent_spmoney_print_integer_continue:
  {
    \MathMLintent { \l__spintent_spmoney_res_main_voice_str :postfix($n) }
      {
        \MathMLarg { n } { \__spintent_spmoney_render_only_integer_node: }
      }
  }
\cs_new_protected:Nn \__spintent_spmoney_print_sep_con:
  {
    \MathMLintent { con }
      {
        \mathord { \l__spintent_luaset_dec_sep_tl }
      }
  }
\cs_new_protected:Nn \__spintent_spmoney_print_dec_sub_unit:
  {
    \MathMLintent { \l__spintent_spmoney_intent_sub_unit_str }
      {
        \__spintent_invisible_sep:
        \MathMLarg { sub }
          {
            \__spintent_spnum_render_dec:
          }
      }
  }
```

(*End of definition for* `\__spintent_spmoney_build_tree:` *and others.*)

`\__spintent_spmoney_parse_and_route:n`

The internal function `\__spintent_spmoney_parse_and_route:n` processes the {⟨*argument*⟩} passed in '`#1`' using Lua module to extract the numeric value and any *"currency"* identifier. If a *"currency"* is found (`\l__spintent_spmoney_extracted_curr_str`), it sets the corresponding variables. It then calls `\__spintent_spmoney_validate_money:` to verify the *"currency"*. If *"subunit"* rendering is enabled (`\l__spintent_spmoney_sub_units_bool`), it calls `\__spintent_spmoney_check_sub_units:`. Finally, it calls `\__spintent_spmoney_print_int:` to format the output.

```
\cs_new_protected:Nn \__spintent_spmoney_parse_and_route:n
  {
    \__spintent_luafun_spmoney_prepare_input:n { #1 }
    \__spintent_luafun_clean_split_and_set:V \l__spintent_spmoney_extracted_num_tl
    \str_if_empty:NF \l__spintent_spmoney_extracted_curr_str
      {
        \str_set:Ne \l__spintent_luaset_has_units_str { true }
        \str_set:NV \l__spintent_luaset_arg_above_tl \l__spintent_spmoney_extracted_curr_str
      }
    \__spintent_spmoney_validate_money:
    \bool_if:NT \l__spintent_spmoney_sub_units_bool
      { \__spintent_spmoney_check_sub_units: }
    \__spintent_spmoney_print_int:
  }
```

(*End of definition for* `\__spintent_spmoney_parse_and_route:n`.)

`\spmoney`

The public document command `\spmoney` formats monetary values with accessible speech intents. It accepts an optional argument '`#1`' for local key-value configurations and a mandatory argument '`#2`' containing the numeric value and optional embedded currency identifier. It first evaluates the typesetting context, triggering a fatal error if executed outside of math mode. Upon validation, it establishes a local TEX group to safely isolate the key assignments, delegates the raw input string to `\__spintent_spmoney_parse_and_route:n` for parsing and rendering, and subsequently closes the group to prevent scope leakage.

```
\NewDocumentCommand \spmoney { O{} m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \keys_set:nn { spintent / spmoney } { #1 }
          \__spintent_spmoney_parse_and_route:n { #2 }
        \group_end:
```

©2026 by Pablo González L

```
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spmoney } }
  }
```

(*End of definition for* `\spmoney`. *This function is documented on page 6.*)

11.16 The command `\spshort`

The user command `\spshort` is the *fourth* and *final* of *"the big four"*. It manages the semantic injection and typographical formatting of *text mode* abbreviations, ordinals, and acronyms. Unlike *math mode* commands, which rely on `MathML` attributes, this command uses the `PDF` *tagging tools* provided by the `tagpdf`[29] package. Specifically, it uses the `E` (expanded) key to ensure that `NVDA` interpret the full form of the abbreviation, ordinal, or acronym in *text mode*.

`\l__spintent_spshort_luaset_status_str`
`\l__spintent_spshort_luaset_layout_str`
`\l__spintent_spshort_luaset_expanded_str`
`\l__spintent_spshort_luaset_output_str`
`\l__spintent_spshort_luaset_base_str`
`\l__spintent_spshort_luaset_suffix_str`

These internal string variables serve as the primary data interface with the Lua module `spintent.lua`. During processing, the Lua module *"stores"* within these registers the execution status, the targeted typographical layout parameters, the phonetic replacement text for the `E` key intended for `NVDA`, the final sanitized output string, and the isolated structural components for ordinals (base and suffix).

```
\str_new:N  \l__spintent_spshort_luaset_status_str
\str_new:N  \l__spintent_spshort_luaset_layout_str
\str_new:N  \l__spintent_spshort_luaset_expanded_str
\str_new:N  \l__spintent_spshort_luaset_output_str
\str_new:N  \l__spintent_spshort_luaset_base_str
\str_new:N  \l__spintent_spshort_luaset_suffix_str
```

(*End of definition for* `\l__spintent_spshort_luaset_status_str` *and others.*)

11.16.1 Definition of error messages for `\spshort`

`unknown-abbreviation`

The internal "error" message `unknown-abbreviation` is triggered when the command `\spshort` encounters an invalid lookup state from the Lua module. It acts as a strict fallback exception when the provided {⟨*argument*⟩} cannot be mapped to any registered database entry in `spintent.lua` nor successfully resolved through the grammatical ordinal expansion rules.

```
\msg_new:nnnn { spintent } { unknown-abbreviation }
  { The~abbreviation~or~ordinal~'#1'~is~invalid~or~not~recognized. }
  { Verify~the~spelling~or~check~the~grammar~rules~for~ordinals. }
```

(*End of definition for* `unknown-abbreviation`.)

11.16.2 Accessibility interface

`\__spintent_spshort_tag_span:nn`
`\__spintent_spshort_tag_span:en`

The internal function `\__spintent_spshort_tag_span:nn` processes the {⟨*argument*⟩} passed in '`#2`' using the tools provided by `tagpdf`. By utilizing the `tag` and `E` keys, it creates a native `PDF /E (#1)` structure and expands the {⟨*argument*⟩} passed in '`#1`' for proper reading with `NVDA`.

```
\cs_new_protected:Nn \__spintent_spshort_tag_span:nn
  {
    \tag_mc_end_push:
      \tag_struct_begin:n { tag=Span, E={ #1 } }
        \tag_mc_begin:n { tag=Span }
          \tag_suspend:n { \spshort }
            #2
          \tag_resume:n { \spshort }
        \tag_mc_end:
      \tag_struct_end:
    \tag_mc_begin_pop:n {}
  }
\cs_generate_variant:Nn \__spintent_spshort_tag_span:nn { en }
```

(*End of definition for* `\__spintent_spshort_tag_span:nn`.)

11.16.3 Definition of keys for `\spshort`

`small-caps`
`read-arg`

Now we add the keys `small-caps` and `read-arg`.

```
\keys_define:nn { spintent / spshort }
  {
    small-caps .bool_set:N = \l__spintent_spshort_smallcaps_bool,
    small-caps .initial:n  = false,
    small-caps .value_required:n = true,
    read-arg   .str_set:N  = \l__spintent_spshort_read_arg_str,
    read-arg   .initial:n  = {},
    read-arg   .value_required:n = true,
    unknown    .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spshort }
        \__spintent_unknown_keys_cmd:n {#1}
```

```
      },
  }
```

(*End of definition for* `small-caps` *and* `read-arg`.)

11.16.4 Internal functions for `\spshort`

`\__spintent_spshort_print:nn`

The internal function `\__spintent_spshort_print:nn` acts as the primary formatter for text abbreviations. It clears the string variable `\l__spintent_spshort_read_arg_str` and evaluates the optional ⟨*keys*⟩ passed in '`#1`'. It then checks if a manual reading override was provided via those keys. If that string variable remains empty, it calls `\__spintent_spshort_process_and_render:n` to process the {⟨*argument*⟩} passed in '`#2`'. Otherwise, if an override is detected, it directly delegates the execution to `\__spintent_spshort_render_bypass:n`.

```
\cs_new_protected:Nn \__spintent_spshort_print:nn
  {
    \str_clear:N \l__spintent_spshort_read_arg_str
    \keys_set:nn { spintent / spshort } {#1}
    \str_if_empty:NTF \l__spintent_spshort_read_arg_str
      {
        \__spintent_spshort_process_and_render:n {#2}
      }
      {
        \__spintent_spshort_render_bypass:n {#2}
      }
  }
```

(*End of definition for* `\__spintent_spshort_print:nn`.)

`\__spintent_spshort_process_and_render:n`

The internal function `\__spintent_spshort_process_and_render:n` interfaces with the Lua module `spintent.lua` function `\__spintent_luafun_spshort_process:n` to resolve the abbreviation or ordinal {⟨*argument*⟩} passed in '`#1`'.

It subsequently evaluates the execution state stored within the string variable `\l__spintent_spshort_luaset_statu` and dynamically routes execution to the corresponding rendering subroutine `success`, `fallback` or `error`. Any unrecognized status automatically defaults to the "error" handler.

```
\cs_new_protected:Nn \__spintent_spshort_process_and_render:n
  {
    \__spintent_luafun_spshort_process:n {#1}
    \str_case:VnF \l__spintent_spshort_luaset_status_str
      {
        { success }  { \__spintent_spshort_render_success: }
        { fallback } { \__spintent_spshort_render_fallback: }
        { error }    { \__spintent_spshort_render_error:n {#1} }
      }
      { \__spintent_spshort_render_error:n {#1} }
  }
```

(*End of definition for* `\__spintent_spshort_process_and_render:n`.)

`\__spintent_spshort_render_success:`
`\__spintent_spshort_render_fallback:`
`\__spintent_spshort_render_bypass:n`
`\__spintent_spshort_render_error:n`

The function `\__spintent_spshort_render_success:` injects the key `E` and executes the targeted typographical layout. The `\__spintent_spshort_render_fallback:` function handles default cases, applying optional formatting such as SMALL CAPS to unrecognized or partially resolved terms.

Conversely, the function `\__spintent_spshort_render_bypass:n` directly maps the user-defined *"speech"* to the {⟨*argument*⟩}, bypassing the automated lookup Lua module. Finally, the function `\__spintent_spshort_rende` triggers a structural exception while yielding the unformatted {⟨*argument*⟩} to prevent fatal compilation.

```
\cs_new_protected:Nn \__spintent_spshort_render_success:
  {
    \__spintent_spshort_tag_span:en
      { \l__spintent_spshort_luaset_expanded_str }
      { \__spintent_spshort_execute_layout: }
  }
\cs_new_protected:Nn \__spintent_spshort_render_fallback:
  {
    \__spintent_spshort_tag_span:en
      { \l__spintent_spshort_luaset_expanded_str }
      {
        \bool_if:NTF \l__spintent_spshort_smallcaps_bool
          { \textsc{ \l__spintent_spshort_luaset_output_str } }
          { \l__spintent_spshort_luaset_output_str }
      }
  }
\cs_new_protected:Nn \__spintent_spshort_render_bypass:n
```

```
  {
    \__spintent_spshort_tag_span:en { \l__spintent_spshort_read_arg_str } { #1 }
  }
\cs_new_protected:Nn \__spintent_spshort_render_error:n
  {
    \msg_error:nnn { spintent } { unknown-abbreviation } {#1}
  }
```

(*End of definition for* `\__spintent_spshort_render_success:` *and others.*)

`\__spintent_spshort_execute_layout:`

The internal function `\__spintent_spshort_execute_layout:` executes the typographical layout. It evaluates the descriptor *"stored"* within the string variable `\l__spintent_spshort_luaset_layout_str` (such as `linear_regular`, `superscript`, or `small_caps_pure`) and maps it to the corresponding LaTeX formatting. Furthermore, it manages complex nested configurations, dynamically applying combinations such as superscripts integrated with conditional SMALL CAPS based on the active boolean variables.

```
\cs_new_protected:Nn \__spintent_spshort_execute_layout:
  {
    \str_case:VnF \l__spintent_spshort_luaset_layout_str
      {
        { linear_regular }
          {
            \bool_if:NTF \l__spintent_spshort_smallcaps_bool
              { \textsc{ \l__spintent_spshort_luaset_output_str } }
              { \l__spintent_spshort_luaset_output_str }
          }
        { linear_caps }
          { \l__spintent_spshort_luaset_output_str }
        { linear_mixed }
          { \l__spintent_spshort_luaset_output_str }
        { superscript }
          {
            \l__spintent_spshort_luaset_base_str
            \bool_if:NTF \l__spintent_spshort_smallcaps_bool
              {
                \textsuperscript{
                  \textsc{
                    \str_lowercase:V \l__spintent_spshort_luaset_suffix_str
                  }
                }
              }
              {
                \textsuperscript{ \l__spintent_spshort_luaset_suffix_str }
              }
          }
        { small_caps_pure }
          {
            \bool_if:NTF \l__spintent_spshort_smallcaps_bool
              {
                \textsc{
                  \str_lowercase:V \l__spintent_spshort_luaset_output_str
                }
              }
              { \l__spintent_spshort_luaset_output_str }
          }
        { acronym_long }
          { \l__spintent_spshort_luaset_output_str }
      }
      { \l__spintent_spshort_luaset_output_str }
  }
```

(*End of definition for* `\__spintent_spshort_execute_layout:`.)

`\spshort`

The public document command `\spshort` manages the accessible formatting of *text mode* abbreviations, acronyms, and ordinals. It first triggers an error if invoked within a *math mode*, upon validation, puts `\mode_leave_vertical:` and establishes a local group to call the function `\__spintent_spshort_print:nn` for processing and printing.

```
\NewDocumentCommand \spshort { O{} m }
  {
    \mode_if_math:TF
      {
        \msg_error:nnn { spintent } { need-text-mode } { \spshort }
```

```
      }
      {
        \mode_leave_vertical:
        \group_begin:
          \__spintent_spshort_print:nn {#1} {#2}
        \group_end:
      }
  }
```

(*End of definition for* `\spshort`. *This function is documented on page* 7.)

11.17 The command `\spsiglo`

The public document command `\spsiglo` provides a semantic interface for typesetting century designations (e.g., siglo XXI). It accepts both Arabic and Roman numerals as {⟨*argument*⟩}, interfacing with the Lua module `spintent.lua` to validate, parse, and normalize chronological values.

This command work in dual-mode context awareness, operating uniformly across both *text mode* and *math mode*. When executed within *math mode*, it constructs an accessible `MathML intent` expression tree. Conversely, when invoked in standard *text mode*, it encapsulates the rendered output within a native `PDF` tagging structure (such as `/Alt (#1)`) containing an `alt` key.

`\l__spintent_spsiglo_luaset_error_str`
`\l__spintent_spsiglo_luaset_arabic_str`
`\l__spintent_spsiglo_luaset_roman_min_str`
`\l__spintent_spsiglo_print_era_str`

These internal string variables handle data transfer and synchronization with the Lua module `spintent.lua`. They isolate the exception evaluation states, the Arabic numeral required for speech synthesis, the lowercase Roman numeral designated for small caps typesetting via `\textsc`, and the localized chronological era modifier.

```
\str_new:N \l__spintent_spsiglo_luaset_error_str
\str_new:N \l__spintent_spsiglo_luaset_arabic_str
\str_new:N \l__spintent_spsiglo_luaset_roman_min_str
\str_new:N \l__spintent_spsiglo_print_era_str
```

(*End of definition for* `\l__spintent_spsiglo_luaset_error_str` *and others.*)

11.17.1 Definition of error messages for `\spsiglo`

`invalid-century-format`

This internal error message is issued when the provided century identifier fails both Arabic and Roman numeral validation. It is triggered when the Lua module `spintent.lua` cannot map the {⟨*argument*⟩} to an integer within the valid range of `1` to `4000`, preventing incorrect speech synthesis.

```
\msg_new:nnnn { spintent } { invalid-century-format }
  {
    Invalid~century~format~in~'#1'.
  }
  {
    The~argument~'#2'~is~not~a~valid~Arabic~or~Roman~numeral.~
    Ensure~the~value~is~between~1~and~4000.
  }
```

(*End of definition for* `invalid-century-format`.)

11.17.2 Definition of keys for `\spsiglo`

`print-era`

Now we add the key `print-era`. User configuration determining if an era designation is appended, available values are `none`, `ac` (antes de cristo) and `dc` (después de cristo).

```
\keys_define:nn { spintent / spsiglo }
  {
    print-era           .choices:nn =
      { none , ac , dc }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \str_set:Nn \l__spintent_spsiglo_print_era_str { none } }
            {2} { \str_set:Nn \l__spintent_spsiglo_print_era_str { ac } }
            {3} { \str_set:Nn \l__spintent_spsiglo_print_era_str { dc } }
          }
      },
    print-era           .initial:n  = none,
    print-era           .value_required:n = true,
    print-era / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { print-era } { none, ~ac, ~dc } { \exp_not:n {#1} },
    unknown             .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spsiglo }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

©2026 by Pablo González L

(*End of definition for* `print-era`.)

11.17.3 Internal functions for `\spsiglo`

`\__spintent_spsiglo_render_simple:nn`
`\__spintent_spsiglo_render_simple:VV`
`\__spintent_spsiglo_render_with_era:nnn`
`\__spintent_spsiglo_render_with_era:VVV`

The functions `\__spintent_spsiglo_render_simple:nn` and `\__spintent_spsiglo_render_with_era:nnn` generate MathML structures when century designations are evaluated within *math mode*. They construct MathML `intent` structures, mapping the Arabic numerals to their phonetic equivalents. Furthermore, `\__spintent_spsiglo_render_with_era:nnn` integrates the era modifiers `ac` or `dc`. It adds invisible spacing and postfix intents to ensure correct NVDA/MathCAT pronunciation.

```
\cs_new_protected:Nn \__spintent_spsiglo_render_simple:nn
  {
    \MathMLintent { #1:number } { \text { \textsc {#2} } }
  }
\cs_generate_variant:Nn \__spintent_spsiglo_render_simple:nn { VV }
\cs_new_protected:Nn \__spintent_spsiglo_render_with_era:nnn
  {
    \str_if_eq:nnTF { #1 } { ac }
      {
        \MathMLintent { antes-de-cristo:postfix($z) }
          {
            \__spintent_invisible_sep:
            \MathMLarg { z }
              {
                \MathMLintent { #2:number }
                  {
                    \text { \textsc {#3}~a.~C. }
                  }
              }
          }
      }
      {
        \MathMLintent { después-de-cristo:postfix($z) }
          {
            \__spintent_invisible_sep:
            \MathMLarg { z }
              {
                \MathMLintent { #2:number }
                  {
                    \text { \textsc {#3}~d.~C. }
                  }
              }
          }
      }
  }
\cs_generate_variant:Nn \__spintent_spsiglo_render_with_era:nnn { VVV }
```

(*End of definition for* `\__spintent_spsiglo_render_simple:nn` *and* `\__spintent_spsiglo_render_with_era:nnn`.)

`\__spintent_spsiglo_tag_span:nn`
`\__spintent_spsiglo_tag_span:en`

The function `\__spintent_spsiglo_tag_span:nn` handles century formatting within *text mode*. It utilizes the tagpdf package to enclose the content inside a PDF `/Alt (#1)` structure. By passing the {⟨*argument*⟩} to the `alt` key, it embeds the chronological value directly into the PDF, ensuring correct NVDA/MathCAT pronunciation without relying on MathML intents.

```
\cs_new_protected:Nn \__spintent_spsiglo_tag_span:nn
  {
    \tag_mc_end_push:
      \tag_struct_begin:n { tag=Span, alt={ #1 } }
        \tag_mc_begin:n { tag=Span }
          \tag_suspend:n { \spsiglo }
            #2
          \tag_resume:n { \spsiglo }
        \tag_mc_end:
      \tag_struct_end:
    \tag_mc_begin_pop:n {}
  }
\cs_generate_variant:Nn \__spintent_spsiglo_tag_span:nn { en }
```

(*End of definition for* `\__spintent_spsiglo_tag_span:nn`.)

`\__spintent_spsiglo_print_math:n`
`\__spintent_spsiglo_print_text:n`

The functions `\__spintent_spsiglo_print_math:n` and `\__spintent_spsiglo_print_text:n` format century designations based on the active mode. Both first check the "error" status in the variable `\l__spintent_spsiglo_luaset_error_str` returned by the Lua module `spintent.lua` and issue an "error" if validation fails.

If the {⟨*argument*⟩} is valid, the function `\__spintent_spsiglo_print_math:n` operates in *math mode*, applying MathML structures depending on whether an era modifier is present. Conversely, the function `\__spintent_spsiglo_print_text:n` operates in *text mode*, utilizing the `tag` and `alt` keys to expand the value of the `print-era` option into full text (e.g., "antes de cristo") for NVDA/MathCAT.

```
\cs_new_protected:Nn \__spintent_spsiglo_print_math:n
  {
    \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
      {
        \msg_error:nnnn { spintent } { invalid-century-format }
          { \spsiglo } { #1 }
      }
      {
        \str_if_eq:VnTF \l__spintent_spsiglo_print_era_str { none }
          {
            \__spintent_spsiglo_render_simple:VV
              \l__spintent_spsiglo_luaset_arabic_str
              \l__spintent_spsiglo_luaset_roman_min_str
          }
          {
            \__spintent_spsiglo_render_with_era:VVV
              \l__spintent_spsiglo_print_era_str
              \l__spintent_spsiglo_luaset_arabic_str
              \l__spintent_spsiglo_luaset_roman_min_str
          }
      }
  }
\cs_new_protected:Nn \__spintent_spsiglo_print_text:n
  {
    \str_if_eq:VnTF \l__spintent_spsiglo_luaset_error_str { true }
      {
        \msg_error:nnnn { spintent } { invalid-century-format }
          { \spsiglo } { #1 }
      }
      {
       \str_case:Vn \l__spintent_spsiglo_print_era_str
          {
            { none }
            {
              \__spintent_spsiglo_tag_span:en
                { \l__spintent_spsiglo_luaset_arabic_str }
                { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } }
            }
            { ac }
            {
              \__spintent_spsiglo_tag_span:en
                { \l__spintent_spsiglo_luaset_arabic_str ~ antes ~ de ~ cristo }
                { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ a.~C. }
            }
            { dc }
            {
              \__spintent_spsiglo_tag_span:en
                { \l__spintent_spsiglo_luaset_arabic_str ~ después ~ de ~ cristo }
                { \textsc { \l__spintent_spsiglo_luaset_roman_min_str } ~ d.~C. }
            }
          }
      }
  }
```

(*End of definition for* `\__spintent_spsiglo_print_math:n` *and* `\__spintent_spsiglo_print_text:n`.)

`\spsiglo` The public command `\spsiglo` is the main interface for century typesetting. It opens a local group and evaluates any optional ⟨*keys*⟩ passed in '`#1`', then executes the function `\__spintent_luafun_spsiglo_parse:n` defined in the Lua module `spintent.lua` passed in '`#2`' and formats the output for *math mode* or *text mode*.

```
\NewDocumentCommand \spsiglo { O{} m }
  {
    \group_begin:
      \keys_set:nn { spintent / spsiglo } { #1 }
      \__spintent_luafun_spsiglo_parse:n { #2 }
      \mode_if_math:TF
        { \__spintent_spsiglo_print_math:n { #2 } }
        {
```

©2026 by Pablo González L

```
      \mode_leave_vertical:
      \__spintent_spsiglo_print_text:n { #2 }
    }
  \group_end:
}
```

(*End of definition for* `\spsiglo`. *This function is documented on page* 11.)

11.18 The command `\sptime`

The command `\sptime` is the main public interface for typesetting and tagging time expressions. It parses time string formats (such as `HH:MM` or `HH:MM:SS`) using the Lua function `\__spintent_luafun_sptime_parse:n`. This separates the hours, minutes, seconds, and fractions to construct a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_sptime_luaset_seconds_str`
`\l__spintent_sptime_luaset_has_seconds_str`
`\l__spintent_sptime_luaset_error_str`
`\l__spintent_sptime_luaset_is_fraction_str`
`\l__spintent_sptime_luaset_base_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted time units, error flags, and boolean indicators (such as the presence of seconds or decimal fractions) used during formatting.

```
\str_new:N \l__spintent_sptime_luaset_seconds_str
\str_new:N \l__spintent_sptime_luaset_has_seconds_str
\str_new:N \l__spintent_sptime_luaset_error_str
\str_new:N \l__spintent_sptime_luaset_is_fraction_str
\str_new:N \l__spintent_sptime_luaset_base_str
```

(*End of definition for* `\l__spintent_sptime_luaset_seconds_str` *and others.*)

11.18.1 Definition of error messages

`invalid-time-format`

The error message `invalid-time-format` defines the text displayed when an invalid time input is detected. It is triggered when an argument fails the format checks or contains values outside normal ranges, such as hours exceeding 23 or minutes and seconds exceeding 59.

```
\msg_new:nnnn { spintent } { invalid-time-format }
  {
    Invalid~time~format~in~#1. \\
    The~argument~'#2'~does~not~match~any~supported~time~format.
  }
  {
    Allowed~formats:~HH:MM,~HH:MM:SS,~or~HH:MM:SS.ss. \\
    Please~verify~that~hours~are~0-23~and~minutes/seconds~are~0-59.
  }
```

(*End of definition for* `invalid-time-format`.)

11.18.2 Internal functions for `\sptime`

`\__spintent_sptime_build_seconds:nn`

The internal function `\__spintent_sptime_build_seconds:nn` builds the MathML structure for the seconds component of a time expression. It wraps the numeric value '`#2`' in speech intent tags, applying a connector prefix (passed in '`#1`') and the "segundos" postfix so it is read correctly by NVDA/MathCAT.

```
\cs_new_protected:Nn \__spintent_sptime_build_seconds:nn
  {
    \MathMLintent { #1 : prefix($z) }
      {
        \text { : }
        \MathMLarg { z }
          {
            \MathMLintent { segundos : postfix($m) }
              {
                \__spintent_invisible_sep:
                \MathMLarg { m }
                  {
                    \text { #2 }
                  }
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_sptime_build_seconds:nn`.)

`\__spintent_sptime_build_tree:`

The internal function `\__spintent_sptime_build_tree:` builds the complete MathML structure for the time expression. It checks the string variables `\l__spintent_sptime_luaset_has_seconds_str` and `\l__spintent_sptime_luaset_is_fraction_str` to determine if the time includes seconds or decimal fractions. Based on these values, it inserts the base time string and either calls `\__spintent_sptime_build_seconds:`

with the appropriate prefix ('con' or 'minutos-con'), or simply appends the 'minutos' intent tag if no seconds are present.

```
\cs_new_protected:Nn \__spintent_sptime_build_tree:
  {
    \str_if_eq:VnTF \l__spintent_sptime_luaset_has_seconds_str { true }
      {
        \MathMLintent { :time }
          {
            \text { \l__spintent_sptime_luaset_base_str }
          }
        \str_if_eq:VnTF \l__spintent_sptime_luaset_is_fraction_str { true }
          {
            \__spintent_sptime_build_seconds:nn { con }
              { \l__spintent_sptime_luaset_seconds_str }
          }
          {
            \__spintent_sptime_build_seconds:nn { minutos-con }
              { \l__spintent_sptime_luaset_seconds_str }
          }
      }
      {
        \MathMLintent { :time }
          {
            \text { \l__spintent_sptime_luaset_base_str }
          }
        \str_if_eq:VnT \l__spintent_sptime_luaset_is_fraction_str { false }
          {
            \MathMLintent { minutos }
              {
                \__spintent_invisible_sep:
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_sptime_build_tree:`.)

`\sptime` The command `\sptime` is the public interface for formatting time expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it parses the input using `\__spintent_luafun_sptime_parse:n` and checks the error status in `\l__spintent_sptime_luaset_error_str`. If an invalid format is detected, it issues the `invalid-time-format` error; otherwise, it calls `\__spintent_sptime_build_tree:` to assemble the MathML output.

```
\NewDocumentCommand \sptime { m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_luafun_sptime_parse:n { #1 }
          \str_if_eq:VnTF \l__spintent_sptime_luaset_error_str { true }
            {
               \msg_error:nnnn { spintent } { invalid-time-format }
                 { \sptime } { #1 }
            }
            {
               \__spintent_sptime_build_tree:
            }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \sptime } }
  }
```

(*End of definition for* `\sptime`. *This function is documented on page* 11.)

11.19 The command \spdate

The command `\spdate` is the main public command for typesetting and tagging date expressions. It parses date string formats (such as `DD/MM/YYYY` or `YYYY-MM-DD`) using the Lua function `\__spintent_luafun_spdate_parse:n`. This verifies the input and constructs a MathML `intent` tree for NVDA/MathCAT.

`\l__spintent_spdate_luaset_output_str` `\l__spintent_spdate_luaset_error_str` These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the formatted date string and the error status used during formatting.

```
\str_new:N \l__spintent_spdate_luaset_output_str
```

©2026 by Pablo González L

```
\str_new:N \l__spintent_spdate_luaset_error_str
```

(*End of definition for* `\l__spintent_spdate_luaset_output_str` *and* `\l__spintent_spdate_luaset_error_str`.)

11.19.1 Definition of error messages

`invalid-date-format`

The error message `invalid-date-format` defines the text displayed when an invalid date input is detected. It is triggered when an argument fails the format checks or represents an invalid calendar date, such as `31/02/2024` or using an unsupported pattern like `YYYY/MM/DD`.

```
\msg_new:nnnn { spintent } { invalid-date-format }
  {
    Invalid~date~or~unsupported~format~in~#1. \\
    The~argument~'#2'~does~not~match~a~supported~format.
  }
  {
    Allowed~formats:~DD\slash MM\slash YYYY,~DD-MM-YYYY,~or~YYYY-MM-DD. \\
    Ensure~the~day~and~month~are~valid~(e.g.,~avoid~31\slash 02\slash 2024).
  }
```

(*End of definition for* `invalid-date-format`.)

11.19.2 Internal functions for \spdate

`\__spintent_spdate_process:n`

The internal function `\__spintent_spdate_process:n` parses the input using `\__spintent_luafun_spdate_parse:n` and checks the error status in `\l__spintent_spdate_luaset_error_str`. If an invalid format is detected, it issues the `invalid-date-format` error. Otherwise, it builds the MathML intent structure using the formatted date stored in `\l__spintent_spdate_luaset_output_str`.

```
\cs_new_protected:Nn \__spintent_spdate_process:n
  {
    \__spintent_luafun_spdate_parse:n { #1 }
    \str_if_eq:VnTF \l__spintent_spdate_luaset_error_str { true }
      {
        \msg_error:nnnn { spintent } { invalid-date-format }
          { \spdate } { #1 }
      }
      {
        \MathMLintent { :date }
          {
            \text { \l__spintent_spdate_luaset_output_str }
          }
      }
  }
```

(*End of definition for* `\__spintent_spdate_process:n`.)

`\spdate`

The command `\spdate` is the public interface for formatting date expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spdate_process:n` to process the input.

```
\NewDocumentCommand \spdate { m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spdate_process:n { #1 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spdate } }
  }
```

(*End of definition for* `\spdate`. *This function is documented on page* 11.)

11.20 Semantic Operators

This section provides purely semantic operators. Unlike their formatting counterparts (which parse and format numbers), these commands do not process mandatory arguments. Instead, they solely inject a mathematical glyph wrapped in its corresponding regional MathCAT intent. This grants teachers absolute control when building complex algebraic expressions, mixed numbers, or fraction divisions.

11.21 The command \spusep

For the use of parentheses, we will provide the command \spusep with only two ⟨*keys*⟩ size and silent. Most of the time we can use \left and \right or \Uleft and \Uright, but both forms create a group, and this hinders the code for *tagged* PDF when we want to interrupt something between them.

spusep-empty-arg
spusep-invalid-arg

The first "error" message spusep-empty-arg is raised when \spusep receives an empty or blank {⟨*argument*⟩}, the second this is when an input is invalid.

```
\msg_new:nnnn { spintent } { spusep-empty-arg }
  { Empty~argument~in~\token_to_str:N \spusep. }
  { \token_to_str:N \spusep~requires~a~non-empty~argument. }
\msg_new:nnn { spintent } { spusep-invalid-arg }
  {
    Invalid~argument~'#1'~for~\token_to_str:N \spusep.
  }
```

(*End of definition for* spusep-empty-arg *and* spusep-invalid-arg.)

\l__spintent_spusep_arg_tl
\l__spintent_spusep_left_tl
\l__spintent_spusep_right_tl

The variable \l__spintent_spusep_arg_tl stores the {⟨*argument*⟩}, the variable \l__spintent_spusep_left_tl and \l__spintent_spusep_right_tl stores the parentheses.

```
\tl_new:N \l__spintent_spusep_arg_tl
\tl_new:N \l__spintent_spusep_left_tl
\tl_new:N \l__spintent_spusep_right_tl
```

(*End of definition for* \l__spintent_spusep_arg_tl, \l__spintent_spusep_left_tl, *and* \l__spintent_spusep_right_tl.)

size
silent
unknown

We define the ⟨*keys*⟩ size and silent.

```
\keys_define:nn { spintent / spusep }
  {
    size           .choices:nn =
      { none , big , Big, bigg, Bigg }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            { 1 }
              {
                \tl_clear:N \l__spintent_spusep_left_tl
                \tl_clear:N \l__spintent_spusep_right_tl
              }
            { 2 }
              {
                \tl_set:Nn \l__spintent_spusep_left_tl  { \bigl }
                \tl_set:Nn \l__spintent_spusep_right_tl { \bigr }
              }
            { 3 }
              {
                \tl_set:Nn \l__spintent_spusep_left_tl  { \Bigl }
                \tl_set:Nn \l__spintent_spusep_right_tl { \Bigr }
              }
            { 4 }
              {
                \tl_set:Nn \l__spintent_spusep_left_tl  { \biggl }
                \tl_set:Nn \l__spintent_spusep_right_tl { \biggr }
              }
            { 5 }
              {
                \tl_set:Nn \l__spintent_spusep_left_tl  { \Biggl }
                \tl_set:Nn \l__spintent_spusep_right_tl { \Biggr }
              }
          }
      },
    size           .initial:n  = none,
    size           .value_required:n = true,
    size / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { size } { none,~big,~Big,~bigg,~Bigg } { \exp_not:n {#1} },
    silent         .bool_set:N = \l__spintent_spusep_silent_bool,
    silent         .initial:n = false,
    silent         .value_required:n = true,
    unknown        .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spusep }
```

©2026 by Pablo González L

```
          \__spintent_unknown_keys_cmd:n {#1}
        },
    }
```

(*End of definition for* `size` , `silent` , *and* `unknown`.)

`\__spintent_spusep_set_parens:nn` The function `\__spintent_spusep_set_parens:nn` will place the argument passed in '`#2`' to the right of the variables `\l__spintent_spusep_left_tl` and `\l__spintent_spusep_right_tl` and then check the state of the variable `\l__spintent_spusep_silent_bool` handled by the key `silent` to print.

```
\cs_new_protected:Nn \__spintent_spusep_set_parens:nn
  {
    \tl_put_right:cn { l__spintent_spusep_ #1 _tl } { #2 }
    \bool_if:NTF \l__spintent_spusep_silent_bool
      {
         \MathMLintent { :silent } { \tl_use:c { l__spintent_spusep_ #1 _tl } }
      }
      {
        \tl_use:c { l__spintent_spusep_ #1 _tl }
      }
  }
```

(*End of definition for* `\__spintent_spusep_set_parens:nn`.)

`\__spintent_spusep_convert:n` The function `\__spintent_spusep_convert:n` captures and stores the {⟨*argument*⟩} passed in '`#1`' in the variable `\l__spintent_spusep_arg_tl`, validates the cases by converting them to the macros provided by `unicode-math`, and passes them to the function `\__spintent_spusep_set_parens:nn` for printing. If the input is not supported or is empty, we return an "error".

```
\cs_new_protected:Nn \__spintent_spusep_convert:n
  {
    \tl_set:Ne \l__spintent_spusep_arg_tl { \tl_trim_spaces:n {#1} }
    \tl_if_empty:NT \l__spintent_spusep_arg_tl
      {
        \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
      }
    \str_case:VnF \l__spintent_spusep_arg_tl
      {
        { (  }
          {
            \__spintent_spusep_set_parens:nn { left } { \lparen }
          }
        { )  }
          {
            \__spintent_spusep_set_parens:nn { right } { \rparen }
          }
        { [  }
          {
            \__spintent_spusep_set_parens:nn { left } { \lbrack }
          }
        { ]  }
          {
            \__spintent_spusep_set_parens:nn { right } { \rbrack }
          }
        { \{ }
          {
            \__spintent_spusep_set_parens:nn { left } { \lbrace }
          }
        { \} }
          {
            \__spintent_spusep_set_parens:nn { right } { \rbrace }
          }
      }
      {
        \msg_error:nnn { spintent } { spusep-invalid-arg } {#1}
      }
  }
```

(*End of definition for* `\__spintent_spusep_convert:n`.)

`\spusep` The command `\spusep` checks that it is running in math mode, opens a local group, sets the keys passed in '`#1`' and processes the {⟨*argument*⟩} passed in '`#2`' by means of the function `\__spintent_spusep_convert:n`.

```
\NewDocumentCommand \spusep { O{} m }
```

```
  {
    \mode_if_math:TF
      {
        \group_begin:
          \keys_set:nn { spintent / spusep } {#1}
          \__spintent_spusep_convert:n {#2}
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spusep } }
  }
```

(*End of definition for* `\spusep`. *This function is documented on page* ??.)

11.21.1 The command \spread

The command `\spread` provides a semantic read for `MathCAT` of mathematical symbols for situations in which it is necessary to specify gender or plurality in Spanish.

`spread-empty-args` The error message `spread-empty-args` is raised when `\spread` receives an empty or blank {⟨*argument*⟩}.

```
\msg_new:nnnn { spintent } { spread-empty-args }
  { Empty~argument~in~\token_to_str:N \spread. }
  { \token_to_str:N \spread~requires~two~non-empty~arguments. }
```

(*End of definition for* `spread-empty-args`.)

`\l__spintent_spread_intent_tl` The variable `\l__spintent_spread_intent_tl` stores the sanitized semantic reading {⟨*argument*⟩} for symbol to be injected into the `\MathMLintent`.

```
\tl_new:N \l__spintent_spread_intent_tl
```

(*End of definition for* `\l__spintent_spread_intent_tl`.)

`\__spintent_spread_exec:nn` The function `\__spintent_spread_exec:nn` implements the core logic for `\spread`. It verifies that neither {⟨*argument*⟩} is empty or blank. If valid, it evaluates the *reading* {⟨*argument*⟩} '`#1`' using `\__spintent_sanitize_affix:Nn` and embeds it via `\MathMLintent` into the *math symbol* '`#2`'.

```
\cs_new_protected:Nn \__spintent_spread_exec:nn
  {
    \bool_lazy_or:nnTF { \tl_if_blank_p:n { #1 } } { \tl_if_blank_p:n { #2 } }
      { \msg_error:nn { spintent } { spread-empty-args } }
      {
        \__spintent_sanitize_affix:Nn \l__spintent_spread_intent_tl { #1 }
        \MathMLintent { \l__spintent_spread_intent_tl } { #2 }
      }
  }
```

(*End of definition for* `\__spintent_spread_exec:nn`.)

`\spread` The command `\spread` assigns a custom semantic *reading* '`#1`' to any mathematical *symbol* '`#2`'. It enforces math mode and passes execution to `\__spintent_spread_exec:nn`.

```
\NewDocumentCommand \spread { m m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spread_exec:nn { #1 } { #2 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spread } }
  }
```

(*End of definition for* `\spread`. *This function is documented on page* 11.)

11.21.2 The command \spdsym

`\l__spintent_spdsym_read_sym_tl` `\l__spintent_spdsym_symbol_tl` `\l__spintent_spdsym_prefix_tl` `\l__spintent_spdsym_suffix_tl` Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spdsym`.

```
\tl_new:N \l__spintent_spdsym_read_sym_tl
\tl_new:N \l__spintent_spdsym_symbol_tl
\tl_new:N \l__spintent_spdsym_prefix_tl
\tl_new:N \l__spintent_spdsym_suffix_tl
```

(*End of definition for* `\l__spintent_spdsym_read_sym_tl` *and others.*)

prefix
suffix
set-sym
read-sym
unknown

We define the ⟨*keys*⟩ `prefix`, `suffix`, `set-sym` and `read-sym` for `\spdsym`.

```
\keys_define:nn { spintent / spdsym }
  {
    prefix             .code:n =
      { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_prefix_tl {#1} },
    prefix             .initial:n = ,
    prefix             .value_required:n = true,
    suffix             .code:n =
      { \__spintent_sanitize_affix:Nn \l__spintent_spdsym_suffix_tl {#1} },
    suffix             .initial:n = ,
    suffix             .value_required:n = true,
    set-sym            .choices:nn =
      { old-style , two-dots , slash }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \tl_set:Nn \l__spintent_spdsym_symbol_tl { \div } }
            {2} { \tl_set:Nn \l__spintent_spdsym_symbol_tl {   :  } }
            {3} { \tl_set:Nn \l__spintent_spdsym_symbol_tl {   /  } }
          }
      },
    set-sym            .initial:n  = two-dots,
    set-sym            .value_required:n = true,
    set-sym / unknown  .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { set-sym } { old-style, ~two-dots, ~slash } { \exp_not:n {#1} },
    read-sym           .choices:nn =
      { dividido , dividido-por , dividido-entre }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido       } }
            {2} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-por   } }
            {3} { \tl_set:Nn \l__spintent_spdsym_read_sym_tl { dividido-entre } }
          }
      },
    read-sym           .initial:n  = dividido,
    read-sym           .value_required:n = true,
    read-sym / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { read-sym } { dividido, ~dividido-por, ~dividido-entre }
        { \exp_not:n {#1} },
    unknown            .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spdsym }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `prefix` *and others.*)

`\spdsym`

Defines the `\spdsym` operator. It takes a single optional argument for local setup and outputs the configured intent structure, safely attaching prefixes and suffixes if present.

```
\NewDocumentCommand \spdsym { O{} }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \keys_set:nn { spintent / spdsym } { #1 }
          \MathMLintent
            {
              \tl_if_empty:NF \l__spintent_spdsym_prefix_tl
                { \l__spintent_spdsym_prefix_tl - }
              \l__spintent_spdsym_read_sym_tl
              \tl_if_empty:NF \l__spintent_spdsym_suffix_tl
                { - \l__spintent_spdsym_suffix_tl }
            }
            { \l__spintent_spdsym_symbol_tl }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spdsym } }
  }
```

©2026 by Pablo González L

(*End of definition for* `\spdsym`. *This function is documented on page* 12.)

11.21.3 The command `\spmsym`

`\l__spintent_spmsym_read_sym_tl`
`\l__spintent_spmsym_symbol_tl`
`\l__spintent_spmsym_prefix_tl`
`\l__spintent_spmsym_suffix_tl`

Variables to store the regional reading string, the visual operator, and the optional semantic affixes specifically for `\spmsym`.

```
\tl_new:N \l__spintent_spmsym_read_sym_tl
\tl_new:N \l__spintent_spmsym_symbol_tl
\tl_new:N \l__spintent_spmsym_prefix_tl
\tl_new:N \l__spintent_spmsym_suffix_tl
```

(*End of definition for* `\l__spintent_spmsym_read_sym_tl` *and others.*)

`prefix` `suffix` `set-sym` `read-sym` `not-print` `unknown`

We define the ⟨*keys*⟩ `prefix`, `suffix`, `set-sym`, `read-sym` and `not-print` for `\spmsym`.

```
\keys_define:nn { spintent / spmsym }
  {
    prefix             .code:n =
      { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_prefix_tl {#1} },
    prefix             .initial:n = ,
    prefix             .value_required:n = true,
    suffix             .code:n =
      { \__spintent_sanitize_affix:Nn \l__spintent_spmsym_suffix_tl {#1} },
    suffix             .initial:n = ,
    suffix             .value_required:n = true,
    set-sym            .choices:nn =
      { cross , dot , asterisk }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \times } }
            {2} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \cdot  } }
            {3} { \tl_set:Nn \l__spintent_spmsym_symbol_tl { \ast   } }
          }
      },
    set-sym            .initial:n  = dot,
    set-sym            .value_required:n = true,
    set-sym / unknown  .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { set-sym } { cross, ~dot, ~asterisk } { \exp_not:n {#1} },
    read-sym           .choices:nn =
      { por , multiplicado-por , veces }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { por } }
            {2} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { multiplicado-por } }
            {3} { \tl_set:Nn \l__spintent_spmsym_read_sym_tl { veces } }
          }
      },
    read-sym           .initial:n  = por,
    read-sym           .value_required:n = true,
    read-sym / unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { read-sym } { por, ~multiplicado-por, ~veces } { \exp_not:n {#1} },
    not-print          .bool_set:N = \l__spintent_spmsym_not_print_bool,
    not-print          .initial:n  = false,
    not-print          .value_required:n = true,
    unknown            .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spmsym }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `prefix` *and others.*)

`\spmsym`

Defines the `\spmsym` operator. Similar to `\spdsym`, it outputs the correct mathematical glyph based on the local ⟨*keys*⟩, safely attaching prefixes and suffixes if present.

```
\NewDocumentCommand \spmsym { O{} }
  {
    \mode_if_math:TF
      {
```

```
      \group_begin:
        \keys_set:nn { spintent / spmsym } { #1 }
        \MathMLintent
          {
            \tl_if_empty:NF \l__spintent_spmsym_prefix_tl
              { \l__spintent_spmsym_prefix_tl - }
            \l__spintent_spmsym_read_sym_tl
            \tl_if_empty:NF \l__spintent_spmsym_suffix_tl
              { - \l__spintent_spmsym_suffix_tl }
          }
          {
            \bool_if:NTF \l__spintent_spmsym_not_print_bool
              {
                \invisibletimes
              }
              { \l__spintent_spmsym_symbol_tl }
          }
      \group_end:
    }
    { \msg_error:nnn { spintent } { need-math-mode } { \spmsym } }
}
```

(*End of definition for* `\spmsym`. *This function is documented on page* 12.)

11.22 The command `\spdiv`

The user-level command `\spdiv` formats division representations. It takes two mandatory arguments: the {⟨*dividend*⟩} and the {⟨*divisor*⟩}. Both {⟨*arguments*⟩} are internally processed by the `\spnum` command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the {⟨*dividend*⟩} nor the {⟨*divisor*⟩} is *blank*.

11.22.1 Definition of keys for `\spdiv`

cifras-sep
read-sign
read-period-sep
notation-sym
set-sym
read-sym
wrap-parens
read-parens

We define the ⟨*keys*⟩ `wrap-parens` and `read-parens` for `\spdiv` command. Number-related ⟨*keys*⟩ `cifras-sep`, `read-sign`, `read-period-sep` and `notation-sym` are forwarded to `\spnum` command using `.meta:nn`, while semantic operator ⟨*keys*⟩ `set-sym` and `read-sym` are forwarded to the `\spdsym` command using `.meta:nn`.

```
\keys_define:nn { spintent / spdiv }
  {
    cifras-sep      .meta:nn = { spintent / spnum  } { cifras-sep      = {#1} },
    read-sign       .meta:nn = { spintent / spnum  } { read-sign       = {#1} },
    read-period-sep .meta:nn = { spintent / spnum  } { read-period-sep = {#1} },
    notation-sym    .meta:nn = { spintent / spnum  } { notation-sym    = {#1} },
    set-sym         .meta:nn = { spintent / spdsym } { set-sym         = {#1} },
    read-sym        .meta:nn = { spintent / spdsym } { read-sym        = {#1} },
    wrap-parens     .bool_set:N = \l__spintent_spdiv_wrap_parens_bool,
    wrap-parens     .initial:n  = false,
    wrap-parens     .value_required:n = true,
    read-parens     .bool_set:N = \l__spintent_spdiv_read_parens_bool,
    read-parens     .initial:n  = true,
    read-parens     .value_required:n = true,
    unknown         .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spdiv }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `cifras-sep` *and others.*)

11.22.2 Definition of error messages

spdiv-empty-argument

Raised when either the {⟨*dividend*⟩} or the {⟨*divisor*⟩} is missing or *blank*.

```
\msg_new:nnnn { spintent } { spdiv-empty-argument }
  { The~arguments~for~\token_to_str:N \spdiv~cannot~be~empty. }
  {
    You~must~provide~both~a~dividend~and~a~divisor.\\
  }
```

(*End of definition for* `spdiv-empty-argument`.)

11.22.3 Internal functions for \spdiv

__spintent_spdiv_render_op:nn

The function __spintent_spdiv_render_op:nn parses both {⟨*arguments*⟩} through the \spnum command and embeds the semantic operator from \spdsym.

```
\cs_new_protected:Nn \__spintent_spdiv_render_op:nn
  {
    \spnum {#1}
    \MathMLintent { \l__spintent_spdsym_read_sym_tl }
      {
        \l__spintent_spdsym_symbol_tl
      }
    \spnum {#2}
  }
```

(*End of definition for* __spintent_spdiv_render_op:nn.)

__spintent_spdiv_render_parens:nn

The function __spintent_spdiv_render_parens:nn handles the structural parentheses around the division.

```
\cs_new_protected:Nn \__spintent_spdiv_render_parens:nn
  {
    \bool_if:NTF \l__spintent_spdiv_read_parens_bool
      {
        ( \__spintent_spdiv_render_op:nn {#1} {#2} )
      }
      {
        \MathMLintent { :silent } { ( }
        \__spintent_spdiv_render_op:nn {#1} {#2}
        \MathMLintent { :silent } { ) }
      }
  }
```

(*End of definition for* __spintent_spdiv_render_parens:nn.)

__spintent_spdiv_process:nnn

The function __spintent_spdiv_process:nnn sets ⟨*keys*⟩, validates {⟨*arguments*⟩}, and decides on parenthesis wrapping.

```
\cs_new_protected:Nn \__spintent_spdiv_process:nnn
  {
    \keys_set:nn { spintent / spdiv } { #1 }
    \tl_if_blank:nTF { #2 }
      { \msg_error:nnn { spintent } { spdiv-empty-argument } }
      {
        \tl_if_blank:nTF { #3 }
          { \msg_error:nnn { spintent } { spdiv-empty-argument } }
          {
            \bool_if:NTF \l__spintent_spdiv_wrap_parens_bool
              { \__spintent_spdiv_render_parens:nn {#2} {#3} }
              { \__spintent_spdiv_render_op:nn {#2} {#3} }
          }
      }
  }
```

(*End of definition for* __spintent_spdiv_process:nnn.)

\spdiv

Defines the user command \spdiv. It strictly relies on *math mode*.

```
\NewDocumentCommand \spdiv { O{} m m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spdiv_process:nnn { #1 } { #2 } { #3 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spdiv } }
  }
```

(*End of definition for* \spdiv. *This function is documented on page* 13.)

11.23 The command \spmul

The user-level command \spmul formats multiplication representations. It takes two mandatory arguments: the {⟨*first factor*⟩} and the {⟨*second factor*⟩}. Both {⟨*arguments*⟩} are internally processed by the \spnum command to ensure proper digit separation. This command enforces strict validation, ensuring that neither the {⟨*first factor*⟩} nor the {⟨*second factor*⟩} is *blank*.

©2026 by Pablo González L

11.23.1 Definition of keys for \spmul

cifras-sep
read-sign
read-period-sep
notation-sym
set-sym
read-sym
wrap-parens
read-parens

We define the ⟨*keys*⟩ wrap-parens and read-parens for \spmul command. Number-related ⟨*keys*⟩ cifras-sep, read-sign, read-period-sep and notation-sym are forwarded to \spnum command using .meta:nn, while semantic operator ⟨*keys*⟩ set-sym and read-sym are forwarded to the \spmsym command using .meta:nn.

```
\keys_define:nn { spintent / spmul }
  {
    cifras-sep      .meta:nn = { spintent / spnum  } { cifras-sep       = {#1} },
    read-sign       .meta:nn = { spintent / spnum  } { read-sign        = {#1} },
    read-period-sep .meta:nn = { spintent / spnum  } { read-period-sep  = {#1} },
    notation-sym    .meta:nn = { spintent / spnum  } { notation-sym     = {#1} },
    set-sym         .meta:nn = { spintent / spmsym } { set-sym          = {#1} },
    read-sym        .meta:nn = { spintent / spmsym } { read-sym         = {#1} },
    wrap-parens     .bool_set:N = \l__spintent_spmul_wrap_parens_bool,
    wrap-parens     .initial:n  = false,
    wrap-parens     .value_required:n = true,
    read-parens     .bool_set:N = \l__spintent_spmul_read_parens_bool,
    read-parens     .initial:n  = true,
    read-parens     .value_required:n = true,
    unknown         .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spmul }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `cifras-sep and others`.)

11.23.2 Definition of error messages

spmul-empty-argument

Raised when either the {⟨*first factor*⟩} or the {⟨*second factor*⟩} is missing or *blank*.

```
\msg_new:nnnn { spintent } { spmul-empty-argument }
  { The~arguments~for~\token_to_str:N \spmul~cannot~be~empty. }
  {
    You~must~provide~both~first~factor~and~second~factor.\\
  }
```

(*End of definition for* `spmul-empty-argument`.)

11.23.3 Internal functions for \spmul

__spintent_spmul_render_op:nn

The function __spintent_spmul_render_op:nn parses both {⟨*arguments*⟩} through the \spnum command and embeds the semantic operator from \spmsym.

```
\cs_new_protected:Nn \__spintent_spmul_render_op:nn
  {
    \spnum {#1}
    \MathMLintent { \l__spintent_spmsym_read_sym_tl }
      {
        \l__spintent_spmsym_symbol_tl
      }
    \spnum {#2}
  }
```

(*End of definition for* `\__spintent_spmul_render_op:nn`.)

__spintent_spmul_render_parens:nn

The function __spintent_spmul_render_parens:nn handles the structural parentheses around the multiplication.

```
\cs_new_protected:Nn \__spintent_spmul_render_parens:nn
  {
    \bool_if:NTF \l__spintent_spmul_read_parens_bool
      {
        ( \__spintent_spmul_render_op:nn {#1} {#2} )
      }
      {
        \MathMLintent { :silent } { ( }
        \__spintent_spmul_render_op:nn {#1} {#2}
        \MathMLintent { :silent } { ) }
      }
  }
```

(*End of definition for* `\__spintent_spmul_render_parens:nn`.)

__spintent_spmul_process:nnn

The function __spintent_spmul_process:nnn sets ⟨*keys*⟩, validates {⟨*arguments*⟩}, and decides on parenthesis wrapping.

```
\cs_new_protected:Nn \__spintent_spmul_process:nnn
  {
    \keys_set:nn { spintent / spmul } { #1 }
    \tl_if_blank:nTF { #2 }
      { \msg_error:nnn { spintent } { spmul-empty-argument } }
      {
        \tl_if_blank:nTF { #3 }
          { \msg_error:nnn { spintent } { spmul-empty-argument } }
          {
            \bool_if:NTF \l__spintent_spmul_wrap_parens_bool
              { \__spintent_spmul_render_parens:nn {#2} {#3} }
              { \__spintent_spmul_render_op:nn {#2} {#3} }
          }
      }
  }
```

(*End of definition for* __spintent_spmul_process:nnn.)

\spmul

Defines the user command \spmul. It strictly relies on *math mode*.

```
\NewDocumentCommand \spmul { O{} m m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spmul_process:nnn { #1 } { #2 } { #3 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spmul } }
  }
```

(*End of definition for* \spmul. *This function is documented on page* 13.)

11.24 The commands \spmcd and \spmcm

The user-level commands \spmcd and \spmcm provide a semantic interface for the Greatest Common Divisor (MCD in Spanish) and the Least Common Multiple (MCM in Spanish). They delegate argument validation and calculation to the Lua module spintent.lua, constructing a valid MathCAT intent.

11.24.1 Variables for \spmcd and \spmcm

\l__spintent_spmcm_spmcd_luaset_error_str
\l__spintent_mc_args_seq
\l__spintent_spmcd_print_result_bool
\l__spintent_spmcm_print_result_bool
\l__spintent_spmcd_read_terse_bool
\l__spintent_spmcm_read_terse_bool
\l__spintent_spmcd_print_upper_bool
\l__spintent_spmcm_print_upper_bool
\l__spintent_spmcd_prefix_tl
\l__spintent_spmcm_prefix_tl
\l__spintent_spmcd_suffix_tl
\l__spintent_spmcm_suffix_tl

Internal variables generated dynamically via a comma-separated list to avoid code duplication for both symmetric commands.

```
\str_new:N \l__spintent_spmcm_spmcd_luaset_error_str
\seq_new:N \l__spintent_mc_args_seq
\clist_map_inline:nn { mcd , mcm }
  {
    \tl_new:c   { l__spintent_sp #1 _luaset_ #1 _value_tl }
    \bool_new:c { l__spintent_sp #1 _print_result_bool }
    \bool_new:c { l__spintent_sp #1 _read_terse_bool }
    \bool_new:c { l__spintent_sp #1 _print_upper_bool }
    \tl_new:c   { l__spintent_sp #1 _prefix_tl }
    \tl_new:c   { l__spintent_sp #1 _suffix_tl }
  }
```

(*End of definition for* \l__spintent_spmcm_spmcd_luaset_error_str *and others.*)

11.24.2 Definition of error messages

mc-invalid-arguments

Exception triggered when the comma-separated sequence contains negative integers, floats, or alphabetical characters.

```
\msg_new:nnnn { spintent } { mc-invalid-arguments }
  {
    Invalid~arguments~provided~to~#1.
  }
  {
    The~arguments~must~be~a~comma-separated~list. \\
  }
```

(*End of definition for* mc-invalid-arguments.)

11.24.3 Keys for \spmcd and \spmcm

result
prefix
read-cmd
upper-case
sample-arg
silent-cmd

We define the shared ⟨*keys*⟩ result, prefix, read-cmd, upper-case, sample-arg and silent-cmd.

```
\clist_map_inline:nn { mcd , mcm }
  {
    \keys_define:nn { spintent / sp #1 }
      {
        result             .bool_set:c = { l__spintent_sp #1 _print_result_bool },
        result             .initial:n  = false,
        result             .value_required:n = true,
        prefix             .code:n =
          { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
        prefix             .initial:n = ,
        prefix             .value_required:n = true,
        read-cmd           .choices:nn =
          { terse , clear }
          {
            \int_case:nn { \l_keys_choice_int }
              {
                {1} { \bool_set_true:c  { l__spintent_sp #1 _read_terse_bool } }
                {2} { \bool_set_false:c { l__spintent_sp #1 _read_terse_bool } }
              }
          },
        read-cmd           .initial:n = clear,
        read-cmd           .value_required:n = true,
        read-cmd / unknown .code:n =
          \msg_error:nneee { spintent } { unknown-choice }
            { read-cmd } { terse , ~clear } { \exp_not:n {##1} },
        upper-case         .bool_set:c = { l__spintent_sp #1 _print_upper_bool },
        upper-case         .initial:n  = false,
        upper-case         .value_required:n = true,
        sample-arg         .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
        sample-arg         .initial:n  = false,
        sample-arg         .value_required:n = true,
        silent-cmd         .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
        silent-cmd         .initial:n  = false,
        silent-cmd         .value_required:n = true,
        unknown            .code:n =
          {
            \str_set:Nn \l__spintent_command_name_str { sp #1 }
            \__spintent_unknown_keys_cmd:n {##1}
          },
      }
  }
```

(*End of definition for* `result` *and others.*)

11.24.4 Internal functions for \spmcd and \spmcm

__spintent_mc_process_args:nn

The function __spintent_mc_process_args:nn parses the operation '#1' and its arguments '#2', injecting MathML intents.

```
\cs_new_protected:Nn \__spintent_mc_process_args:nn
  {
    \seq_set_from_clist:Nn \l__spintent_mc_args_seq {#2}
    \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
      {
        \seq_set_map:NNn \l__spintent_mc_args_seq \l__spintent_mc_args_seq
          { \MathMLintent { :silent } { ##1 } }
        \MathMLintent { entre:silent } { ( }
        \seq_use:Nnnn \l__spintent_mc_args_seq
          { \MathMLintent { _y:silent } { , } }
          { \MathMLintent { :silent }   { , } }
          { \MathMLintent { _y:silent } { , } }
      }
      {
        \MathMLintent { entre } { ( }
        \seq_use:Nnnn \l__spintent_mc_args_seq
          { \MathMLintent { _y }      { , } }
          { \MathMLintent { :silent } { , } }
          { \MathMLintent { _y }      { , } }
      }
    \MathMLintent { :silent } { ) }
  }
```

©2026 by Pablo González L

(*End of definition for* `\__spintent_mc_process_args:nn`.)

`\__spintent_spmc_process:nnnn`

The internal function `\__spintent_spmc_process:nnnn` serves as a shared central handler for both the `\spmcd` and `\spmcm` commands. It takes the command identifier '`#1`', its full reading string '`#2`', the local ⟨*keys*⟩ '`#3`', and the optional values '`#4`'. It dynamically resolves variables using the identifier to build the `MathML intent` string (including any prefixes or suffixes) and prints the math operator. If values are provided in '`#4`', it calculates the result using the Lua backend. If an error occurs, it throws the `mc-invalid-arguments` error; otherwise, it processes the arguments and conditionally prints the calculated result.

```
\cs_new_protected:Nn \__spintent_spmc_process:nnnn
  {
    \keys_set:nn { spintent / sp #1 } { #3 }
    \MathMLintent
      {
        \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
          { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
        \bool_if:cTF { l__spintent_sp #1 _read_terse_bool }
          { #1 } { #2 }
        \bool_if:cT { l__spintent_sp #1 _silent_cmd_bool }
          {
            \tl_if_novalue:nTF {#4}
              { :silent }
              { \bool_if:cT { l__spintent_sp #1 _sample_arg_bool } { :silent } }
          }
      }
      {
        \operatorname
          {
            \bool_if:cTF { l__spintent_sp #1 _print_upper_bool }
              { \text_uppercase:n {#1} } { #1 }
          }
      }
    \tl_if_novalue:nF { #4 }
      {
        \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
          {
            \__spintent_mc_process_args:nn {#1} {#4}
          }
          {
            \use:c { __spintent_luafun_calculate_ #1 :n } { #4 }
            \str_if_eq:VnTF \l__spintent_spmcm_spmcd_luaset_error_str { true }
              {
                \msg_error:nne { spintent } { mc-invalid-arguments } { \exp_not:c { sp #1 } }
              }
              {
                \__spintent_mc_process_args:nn {#1} {#4}
                \bool_if:cT { l__spintent_sp #1 _print_result_bool }
                  {
                    \MathMLintent { es-igual-a } { = }
                    \tl_use:c { l__spintent_sp #1 _luaset_ #1 _value_tl }
                  }
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_spmc_process:nnnn`.)

`\spmcd` `\spmcm`

The commands `\spmcd` and `\spmcm` act as the public interface wrappers for typesetting the greatest common divisor and least common multiple. They enforce math mode, open a local group, and defer execution to the main processing macro `\__spintent_spmc_process:nnnn`, passing along the appropriate reading strings and arguments.

```
\NewDocumentCommand \spmcd { O{} d() }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spmc_process:nnnn { mcd } { máximo-común-divisor } {#1} {#2}
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spmcd } }
  }
```

```
\NewDocumentCommand \spmcm { O{} d() }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spmc_process:nnnn { mcm } { mínimo-común-múltiplo } {#1} {#2}
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spmcm } }
  }
```

(*End of definition for* `\spmcd` *and* `\spmcm`*. These functions are documented on page 14.*)

11.25 The commands \spnM and \spnD

The user-level commands `\spnM` and `\spnD` provide a semantic interface for the Multiples of a number (*"múltiplos"* in Spanish) and the Divisors of a number (*"divisores"* in Spanish). They delegate argument validation and calculation to the Lua module `spintent.lua`, constructing a valid `MathCAT` intent.

11.25.1 Variables for \spnM and \spnD

`\l__spintent_md_out_seq`
`\l__spintent_spnM_luaset_result_tl`
`\l__spintent_spnD_luaset_result_tl`
`\l__spintent_spnM_luaset_is_truncated_str`
`\l__spintent_spnD_luaset_is_truncated_str`
`\l__spintent_spnM_print_result_bool`
`\l__spintent_spnD_print_result_bool`
`\l__spintent_spnM_silent_cmd_bool`
`\l__spintent_spnD_silent_cmd_bool`
`\l__spintent_spnM_prefix_tl`
`\l__spintent_spnD_prefix_tl`
`\l__spintent_spnM_print_result_num_tl`
`\l__spintent_spnD_print_result_num_tl`

Internal variables generated dynamically for multiples and divisors.

```
\seq_new:N \l__spintent_md_out_seq
\clist_map_inline:nn { nM , nD }
  {
    \tl_new:c   { l__spintent_sp #1 _luaset_result_tl        }
    \str_new:c  { l__spintent_sp #1 _luaset_is_truncated_str }
    \bool_new:c { l__spintent_sp #1 _print_result_bool       }
    \bool_new:c { l__spintent_sp #1 _silent_cmd_bool         }
    \tl_new:c   { l__spintent_sp #1 _prefix_tl               }
    \tl_new:c   { l__spintent_sp #1 _print_result_num_tl     }
  }
```

(*End of definition for* `\l__spintent_md_out_seq` *and others.*)

11.25.2 Keys for \spnM and \spnD

`result`
`silent-cmd`
`result-num`
`prefix`
`sample-arg`

We define the shared ⟨*keys*⟩ `result`, `silent-cmd`, `result-num` and `prefix`.

```
\clist_map_inline:nn { nM , nD }
  {
    \keys_define:nn { spintent / sp #1 }
      {
        result     .bool_set:c = { l__spintent_sp #1 _print_result_bool },
        result     .initial:n  = false,
        result     .value_required:n = true,
        silent-cmd .bool_set:c = { l__spintent_sp #1 _silent_cmd_bool },
        silent-cmd .initial:n  = false,
        silent-cmd .value_required:n = true,
        result-num .tl_set:c   = { l__spintent_sp #1 _print_result_num_tl },
        result-num .initial:n  = ,
        prefix     .code:n =
          { \__spintent_sanitize_affix:cn { l__spintent_sp #1 _prefix_tl } {##1} },
        prefix     .initial:n = ,
        prefix     .value_required:n = true,
        sample-arg .bool_set:c = { l__spintent_sp #1 _sample_arg_bool },
        sample-arg .initial:n  = false,
        sample-arg .value_required:n = true,
        unknown    .code:n =
          {
            \str_set:Nn \l__spintent_command_name_str { sp #1 }
            \__spintent_unknown_keys_cmd:n {##1}
          },
      }
  }
```

(*End of definition for* `result` *and others.*)

11.25.3 Definition of error messages for \spnM and \spnD

`md-invalid-argument`

Exception triggered when the argument for multiples or divisors is invalid.

```
\msg_new:nnnn { spintent } { md-invalid-argument }
  { Invalid~argument~provided~to~#1. }
  {
    The~argument~must~be~a~single~positive~integer. \\
  }
```

(*End of definition for* `md-invalid-argument`.)

©2026 by Pablo González L

11.25.4 Internal functions for \spnM and \spnD

__spintent_spnM_spnD_process:nnnnn

The internal function __spintent_spnM_spnD_process:nnnnn acts as the primary entry point for the multiples and divisors commands. It takes the command identifier '#1', its printed mathematical symbol '#2', its base reading string '#3', the local key-value options '#4', and the optional argument '#5'. Its sole purpose is to evaluate and set the provided keys within the current scope before delegating the core branching and typesetting logic to __spintent_spn_M_and_D_exec:nnnn.

```
\cs_new_protected:Nn \__spintent_spnM_spnD_process:nnnnn
  {
    \keys_set:nn { spintent / sp #1 } { #4 }
    \__spintent_spnM_spnD_exec:nnnn { #1 } { #2 } { #3 } { #5 }
  }
```

(*End of definition for* __spintent_spnM_spnD_process:nnnnn.)

__spintent_spnM_spnD_intent:nn

The internal function __spintent_spnM_spnD_intent:nn resolves the main reading intent string. It takes the command identifier '#1' (such as "nM" or "nD") and the base reading string '#2'. It automatically prepends the localized prefix if defined and handles specific string normalizations, such as ensuring the correct accentuation for "múltiplos".

```
\cs_new:Nn \__spintent_spnM_spnD_intent:nn
  {
    \tl_if_empty:cF { l__spintent_sp #1 _prefix_tl }
      { \tl_use:c { l__spintent_sp #1 _prefix_tl } - }
    \str_case:nnF { #2 }
      { { multiplos } { múltiplos } }
      { #2 }
  }
```

(*End of definition for* __spintent_spnM_spnD_intent:nn.)

__spintent_spnM_spnD_silent:nn

The internal function __spintent_spn_M_and_D_silent:nn prints the atomized notation structure in full silent mode. It takes the mathematical symbol '#1' (such as "D" or "M") and the variable or explicit argument '#2'. Every component is wrapped in a silent intent, and an invisible separator is injected between the parentheses to prevent assistive technologies from applying default contextual reading rules like "de".

```
\cs_new_protected:Nn \__spintent_spnM_spnD_silent:nn
  {
    \MathMLintent { :silent } { \operatorname { #1 } }
    \MathMLintent { :silent } { ( }
    \MathMLintent { :silent } { \__spintent_invisible_sep: }
    \MathMLintent { :silent } { #2 }
    \MathMLintent { :silent } { ) }
  }
```

(*End of definition for* __spintent_spnM_spnD_silent:nn.)

__spintent_spnM_spnD_result:n

The internal function __spintent_spn_M_and_D_result:n handles the calculation and accessible formatting of the computed mathematical set. It takes the command identifier '#1'. It invokes the corresponding Lua backend calculation, maps the resulting clist into a sequence, and formats the final printed output utilizing custom accessibility intents for both separators and conjunctions. It also safely manages truncation indicators.

```
\cs_new_protected:Nn \__spintent_spnM_spnD_result:n
  {
    \use:c { __spintent_luafun_calcular_ #1 :nn }
      { \l__spintent_luaset_part_int_tl }
      { \tl_use:c { l__spintent_sp #1 _print_result_num_tl } }
    \MathMLintent { son } { = \{ }
    \seq_set_from_clist:Ne
      \l__spintent_md_out_seq { \tl_use:c { l__spintent_sp #1 _luaset_result_tl } }
    \seq_use:Nnnn \l__spintent_md_out_seq
      { \MathMLintent { _y }      { , } }
      { \MathMLintent { :silent } { , } }
      { \MathMLintent { _y }      { , } }
    \str_if_eq:vnT
      { l__spintent_sp #1 _luaset_is_truncated_str } { true }
        {
          \MathMLintent { :silent } { , } ~ \dots
        }
    \MathMLintent { :silent } { \} }
  }
```

(*End of definition for* __spintent_spnM_spnD_result:n.)

__spintent_spnM_spnD_exec:nnnn

The function __spintent_spn_M_and_D_exec:nnnn implements the multiples and divisors commands. It takes the {⟨*argument*⟩} '#1' (the command name), the {⟨*argument*⟩} '#2' (the mathematical symbol), the base reading {⟨*argument*⟩} '#3', and the optional {⟨*argument*⟩} '#4'. If no {⟨*argument*⟩} is provided, it evaluates the key silent-cmd using the default token 'n'. If an {⟨*argument*⟩} is given, it evaluates the key sample-arg. If true, it processes the {⟨*argument*⟩} directly and evaluates the key silent-cmd. If false, it bypasses the *silent mode*, validates the {⟨*argument*⟩} as a natural number, and evaluates the key result optionally calculate and append the resulting set.

```
\cs_new_protected:Nn \__spintent_spnM_spnD_exec:nnnn
  {
    \tl_if_novalue:nTF { #4 }
      {
        \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
          { \__spintent_spnM_spnD_silent:nn { #2 } { n } }
          {
            \MathMLintent
              { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
              { \operatorname { #2 } ( n ) }
          }
      }
      {
        \bool_if:cTF { l__spintent_sp #1 _sample_arg_bool }
          {
            \bool_if:cTF { l__spintent_sp #1 _silent_cmd_bool }
              { \__spintent_spnM_spnD_silent:nn { #2 } { #4 } }
              {
                \MathMLintent
                  { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
                  { \operatorname { #2 } } ( #4 )
              }
          }
          {
            \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
            \str_if_eq:VnTF \l__spintent_luaset_has_natural_str { true }
              {
                \MathMLintent
                  { \__spintent_spnM_spnD_intent:nn { #1 } { #3 } }
                  { \operatorname { #2 } } ( #4 )
                \bool_if:cT { l__spintent_sp #1 _print_result_bool }
                  { \__spintent_spnM_spnD_result:n { #1 } }
              }
              {
                \msg_error:nne { spintent } { md-invalid-argument } { \exp_not:c { sp #1 } }
              }
          }
      }
  }
```

(*End of definition for* __spintent_spnM_spnD_exec:nnnn.)

\spnM
\spnD

The commands \spnM and \spnD act as the public interface wrappers for typesetting multiples and divisors, respectively. They enforce math mode, open a local group, and defer execution to the function __spintent_spnM_spnD_process:nnnnn, passing along the optional ⟨*keys*⟩ '#1' and the numeric {⟨*argument*⟩} '#2'.

```
\NewDocumentCommand \spnM { O{} d() }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spnM_spnD_process:nnnnn { nM } { M } { multiplos } {#1} {#2}
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spnM } }
  }
\NewDocumentCommand \spnD { O{} d() }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spnM_spnD_process:nnnnn { nD } { D } { divisores } {#1} {#2}
        \group_end:
```

```
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spnD } }
  }
```

(*End of definition for* `\spnM` *and* `\spnD`. *These functions are documented on page* 14.)

11.26 The command \spang

The command `\spang` is the main public command for typesetting and tagging sexagesimal angle expressions (degrees, minutes, and seconds). It parses the input using the Lua function `\__spintent_luafun_spang_parse:n`. This separates the degrees, minutes, and seconds to construct a MathML `intent` tree with the correct postfixes for NVDA/MathCAT.

`\l__spintent_spang_luaset_grado_str`
`\l__spintent_spang_luaset_minuto_str`
`\l__spintent_spang_luaset_segundo_str`
`\l__spintent_spang_luaset_error_str`

These internal string variables store the values returned from the Lua module `spintent.lua`. They hold the extracted numerical values for degrees, minutes, and seconds, as well as the error status used during formatting.

```
\str_new:N  \l__spintent_spang_luaset_grado_str
\str_new:N  \l__spintent_spang_luaset_minuto_str
\str_new:N  \l__spintent_spang_luaset_segundo_str
\str_new:N  \l__spintent_spang_luaset_error_str
```

(*End of definition for* `\l__spintent_spang_luaset_grado_str` *and others.*)

11.26.1 Definition of error messages for \spang

`invalid-angle-format`

The error message `invalid-angle-format` defines the text displayed when an invalid angle input is detected. It is triggered when an argument fails the format checks, does not match the expected sexagesimal pattern, or contains invalid measurement symbols.

```
\msg_new:nnnn { spintent } { invalid-angle-format }
  { Invalid~angle~format~in~#1. }
  {
    The~argument~'#2'~could~not~be~parsed. \\
  }
```

(*End of definition for* `invalid-angle-format`.)

11.26.2 Internal functions for \spang

`\__spintent_spang_process:n`

The internal function `\__spintent_spang_process:n` parses the angle input using `\__spintent_luafun_spang_parse:n` after resetting the error flag in `\l__spintent_spang_luaset_error_str`. If an invalid format is detected, it issues the `invalid-angle-format` error. Otherwise, it checks the extracted string variables `\l__spintent_spang_luaset_grado_str`, `\l__spintent_spang_luaset_minuto_str`, and `\l__spintent_spang_luaset_segundo_str`. For each non-empty variable, it constructs the corresponding MathML `intent` structure with the correct measurement symbols and postfixes for NVDA/MathCAT.

```
\cs_new_protected:Nn \__spintent_spang_process:n
  {
    \str_set:Nn \l__spintent_spang_luaset_error_str { false }
    \__spintent_luafun_spang_parse:n { #1 }
    \str_if_eq:VnTF \l__spintent_spang_luaset_error_str { true }
      {
        \msg_error:nnnn { spintent } { invalid-angle-format } { \spang } { #1 }
      }
      {
        \str_if_empty:NF \l__spintent_spang_luaset_grado_str
          {
            \spunit{ \l__spintent_spang_luaset_grado_str °}
          }
        \str_if_empty:NF \l__spintent_spang_luaset_minuto_str
          {
            \MathMLintent { minutos : postfix($m) }
              {
                \MathMLarg { m }
                  {
                    \spunit{ \l__spintent_spang_luaset_minuto_str '}
                  }
              }
          }
        \str_if_empty:NF \l__spintent_spang_luaset_segundo_str
          {
            \MathMLintent { segundos : postfix($s) }
              {
                \MathMLarg { s }
                  {
```

```
                    \spunit { \l__spintent_spang_luaset_segundo_str "}
                  }
              }
          }
      }
  }
```

(*End of definition for* `\__spintent_spang_process:n`.)

`\spang` The command `\spang` is the public interface for formatting sexagesimal angle expressions. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls `\__spintent_spang_process:n` to process the {⟨*argument*⟩}.

```
\NewDocumentCommand \spang { m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spang_process:n { #1 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spang } }
  }
```

(*End of definition for* `\spang`. *This function is documented on page 16.*)

11.27 The command \spnZ

The user-level command `\spnZ` formats integers (the set $\mathbb{Z}$). It includes specific options to automatically enclose numbers (particularly negative ones) in visual parentheses and allows you to control whether these parentheses are read by MathCAT.

This command verifies that the {⟨*argument*⟩} is indeed an *integer*, checking for the complete absence of decimal separators or attached units of measurement.

11.27.1 Definition of keys for \spnZ

`cifras-sep` `read-sign` `wrap-parens` `read-parens` Now we define the ⟨*keys*⟩ `cifras-sep` and `read-sign`. These are `.meta:nn` ⟨*keys*⟩ passed directly to the `\spnum` command. We also introduce `wrap-parens`, which determines whether the number is enclosed in parentheses, and `read-parens`, which controls if these parentheses are read by MathCAT.

```
\keys_define:nn { spintent / spnZ }
  {
    cifras-sep  .meta:nn = { spintent / spnum } { cifras-sep = {#1} },
    read-sign   .meta:nn = { spintent / spnum } { read-sign  = {#1} },
    wrap-parens .bool_set:N = \l__spintent_spnz_wrap_parens_bool,
    wrap-parens .initial:n  = false,
    wrap-parens .value_required:n = true,
    read-parens .bool_set:N = \l__spintent_spnz_read_parens_bool,
    read-parens .initial:n  = true,
    read-parens .value_required:n = true,
    unknown     .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spnZ }
        \__spintent_unknown_keys_cmd:n {#1}
      },
  }
```

(*End of definition for* `cifras-sep` *and others.*)

11.27.2 Definition of error messages

`spnz-not-integer` Raised when the {⟨*argument*⟩} contains decimal separators or attached units, which conflict with the constraints of a pure integer representation.

```
\msg_new:nnnn { spintent } { spnz-not-integer }
  { The~value~'#1'~is~not~a~valid~integer. }
  {
    Command~\token_to_str:N \spnZ~only~accepts~integers.\\
   }
```

(*End of definition for* `spnz-not-integer`.)

11.27.3 Internal functions for \spnZ

__spintent_spnz_render_parens:

The function __spintent_spnz_render_parens: first checks the state of the boolean variable \l__spintent_spnz (controlled by the read-parens key). If it is true, it prints the number surrounded by parentheses. Otherwise, it applies the :silent *function intent* to the parentheses while printing the number.

```
\cs_new_protected:Nn \__spintent_spnz_render_parens:
  {
    \bool_if:NTF \l__spintent_spnz_read_parens_bool
      {
        ( \__spintent_spnum_print_num_start: )
      }
      {
        \MathMLintent { :silent } { ( }
        \__spintent_spnum_print_num_start:
        \MathMLintent { :silent } { ) }
      }
  }
```

(*End of definition for* __spintent_spnz_render_parens:.)

__spintent_spnz_process:nn

The function __spintent_spnz_process:nn first processes the ⟨*keys*⟩ passed in '#1' and then calls __spintent_luafun_clean_split_and_set:n on '#2'. It verifies that the input is a valid integer by checking if the variable \l__spintent_luaset_has_integer_str is true. If not true, an "error" is raised.

If it is true, it checks whether \l__spintent_luaset_part_int_tl is *empty* and raises an "error" if so. Otherwise, it checks \l__spintent_spnz_wrap_parens_bool set by wrap-parens and calls either the function __spintent_spnz_render_parens: or __spintent_spnum_print_num_start:.

```
\cs_new_protected:Nn \__spintent_spnz_process:nn
  {
    \keys_set:nn { spintent / spnZ } {#1}
    \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#2} }
    \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
      {
        \tl_if_empty:NTF \l__spintent_luaset_part_int_tl
          { \msg_error:nnn { spintent } { spnum-no-digits } {#2} }
          {
            \bool_if:NTF \l__spintent_spnz_wrap_parens_bool
              { \__spintent_spnz_render_parens: }
              { \__spintent_spnum_print_num_start: }
          }
      }
      { \msg_error:nnn { spintent } { spnz-not-integer } {#2} }
  }
```

(*End of definition for* __spintent_spnz_process:nn.)

\spnZ

Defines the user command \spnZ, which is executed within a *group* and strictly enforces *math mode*.

```
\NewDocumentCommand \spnZ { O{} m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spnz_process:nn { #1 } { #2 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spnZ } }
  }
```

(*End of definition for* \spnZ. *This function is documented on page* 16.)

11.28 The command \spmQ

The command \spmQ typesets mixed numbers within Spanish mathematical contexts (e.g., $3\frac{1}{2}$). It accepts an optional key-value list and three mandatory arguments: the integer part, the numerator, and the denominator.

11.28.1 Definition of variables for \spmQ

\l__spintent_spmQ_join_word_str
\l__spintent_spmQ_math_style_tl
\l__spintent_spnQ_intent_read_sign_str
\l__spintent_spnQ_wrap_parens_l_tl
\l__spintent_spnQ_wrap_parens_r_tl
\l__spintent_saved_luamml_flag_int.

The variable \l__spintent_spmQ_join_word_str stores the value set by the key join-word which will be verbalized by MathCAT as a grammatical connector between the integer and fractional parts.

The variable \l__spintent_spmQ_math_style_tl stores the current *mathematical style* used by the functions __spintent_spmQ_wrap_math:n and __spintent_spmQ_capture_math_style:

The variable \l__spintent_spnQ_intent_read_sign_str stores the :intent passed to \MathMLintent when the signs '+' or '−' are present in integer part used by the function __spintent_spmQ_match_sign:.

The variable \l__spintent_spnQ_wrap_parens_l_tl and \l__spintent_spnQ_wrap_parens_r_tl stores the size parens set by the function __spintent_spmQ_set_wrap_parens_size: used by the key wrap-parens.

```
\str_new:N \l__spintent_spmQ_join_word_str
\tl_new:N  \l__spintent_spmQ_math_style_tl
\str_new:N \l__spintent_spnQ_intent_read_sign_str
\tl_new:N  \l__spintent_spnQ_wrap_parens_l_tl
\tl_new:N  \l__spintent_spnQ_wrap_parens_r_tl
\int_new:N \l__spintent_saved_luamml_flag_int
```

(*End of definition for* \l__spintent_spmQ_join_word_str *and others.*)

11.28.2 Definition of error messages for \spmQ

spmQ-zero-denominator
spmQ-not-integer

The error message spmQ-zero-denominator is raised when a zero denominator is passed to \spmQ. The second error is returned when the integer part is not actually an integer.

```
\msg_new:nnnn { spintent } { spmQ-zero-denominator }
  { Zero~denominator~in~\token_to_str:N \spmQ. }
  {
    The~denominator~of~a~fraction~cannot~be~zero.\\
  }
\msg_new:nnnn { spintent } { spmQ-not-integer }
  { The~value~'#1'~is~not~a~valid~integer. }
  {
    Command~\token_to_str:N \spnQ \c_space_tl only~accepts~integers.\\
  }
```

(*End of definition for* spmQ-zero-denominator *and* spmQ-not-integer.)

11.28.3 Definition of keys for \spmQ

join-word
use-spnZ
wrap-parens
read-parens
print-dfrac
unknown

Now we add the keys join-word, use-spnZ, wrap-parens, read-parens and print-dfrac.

```
\keys_define:nn { spintent / spmQ }
  {
    join-word         .choices:nn =
      { entero , enteros , y }
      {
        \int_case:nn { \l_keys_choice_int }
          {
            {1} { \str_set:Nn \l__spintent_spmQ_join_word_str { entero } }
            {2} { \str_set:Nn \l__spintent_spmQ_join_word_str { enteros } }
            {3} { \str_set:Nn \l__spintent_spmQ_join_word_str { y } }
          }
      },
    join-word         .initial:n  = y,
    join-word         .value_required:n = true,
    join-word/unknown .code:n =
      \msg_error:nneee { spintent } { unknown-choice }
        { join-word } { entero, ~enteros, ~y } { \exp_not:n {#1} },
    use-spnZ          .bool_set:N = \l__spintent_spmQ_use_spnZ_bool,
    use-spnZ          .initial:n = true,
    use-spnZ          .value_required:n = true,
    print-dfrac       .bool_set:N = \l__spintent_spmQ_print_dfrac_bool,
    print-dfrac       .initial:n = false,
    print-dfrac       .value_required:n = true,
    wrap-parens       .bool_set:N = \l__spintent_spnQ_wrap_parens_bool,
    wrap-parens       .initial:n  = false,
    wrap-parens       .value_required:n = true,
    read-parens       .bool_set:N = \l__spintent_spnQ_read_parens_bool,
    read-parens       .initial:n  = true,
    read-parens       .value_required:n = true,
    unknown           .code:n =
      {
        \str_set:Nn \l__spintent_command_name_str { spmQ }
```

```
      \__spintent_unknown_keys_cmd:n {#1}
    },
}
```

(*End of definition for* `join-word` *and others.*)

11.28.4 Internal functions for `\spmQ`

`\__spintent_spmQ_set_wrap_parens_size:`

The function `\__spintent_spmQ_set_wrap_parens_size:` capture the current *mathematical style* using the primitive `\mathstyle` from LuaTEX and sets the variables `\l__spintent_spnQ_wrap_parens_l_tl` and `\l__spintent_spnQ_wrap_parens_r_tl` with the appropriate size for the parentheses.

```
\cs_new_protected:Nn \__spintent_spmQ_set_wrap_parens_size:
  {
    \int_case:nnF { \mathstyle }
      {
        { 0 }
          {
            \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \Bigl \lparen }
            \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \Bigr \rparen }
          }
        { 1 }
          {
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_l_tl { \Bigl \lparen }
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_r_tl { \Bigr \rparen }
          }
        { 2 }
          {
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_l_tl { \bigl \lparen }
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_r_tl { \bigr \rparen }
          }
        { 3 }
          {
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_l_tl { \bigl \lparen }
            \tl_set:Nn  \l__spintent_spnQ_wrap_parens_r_tl { \bigr \rparen }
          }
      }
      {
        \tl_set:Nn \l__spintent_spnQ_wrap_parens_l_tl { \lparen }
        \tl_set:Nn \l__spintent_spnQ_wrap_parens_r_tl { \rparen }
      }
```

If the key `print-dfrac` is active, we must reset the variables `\l__spintent_spnQ_wrap_parens_l_tl` and `\l__spintent_spnQ_wrap_parens_r_tl`.

```
    \bool_if:NT \l__spintent_spmQ_print_dfrac_bool
      {
        \tl_set:Nn  \l__spintent_spnQ_wrap_parens_l_tl { \Bigl \lparen }
        \tl_set:Nn  \l__spintent_spnQ_wrap_parens_r_tl { \Bigr \rparen }
      }
  }
```

(*End of definition for* `\__spintent_spmQ_set_wrap_parens_size:`.)

`\__spintent_spmQ_start_wrap_parens:`
`\__spintent_spmQ_stop_wrap_parens:`

The functions `\__spintent_spmQ_start_wrap_parens:` and `\__spintent_spmQ_stop_wrap_parens:` print with the appropriate size for the parentheses set by the key `wrap-parens`. Only if the variable `\l__spintent_spnQ_read_parens_bool` handled by the key `read-parens` is false, we will execute `\MathMLintent{:silent}`.

```
\cs_new_protected:Nn \__spintent_spmQ_start_wrap_parens:
  {
    \bool_if:NT \l__spintent_spnQ_wrap_parens_bool
      {
        \__spintent_spmQ_set_wrap_parens_size:
        \bool_if:NTF \l__spintent_spnQ_read_parens_bool
          {
            \l__spintent_spnQ_wrap_parens_l_tl
          }
          {
            \MathMLintent { :silent } { \l__spintent_spnQ_wrap_parens_l_tl }
          }
      }
  }
\cs_new_protected:Nn \__spintent_spmQ_stop_wrap_parens:
  {
```

```
    \bool_if:NT \l__spintent_spnQ_wrap_parens_bool
      {
        \__spintent_spmQ_set_wrap_parens_size:
        \bool_if:NTF \l__spintent_spnQ_read_parens_bool
          {
            \l__spintent_spnQ_wrap_parens_r_tl
          }
          {
            \MathMLintent { :silent } { \l__spintent_spnQ_wrap_parens_r_tl }
          }
      }
  }
```

(*End of definition for* `\__spintent_spmQ_start_wrap_parens:` *and* `\__spintent_spmQ_stop_wrap_parens:`.)

`\__spintent_spmQ_join_word:`

The function `\__spintent_spmQ_join_word:` captures the '+' or '−' for integer part sign and sets the `:intent` to `\MathMLintent {`⟨*argument*⟩`}` if present.

```
\cs_new_protected:Nn \__spintent_spmQ_set_join_word:
  {
    \str_case:Vn \l__spintent_spmQ_join_word_str
      {
        { entero  }
          {
            \MathMLintent { _entero } { \c__spintent_invisible_sep_tl }
          }
        { enteros }
          {
            \MathMLintent { _enteros } { \c__spintent_invisible_sep_tl }
          }
        { y       }
          {
            \MathMLintent { _y } { \c__spintent_invisible_sep_tl }
          }
      }
   }
```

(*End of definition for* `\__spintent_spmQ_join_word:`.)

`\__spintent_spmQ_print_spnZ_dfrac:nn`

The function `\__spintent_spmQ_print_spnZ_dfrac:nn` checks the state of the variable `\l__spintent_spmQ_use_spnZ_bool` handled by the key `use-spnZ` along with the variable `\l__spintent_spmQ_print_dfrac_bool` handled by the key `print-dfrac`. According to their values, it prints `\frac` or `\dfrac` using `\spnZ` when necessary.

```
\cs_new_protected:Nn \__spintent_spmQ_print_spnZ_dfrac:nn
  {
    \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
    \str_if_eq:VnT \l__spintent_luaset_has_integer_str { false }
      {
        \msg_error:nn { spintent } { spmQ-not-integer }
      }
    \bool_if:NTF \l__spintent_spmQ_use_spnZ_bool
      {
        \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
          { \spnZ { #1 } } { \spnZ { #2 } }
      }
      {
        \use:c { \bool_if:NT \l__spintent_spmQ_print_dfrac_bool { d } frac }
          { #1 } { #2 }
      }
  }
```

(*End of definition for* `\__spintent_spmQ_print_spnZ_dfrac:nn`.)

`\__spintent_spmQ_print_scaled_int:`

The function `\__spintent_spmQ_print_scaled_int:` will print the integer part scaled and aligned correctly.

```
\cs_new_protected:Nn \__spintent_spmQ_print_scale_int:
  {
    % Deshabilitar luamml mientras se inyecta el noad del entero.
    % Esto evita el <math> fantasma del sub_box procesado en solitario
    % antes de que \frac se expanda.
    % Flag 0 = skip completamente.
    %%\int_set:Nn \l__luamml_flag_int { 0 }
```

```
    \int_set_eq:Nc \l__spintent_saved_luamml_flag_int { l__luamml_flag_int }
    \int_set:Nn \l__luamml_flag_int { 0 }
    \bool_if:NTF \l__spintent_spmQ_print_dfrac_bool
      {
        \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl {dfrac}
      }
      {
        \__spintent_luafun_spmQ_scale_int:Vn \l__spintent_luaset_part_int_tl {frac}
      }
    % Restaurar flag normal (11 = mathml-SE: bits 0,1,3)
    % \int_set:Nn \l__luamml_flag_int { 11 }
    \int_set_eq:cN { l__luamml_flag_int } \l__spintent_saved_luamml_flag_int
  }
```

(*End of definition for* `\__spintent_spmQ_print_scaled_int:`.)

`\__spintent_spmQ_print_sacle_int:n`

The function `\__spintent_spmQ_print_int:n` takes as its arguments the integer part passed in '`#1`'. First, the function `\__spintent_luafun_clean_split_and_set:n` executes on '`#1`' and checks the status of the variable `\l__spintent_luaset_has_integer_str`. If it is `true`, checks if the variable `\l__spintent_luaset_sign_tl` is *empty* and calls the function `\__spintent_spmQ_print_scale_int:` to print the scaled integer part; otherwise, captures the '`+`' or '`-`' for integer part sign, sets the `:intent` to `\MathMLintent` and print.

```
\cs_new_protected:Nn \__spintent_spmQ_print_sacle_int:n
  {
    \__spintent_luafun_clean_split_and_set:n { \exp_not:n {#1} }
    \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
      {
        \tl_if_empty:NTF \l__spintent_luaset_sign_tl
          {
            \__spintent_spmQ_print_scale_int:
          }
          {
            \str_if_eq:VnTF \l__spintent_luaset_sign_tl { + }
              {
                \str_set:Nn
                  \l__spintent_spnQ_intent_read_sign_str { _más:prefix $(e) }
              }
              {
                 \str_set:Nn
                   \l__spintent_spnQ_intent_read_sign_str { _menos:prefix $(e) }
              }
            \MathMLintent { \l__spintent_spnQ_intent_read_sign_str }
              {
                \l__spintent_luaset_sign_tl  \__spintent_invisible_sep: % need
                \MathMLarg { e } { \__spintent_spmQ_print_scale_int: }
              }
          }
      }
      { \msg_error:nn { spintent } { spmQ-not-integer } }
  }
```

(*End of definition for* `\__spintent_spmQ_print_sacle_int:n`.)

`\__spintent_spmQ_print:nnn`

The function `\__spintent_spmQ_render:nnn` takes three arguments: the integer part '`#1`', the numerator '`#2`', and the denominator '`#3`'. First, executes the function `\__spintent_spmQ_tart_wrap_parens:` handled by the key `wrap-parens` to open the parentheses if they are active, then we call the function `\__spintent_spmQ_print_sacle_int:n` which will scale the integer part passed in '`#1`' and print it. Then, we inject the "word" that *joins* the integer part with the fraction handled by the key `join-word`.

Finally, we call the function `\__spintent_spmQ_print_spnZ_dfrac:nn` which is responsible for printing the fraction using the keys `print-dfrac` and `use-spnZ`, then executes the function `\__spintent_spmQ_-stop_wrap_parens:` handled by the key `wrap-parens` to close the parentheses if they are active.

```
\cs_new_protected:Nn \__spintent_spmQ_print:nnn
  {
    \__spintent_spmQ_start_wrap_parens:
    \__spintent_spmQ_print_sacle_int:n { #1 }
    \__spintent_spmQ_set_join_word:
    \__spintent_spmQ_print_spnZ_dfrac:nn { #2 } { #3 }
    \__spintent_spmQ_stop_wrap_parens:
  }
```

(*End of definition for* `\__spintent_spmQ_print:nnn`.)

©2026 by Pablo González L

__spintent_spmQ_exec:nnnn

The internal function __spintent_spmQ_exec:nnnn takes four arguments: the optional ⟨*keys*⟩ '#1', the integer '#2', the numerator '#3', and the denominator '#4'. First sets the optional ⟨*keys*⟩ from '#1', then, processes the denominator input in '#4' using __spintent_luafun_clean_split_and_set:n.

Now checks the status of \l__spintent_luaset_has_integer_str. If the status of this variable is true and the integer part is exactly zero, it issues the spmQ-zero-denominator "error". Otherwise, if the status is false or the integer part is *non-zero*, it calls the function __spintent_spmQ_print:nnn to format the *"mixed number"*.

```
\cs_new_protected:Nn \__spintent_spmQ_exec:nnnn
  {
    \keys_set:nn { spintent / spmQ } { #1 }
    \__spintent_luafun_clean_split_and_set:n { \exp_not:n { #4 } }
    \str_if_eq:VnTF \l__spintent_luaset_has_integer_str { true }
      {
        \int_compare:nNnTF { \l__spintent_luaset_part_int_tl } = { 0 }
          { \msg_error:nn { spintent } { spmQ-zero-denominator } }
          { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
      }
      { \__spintent_spmQ_print:nnn { #2 } { #3 } { #4 } }
  }
```

(*End of definition for* __spintent_spmQ_exec:nnnn.)

\spmQ

The command \spmQ is the public interface for formatting mixed numbers. It first checks if it is being used in math mode, issuing an error if not. Inside a local group, it calls __spintent_spmQ_exec:nnnn to process the optional ⟨*keys*⟩ and the {⟨*arguments*⟩}.

```
\NewDocumentCommand \spmQ { O{} m m m }
  {
    \mode_if_math:TF
      {
        \group_begin:
          \__spintent_spmQ_exec:nnnn { #1 } { #2 } { #3 } { #4 }
        \group_end:
      }
      { \msg_error:nnn { spintent } { need-math-mode } { \spmQ } }
  }
```

(*End of definition for* \spmQ. *This function is documented on page* 16.)

11.29 Finish package

Finish package implementation.

```
\file_input_stop:
⟨/package⟩
```

©2026 by Pablo González L

12 Índice de la Implementación

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

©2026 by Pablo González L

©2026 by Pablo González L

©2026 by Pablo González L

U

W

©2026 by Pablo González L